

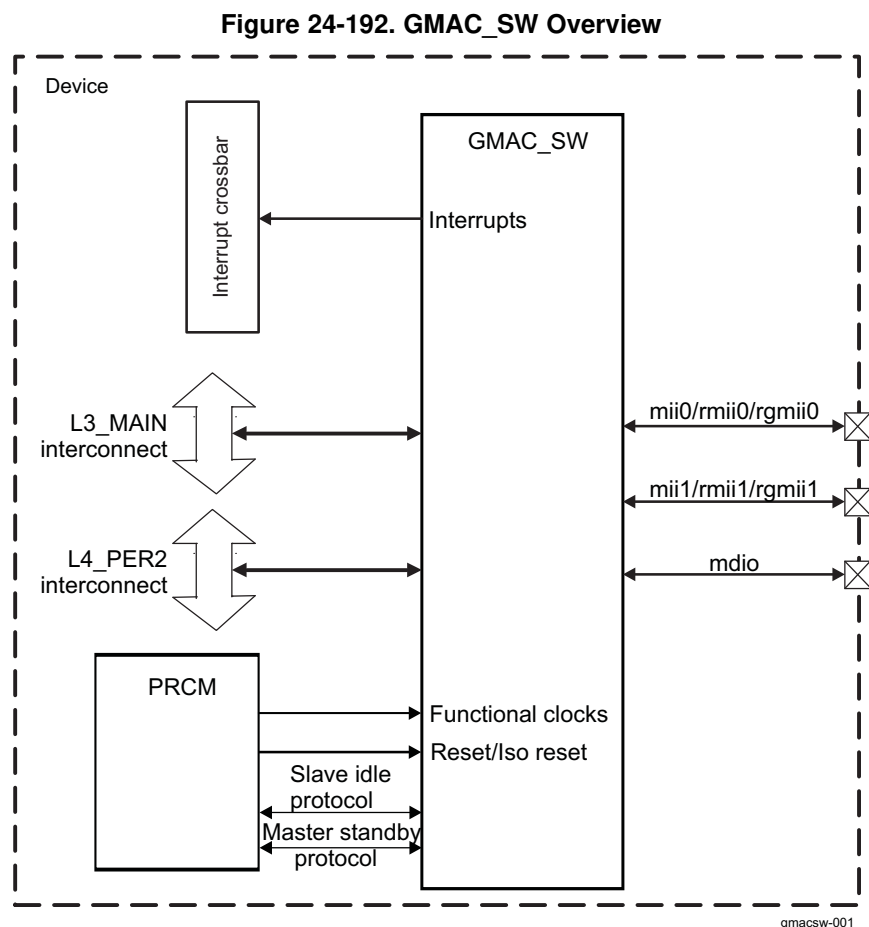
24.11 Gigabit Ethernet Switch (GMAC_SW)

This chapter describes the three-port gigabit switch ethernet subsystem in the device.

24.11.1 GMAC_SW Overview

The three-port gigabit ethernet switch subsystem (GMAC_SW) provides ethernet packet communication and can be configured as an ethernet switch. It provides the gigabit media independent interface (G/MII) in MII mode, reduced gigabit media independent interface (RGMII), reduced media independent interface (RMII), and the management data input output (MDIO) for physical layer device (PHY) management.

Figure 24-192 shows the GMAC_SW subsystem overview.



24.11.1.1 Features

The GMAC_SW subsystem provides the following features:

- Two Ethernet ports (port 1 and port 2) with selectable RGMII, RMII, and G/MII (in MII mode only) interfaces plus internal Communications Port Programming Interface (CPPI 3.1) on port 0
- Synchronous 10/100/1000 Mbit operation
- Wire rate switching (802.1d)
- Non-blocking switch fabric
- Flexible logical FIFO-based packet buffer structure
- Four priority level Quality Of Service (QOS) support (802.1p)
- CPPI 3.1 compliant DMA controllers

- Support for Audio/Video Bridging (P802.1Qav/D6.0)
- Support for IEEE 1588 Clock Synchronization (2008 Annex D and Annex F)
 - Timing FIFO and time stamping logic embedded in the subsystem
- Device Level Ring (DLR) Support
- Energy Efficient Ethernet (EEE) support (802.3az)
- Flow Control Support (802.3x)
- Address Lookup Engine (ALE)
 - 1024 total address entries plus VLANs
 - Wire rate lookup
 - Host controlled time-based aging
 - Multiple spanning tree support (spanning tree per VLAN)
 - L2 address lock and L2 filtering support
 - MAC authentication (802.1x)
 - Receive-based or destination-based multicast and broadcast rate limits
 - MAC address blocking
 - Source port locking
 - OUI (Vendor ID) host accept/deny feature
 - Remapping of priority level of VLAN or ports
- VLAN support
 - 802.1Q compliant
 - Auto add port VLAN for untagged frames on ingress
 - Auto VLAN removal on egress and auto pad to minimum frame size
- Ethernet Statistics:
 - EtherStats and 802.3Stats Remote network Monitoring (RMON) statistics gathering (shared)
 - Programmable statistics interrupt mask when a statistic is above one half its 32-bit value
- Flow Control Support (802.3x)
- Digital loopback and FIFO loopback modes supported
- Maximum frame size 2016 bytes (2020 with VLAN)
- 8k (2048 × 32) internal CPPI buffer descriptor memory
- Management Data Input/Output (MDIO) module for PHY Management
- Programmable interrupt control with selected interrupt pacing
- Emulation support
- Programmable Transmit Inter Packet Gap (IPG)
- Reset isolation (switch function remains active even in case of all device resets except for POR pin reset and ICEPICK cold reset)
- Full duplex mode supported in 10/100/1000 Mbps. Half-duplex mode supported only in 10/100 Mbps.
- IEEE 802.3 gigabit Ethernet conformant

NOTE: G/MII interface is functional in MII mode only.

Terminology:

AVB— Audio Video Bridging
AVBTP— Audio Video Bridging Transport Protocol
BMCA— Best Master Clock Algorithm
CFI— Canonical Format Indicator
CPPI— Communications Port Programming Interface
DLR— Device Level Ring
DSCP— Differentiated Services Code Point
EEE— Energy Efficient Ethernet
EMAC— Ethernet Media Access Control
EOP— End of Packet
EOQ— End of Queue
IPG— Inter-Packet Gap
LPI— Low Power Indicator
MDIO— Management Data Input/Output
MOF— Middle of Frame
PTP— Precision Time Protocol
RMON— Remote Monitoring
RTCP— RTP Control Protocol
RTP— Real-time Transport Protocol
SCR— Switched Central Resource
SRP— Stream Reservation Protocol
TOS— Type of Service
VLAN— Virtual Local Area Network

24.11.2 GMAC_SW Environment

24.11.2.1 G/MII Interface

G/MII Interface can operate only in MII Mode.

- In MII Mode (100/10 Mbps): GMAC_SW operates in full-duplex and half-duplex.

The pin connections of the G/MII Interface is shown in Figure 24-193. The detailed description of the signals are listed in the following tables.

Figure 24-193. MII Interface Typical Application

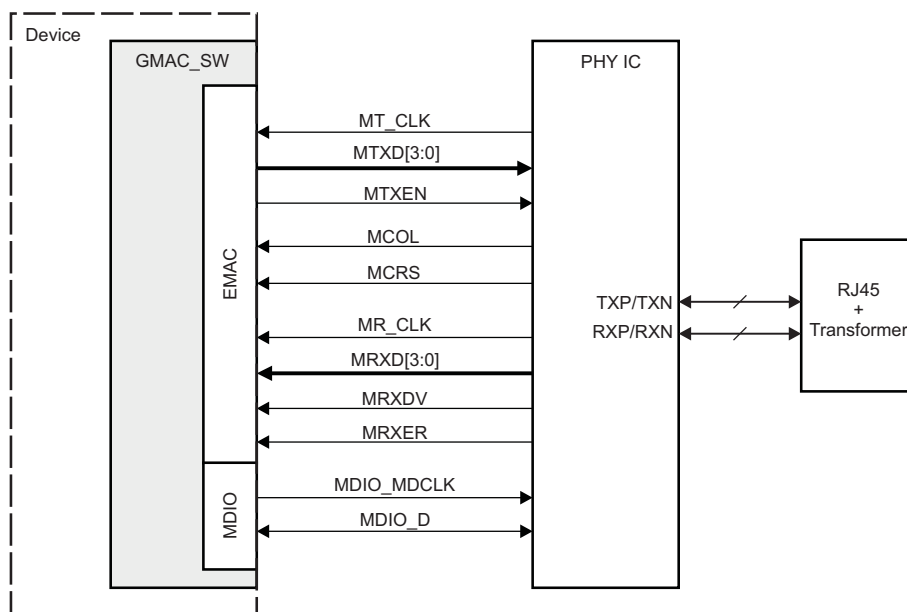


Table 24-1250. MII I/O Description

Signal	Device Pin(s)	I/O ⁽¹⁾	Description
MT_CLK	mii0_txclk mii1_txclk	I	The transmit clock is a continuous clock that provides the timing reference for transmit operations. The MTXD and MTXEN signals are tied to this clock. The clock is generated by the PHY and is 2.5 MHz at 10 Mbps operation and 25 MHz at 100 Mbps operation.
MTXD[3:0]	mii0_txd[3:0] mii1_txd[3:0]	O	The transmit data pins are a collection of 4 data signals MTXD[3:0] comprising 4 bits of data. MTXD0 is the least-significant bit (LSB). The signals are synchronized by MT_CLK and valid only when MTXEN is asserted.
MTXEN	mii0_txen mii1_txen	O	The transmit enable signal indicates that the MTXD pins are generating 4bit data for use by the PHY. It is driven synchronously by MT_CLK.
MTXER	mii0_txer mii1_txer	O	Transmit data error. Used only for EEE to request PHY for low power transition.
MCOL	mii0_col mii1_col	I	In half-duplex operation, the MCOL pin is asserted by the PHY when it detects a collision on the network. It remains asserted while the collision condition persists. This signal is not necessarily synchronous to MT_CLK nor MR_CLK. In full-duplex operation, the MCOL pin is used for hardware transmit flow control. Asserting the MCOL pin will stop packet transmissions; packets in the process of being transmitted when MCOL is asserted will complete transmission. The MCOL pin should be held low if hardware transmit flow control is not used.

⁽¹⁾ I = Input; O = Output

Table 24-1250. MII I/O Description (continued)

Signal	Device Pin(s)	I/O ⁽¹⁾	Description
MCRS	mii0_crs mii1_crs	I	In half-duplex operation, the MCRS pin is asserted by the PHY when the network is not idle in either transmit or receive. The pin is de-asserted when both transmit and receive are idle. This signal is not necessarily synchronous to MT_CLK nor MR_CLK. In full-duplex operation, the MCRS pin should be held low.
MR_CLK	mii0_rxclk mii1_rxclk	I	The receive clock is a continuous clock that provides the timing reference for receive operations. The MRXD, MRXDV, and MRXER signals are tied to this clock. The clock is generated by the PHY and is 2.5 MHz at 10 Mbps operation and 25 MHz at 100 Mbps operation.
MRXD	mii0_rxd[0:3] mii1_rxd[0:3]	I	The receive data pins are a collection of 4 data signals comprising 4 bits of data. MRXD0 is the least-significant bit (LSB). The signals are synchronized by MR_CLK and valid only when MRXDV is asserted.
MRXDV	mii0_rxdv mii1_rxdv	I	The receive data valid signal indicates that the MRXD pins are generating nibble data for use by the GMAC_SW. It is driven synchronously to MR_CLK.
MRXER	mii0_rxer mii1_rxer	I	Receive Data Error input
MDIO_MDCLK	mdio_mclk	O	Management data clock. The MDIO data clock is sourced by the MDIO module on the system. It is used to synchronize MDIO data access operations done on the MDIO_D pin.
MDIO_D	mdio_d	I/O	MDIO data pin drives PHY management data into and out of the PHY by way of an access frame consisting of start of frame, read/write indication, PHY address, register address, and data bit cycles. The MDIO_D pin acts as an output for all but the data bit cycles at which time it is an input for read operations.

24.11.2.2 RMII Interface

Figure 24-194 shows a device with integrated GMAC_SW and MDIO interfaced via a RMII connection in a typical system. The individual CPSW and MDIO signals for the RMII interface are summarized in Table 24-1251.

For more information, refer to either the IEEE 802.3 standard or ISO/IEC 8802-3:2000(E).

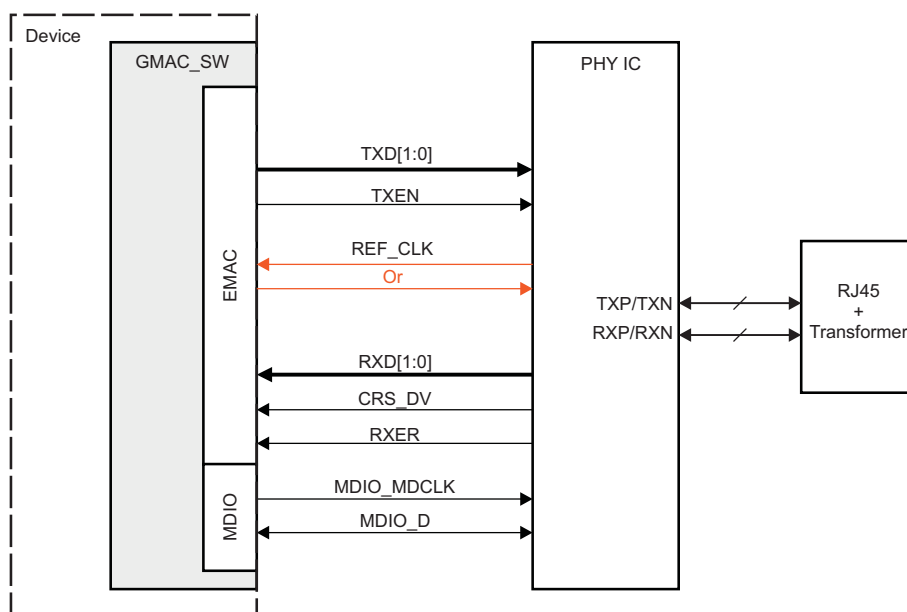
Figure 24-194. RMII Interface Typical Application


Table 24-1251. RMII I/O Description

Signal	Device Pin(s)	I/O ⁽¹⁾	Description
TXD[1:0]	rmii0_txd[1:0] rmii1_txd[1:0]	O	Transmit data . The transmit data pins are a collection of 2 bits of data. TXD0 is the least-significant bit (LSB). The signals are synchronized by RMII_MHZ_50_CLK and valid only when TXEN is asserted.
TXEN	rmii0_txen rmii1_txen	O	Transmit enable. The transmit enable signal indicates that the RMII_TXD pins are generating data for use by the PHY. RMII_TXEN is synchronous to RMII_MHZ_50_CLK
RMII0_MHZ_50_CLK K	RMII_MHZ_50_CLK	I/O	RMII reference clock.
RMII1_MHZ_50_CLK K			The reference clock is used to synchronize all RMII signals. RMII_MHZ_50_CLK must be continuous and fixed at 50 MHz.
RXD[1:0]	rmii0_rxd[1:0] rmii1_rxd[1:0]	I	Receive data. The receive data pins are a collection of 2 bits of data. RXD0 is the least-significant bit (LSB). The signals are synchronized by RMII_MHZ_50_CLK and valid only when CRS_DV is asserted and RXER is de-asserted.
CRS_DV	rmii0_crs rmii1_crs	I	Carrier sense/receive data valid. Multiplexed signal between carrier sense and receive data valid.
RXER	rmii0_rxer rmii1_rxer	I	Receive error. The receive error signal is asserted to indicate that an error was detected in the received frame.
MDIO_MDCLK	mdio_mclk	O	Management data clock (MDIO_MCLK). The MDIO data clock is sourced by the MDIO module on the system. It is used to synchronize MDIO data access operations done on the MDIO_D pin.
MDIO_D	mdio_d	I/O	MDIO data pin drives PHY management data into and out of the PHY by way of an access frame consisting of start of frame, read/write indication, PHY address, register address, and data bit cycles. The MDIO_D pin acts as an output for all but the data bit cycles at which time it is an input for read operations.

⁽¹⁾ I = Input; O = Output

24.11.2.3 RGMII Interface

Figure 24-195 shows a device with integrated CPSW and MDIO interfaced via a RGMII connection in a typical system. The individual CPSW and MDIO signals for the RGMII interface are summarized in Table 24-1252.

Figure 24-195. RGMII Interface Typical Application

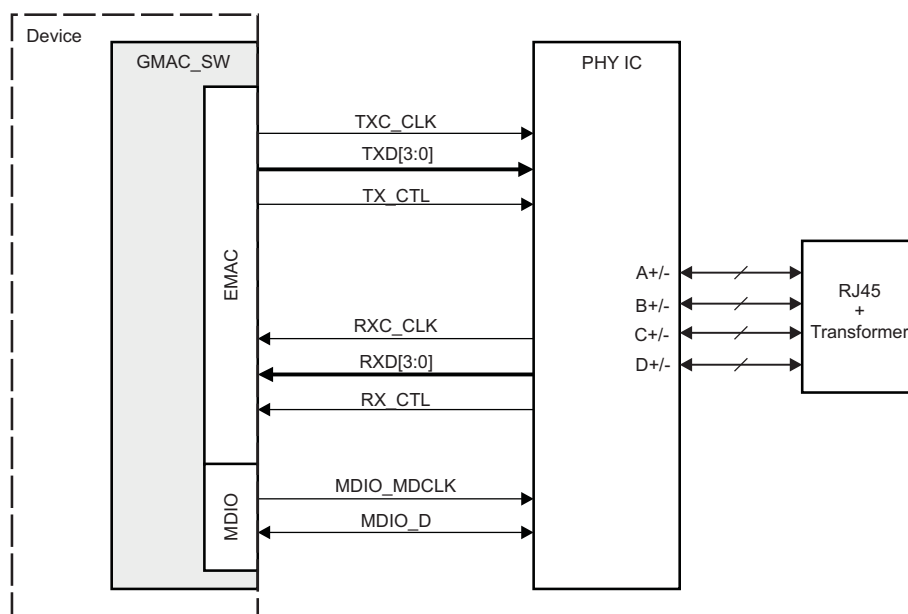


Table 24-1252. RGMII I/O Description

Signal	Device Pin(s)	I/O ⁽¹⁾	Description
TXD[3:0]	rgmii0_txd[3:0] rgmii1_txd[3:0]	O	The transmit data pins are a collection of 4 bits of data. TXD0 is the least-significant bit (LSB). The signals are valid only when TX_CTL is asserted.
TX_CTL	rgmii0_rxctl rgmii1_rxctl	O	Transmit Control/enable. The transmit enable signal indicates that the TXD pins are generating data for use by the PHY.
TXC_CLK	rgmii0_txc rgmii1_txc	O	The transmit reference clock. The clock is 2.5 MHz at 10 Mbps operation, 25 MHz at 100 Mbps operation, and 125 MHz at 1000 Mbps of operation.
RXD[3:0]	rgmii0_rxd[3:0] rgmii1_rxd[3:0]	I	The receive data pins are a collection of 4 bits of data. RXD0 is the least-significant bit (LSB). The signals are valid only when RX_CTL is asserted
RX_CTL	rgmii0_rxctl rgmii1_rxctl	I	The receive data valid/control signal indicates that the RXD pins are nibble data for use by the GMAC_SW.
RXC_CLK	rgmii0_rxc rgmii1_rxc	I	The receive clock is a continuous clock that provides the timing reference for receive operations. The clock is generated by the PHY and is 2.5 MHz at 10 Mbps operation, 25 MHz at 100 Mbps operation, 125 MHz at 1000 Mbps of operation.
MDIO_MDCLK	mdio_mclk	O	Management data clock (MDIO_MCLK). The MDIO data clock is sourced by the MDIO module on the system. It is used to synchronize MDIO data access operations done on the MDIO pin.
MDIO_D	mdio_d	I/O	MDIO data pin drives PHY management data into and out of the PHY by way of an access frame consisting of start of frame, read/write indication, PHY address, register address, and data bit cycles. The MDIO_D pin acts as an output for all but the data bit cycles at which time it is an input for read operations.

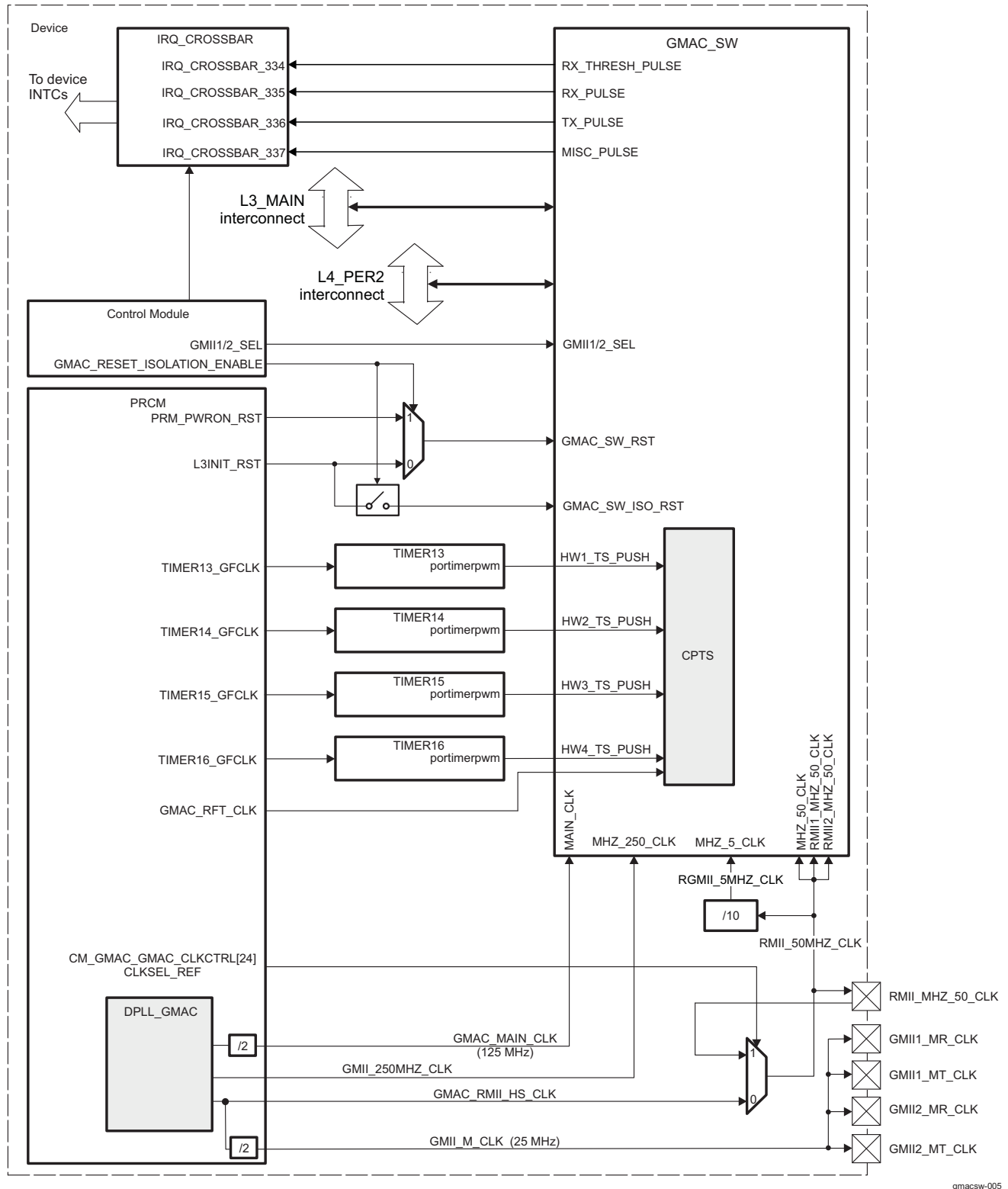
⁽¹⁾ I = Input; O = Output

NOTE: The path from a module pin to device pad(s) is defined at the device I/O logic level. The control module registers assign the specific function to the device pads. For more information on control module settings, see [Section 18.4.6.1.1, Pad Configuration Registers](#) in [Chapter 18, Control Module](#).

24.11.3 GMAC_SW Integration

Figure 24-196 shows the integration of the GMAC_SW module in the device.

Figure 24-196. GMAC_SW Integration



gmactsw-005

NOTE: The device provides the option to receive or output the RMII reference clock (RMII_50MHZ_CLK) from/to external pin as shown in [Figure 24-196](#). See the RMII_CLK_SETTING register bit in the [Section 18.5 Control Module Register Manual](#), and CLKSEL_REF in [Section 3.12, PRCM Register Manual](#) to configure the clocking option of RMII.

See [Chapter 3, PRCM](#) for details about power, clock and reset options for GMAC_SW.

[Table 24-1253](#) through [Table 24-1255](#) summarize the integration of the GMAC_SW module in the device.

Table 24-1253. GMAC_SW Integration Attributes

Module Instance	Attributes		
	Power Domain	Wake-Up Capability	Interconnect
GMAC_SW	PD_COREAON	Yes	L3_MAIN L4_PER2

Table 24-1254. GMAC_SW Clocks and Resets

Clocks				
Module Instance	Destination Signal Name	Source Signal Name	Source	Description
GMAC_SW	MAIN_CLK	GMAC_MAIN_CLK	PRCM	Interface clock for the GMAC_SW module (125 MHz)
	MHZ_5_CLK	RGMII_5MHZ_CLK	PRCM	5-MHz RGMII clock
	MHZ_50_CLK	RMII_50MHZ_CLK	PRCM or RMII_MHZ_50_CLK pin	50-MHz RGMII clock
	MHZ_250_CLK	GMII_250MHZ_CLK	PRCM	250-MHz RGMII clock
	RMII1_MHZ_50_CLK	RMII_50MHZ_CLK	PRCM or RMII_MHZ_50_CLK pin	50-MHz RMII1 clock
	RMII2_MHZ_50_CLK	RMII_50MHZ_CLK	PRCM or RMII_MHZ_50_CLK pin	50-MHz RMII2 clock
	CPTS_RFT_CLK	GMAC_RFT_CLK	PRCM	IEEE 1588 clock
	GMII1_MR_CLK	GMII_M_CLK	PRCM or the external pin	MII1 receive clock
	GMII1_MT_CLK	GMII_M_CLK	PRCM or the external pin	MII1 transmit clock
	GMII2_MR_CLK	GMII_M_CLK	PRCM or the external pin	MII2 receive clock
	GMII2_MT_CLK	GMII_M_CLK	PRCM or the external pin	MII2 transmit clock
Resets				
Module Instance	Destination Signal Name	Source Signal Name	Source	Description
GMAC_SW	GMAC_SW_RST	L3INIT_RST/PRM_PWR_ON_RST	PRCM	GMAC_SW main reset
	GMAC_SW_ISO_RST	tied off/L3INIT_RST	PRCM	GMAC_SW isolate reset (must be enabled in Control module)

Table 24-1255. GMAC_SW Hardware Requests

Interrupt Requests				
Module Instance	Source Signal Name	IRQ_CROSSBAR Input	Default Mapping	Description
GMAC_SW	RX_THRESH_PULSE	IRQ_CROSSBAR_334	-	Receive threshold interrupt

Table 24-1255. GMAC_SW Hardware Requests (continued)

RX_PULSE	IRQ_CROSSBAR_335	-	Receive packet completion interrupt
TX_PULSE	IRQ_CROSSBAR_336	-	Transmit packet completion interrupt
MISC_PULSE	IRQ_CROSSBAR_337	-	Miscellaneous interrupt (SPF2_PEND, SPF1_PEND, EVNT_PEND, STAT_PEND, HOST_PEND, MDIO_LINKINT, MDIO_USERINT)

NOTE: GMAC_SW has no default IRQ mappings through the IRQ_CROSSBAR. For GMAC_SW, the IRQ_CROSSBAR module must be configured prior to unmask interrupts in the interrupt controller(s).
For more information about the IRQ_CROSSBAR module, see [Section 18.4.6.4, IRQ_CROSSBAR Module Functional Description](#), in [Chapter 18, Control Module](#).
For more information about the device interrupt controllers, see [Chapter 17, Interrupt Controllers](#).

NOTE: For the description of the interrupt source, see [Section 24.11.4.5, Interrupt Functionality](#).

24.11.4 GMAC_SW Functional Description

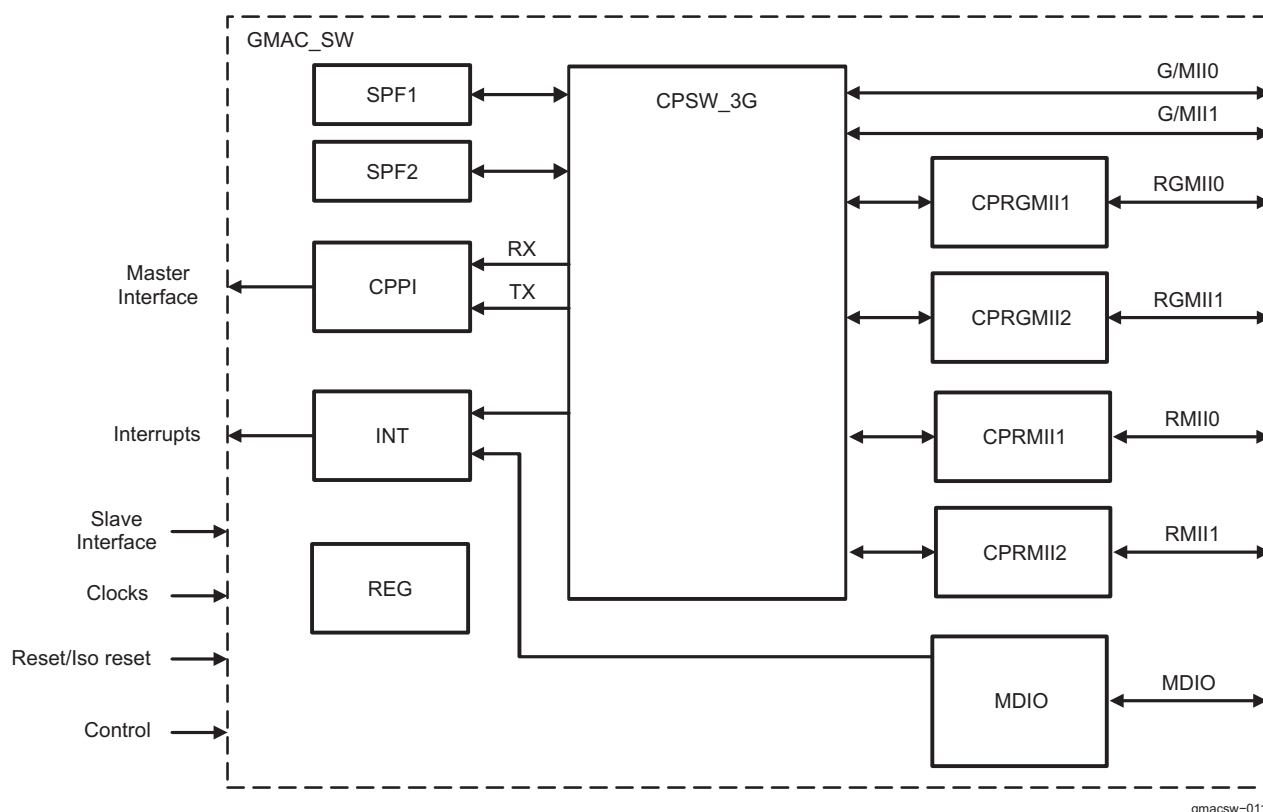
The 3-port switch (GMAC_SW) Ethernet subsystem modules are compliant to the IEEE Std 802.3 Specification. GMAC_SW is shown in [Figure 24-197](#).

24.11.4.1 Functional Block Diagram

The GMAC_SW subsystem consists of:

- CPSW_3G which contains two G/MII interfaces
- Two RGMII interface modules
- Two RMII interface modules
- One MDIO interface module
- One Interrupt Controller module
- One CPPI interface
- Local CPPI memory of size 8 KiB
- Two Static Packet Filters

Figure 24-197. GMAC_SW Top Level Block Diagram



24.11.4.2 GMAC_SW Ports

Ethernet Subsystem has three Ports. Port 0 is the Host port (internal to the Subsystem). Ports 1 and 2 are the external ports connected to G/MII, RGMII, or RMII interfaces as per the interface selected.

Naming conventions followed in this chapter:

- Port0 is referred to the Host Port
- Port1 is referred to the interfaces GMII0/RGMII0/RMII0
- Port2 is referred to the interfaces GMII1/RGMII1/RMII1

24.11.4.2.1 Interface Mode Selection

The 3-port switch (GMAC_SW) Ethernet Subsystem has two 10/100/1000 Ethernet ports with selectable MII, RMII, and RGMII interfaces.

The interface mode is selected by configuring the MII mode selection register bitfields (GMII1_SEL and GMII2_SEL) in the control module. See [Section 18.5, Control Module Register Manual](#) for details.

See device data manual for configuring the pin mux mode as per the interface selected.

24.11.4.3 Clocking

24.11.4.3.1 Subsystem Clocking

GMAC_SW clocking summary is shown in [Section 24.11.3, GMAC_SW Integration](#).

24.11.4.3.2 Interface Clocking

Data is transmitted and received with respect to the reference clocks of the interface pins.

24.11.4.3.2.1 G/MII Interface Clocking

GMII1_MR_CLK, GMII1_MT_CLK, GMII2_MR_CLK, GMII2_MT_CLK frequencies are fixed by the 802.3 specification.

- 2.5 MHz at 10 Mbps
- 25 MHz at 100 Mbps

24.11.4.3.2.2 RGMII Interface Clocking

RGMII_RXC, RGMII_TXC frequencies are:

- 2.5 MHz at 10 Mbps
- 25 MHz at 100 Mbps
- 125 MHz at 1000 Mbps

24.11.4.3.2.3 RMII Interface Clocking

RMII interface clock RMII_50MHZ_CLK frequency is:

- 50 MHz at 10 Mbps
- 50 MHz at 100 Mbps

See the RMII_CLK_SETTING bit in the device Control Module and CLKSEL_REF in PRCM to configure the clocking option of RMII (external pin or PRCM).

24.11.4.3.2.4 MDIO Clocking

The MDIO clock is based on a divide-down of the interface (MAIN_CLK) clock, running at 125 MHz. The application software or driver must control the divide-down value.

See the [MDIO_CONTROL](#) register for configuring the Clock Divider (CLKDIV) value.

24.11.4.4 Software IDLE

The submodule software idle register bits enable CPSW_3G operation to be completely or partially suspended by software control. There are three CPSW_3G submodules that contain software idle register bits (CPGMAC_SL1, CPGMAC_SL2, and CPDMA). Each of the three submodules may be individually commanded to enter the idle state. The idle state is entered at packet boundaries, and no further packet operations will occur on an idled submodule until the idle command is removed. The CPSW_3G module

enters the idle state when all three submodules are commanded to enter and have entered the idle state. Idle status is determined by reading or polling the three submodule idle bits. The CPSW_3G is in the idle state when all three submodules are in the idle state. The [CPSW_SOFT_IDLE\[0\]](#) SOFT_IDLE bit may be set if desired after the submodules are in the idle state. The CPSW_SOFT_IDLE bit causes packets to not be transferred from one FIFO to another FIFO internal to the switch.

24.11.4.5 Interrupt Functionality

GMAC_SW Ethernet Subsystem has four Interrupt outputs:

- RX_PULSE - Receive Interrupt
- TX_PULSE - Transmit Interrupt
- RX_THRESH_PULSE - Receive Threshold Interrupt
- MISC_PULSE - Miscellaneous Interrupt.

24.11.4.5.1 Receive Packet Completion Pulse Interrupt (RX_PULSE)

The RX_PULSE interrupt is a pulse interrupt selected from the GMAC_SW RX_PEND[7:0] interrupts. The receive DMA controller has eight channels with each channel having a corresponding (RX_PEND[7:0]).

The following steps will enable the receive packet completion interrupt:

1. Enable the required channel interrupts of the DMA engine by setting 1 to the appropriate bit in the [CPDMA_RX_INTMASK_SET](#) register.
2. The receive completion interrupt(s) to be routed to RX_PULSE is selected by setting one or more bits in the receive interrupt enable register ([WR_C0_RX_EN](#)). The masked interrupt status can be read in the address location of RX_STAT bit in the [WR_C0_RX_STAT](#) register.

When the GMAC_SW completes a packet reception, the subsystem issues an interrupt to the host processor by writing the packet's last buffer descriptor address to the appropriate channel queue's receive completion pointer located in the state RAM block. The interrupt is generated by the write when enabled by the interrupt mask, regardless of the value written.

Upon interrupt reception, the software processes one or more packets from the buffer chain and then acknowledges one or more interrupt(s) by writing the address of the last buffer descriptor processed to the queue's associated receive completion pointer (RX_n_CP) in the receive DMA state RAM.

Upon reception of an interrupt, software should perform the following:

1. Read the [WR_C0_RX_STAT\[7:0\]](#) RX_STAT bit address location to determine which channel(s) caused the interrupt.
2. Process received packets for the interrupting channel(s).
3. Write the GMAC_SW completion pointer(s) (RX_n_CP). The data written by the host (buffer descriptor address of the last processed buffer) is compared to the data in the register written by the subsystem (address of last buffer descriptor used by the subsystem). If the two values are not equal (which means that the GMAC_SW has received more packets than the software has processed), the receive packet completion interrupt signal remains asserted. If the two values are equal (which means that the host has processed all packets that the system has received), the pending interrupt is de-asserted. The value that the GMAC_SW is expecting is found by reading the receive channel *n* completion pointer register (RX_n_CP).
4. Write the value 1h to the [CPDMA_EOI_VECTOR](#) register.

To disable the interrupt:

1. The eight channel interrupts may be individually disabled by writing to 1 the appropriate bit in the [CPDMA_RX_INTMASK_CLEAR](#) register.
2. The receive completion pulse interrupt could be disabled by clearing to 0 all the bits in the [WR_C0_RX_EN](#) register.

The software could still poll for the [CPDMA_RX_INTSTAT_RAW](#) and [CPDMA_RX_INTSTAT_MASKED](#) registers if the corresponding interrupts are enabled.

24.11.4.5.2 Transmit Packet Completion Pulse Interrupt (TX_PULSE)

The TX_PULSE interrupt is a pulse interrupt selected from the GMAC_SW TX_PEND[7:0] interrupts. The transmit DMA controller has eight channels with each channel having a corresponding (TX_PEND[7:0]).

To enable the transmit packet completion interrupt:

1. Enable the required channel interrupts of the DMA engine by setting 1 to the appropriate bit in the [CPDMA_TX_INTMASK_SET](#) register.
2. The transmit completion interrupt(s) to be routed to TX_PULSE is selected by setting one or more bits in the transmit interrupt enable register [WR_CO_TX_EN](#). The masked interrupt status can be read in the address location of TX_STAT bit in the [WR_CO_TX_STAT](#) register.

When the GMAC_SW completes the transmission of a packet, the GMAC_SW subsystem issues an interrupt to the host processor by writing the packet's last buffer descriptor address to the appropriate channel queue's transmit completion pointer located in the state RAM block. The interrupt is generated by the write when enabled by the interrupt mask, regardless of the value written.

Upon reception of an interrupt, software should perform the following:

1. Read the TX_STAT bit address location to determine which channel(s) caused the interrupt
2. Process received packets for the interrupting channel(s).
3. Write the GMAC_SW completion pointer(s) (TX *n*_CP). The data written by the host (buffer descriptor address of the last processed buffer) is compared to the data in the register written by the GMAC_SW (address of last buffer descriptor used by the GMAC_SW). If the two values are not equal (which means that the GMAC_SW has transmitted more packets than the software has processed), the transmit packet completion interrupt signal remains asserted. If the two values are equal (which means that the host has processed all packets that the subsystem has transferred), the pending interrupt is cleared. The value that the GMAC_SW is expecting is found by reading the transmit channel *n*completion pointer register (TX *n*_CP).
4. Write the 2h to the [CPDMA_EOI_VECTOR](#) register.

To disable the interrupt:

1. The eight channel interrupts may be individually disabled by writing to 1 the appropriate bit in the [CPDMA_TX_INTMASK_CLEAR](#) register.
2. The receive completion pulse interrupt could be disabled by clearing to 0 all the bits in the [WR_CO_TX_EN](#) register.

The software could still poll for the [CPDMA_TX_INTSTAT_RAW](#) and [CPDMA_TX_INTSTAT_MASKED](#) registers, if the corresponding interrupts are enabled.

24.11.4.5.3 Receive Threshold Pulse Interrupt (RX_THRESH_PULSE)

The RX_THRESH_PULSE interrupt is an immediate (non-paced) pulse interrupt selected from the CPSW_3G RX_THRESH_PEND[7:0] interrupts. The receive DMA controller has eight channels with each channel having a corresponding threshold pulse interrupt (RX_THRESH_PEND[7:0]).

To enable the receive threshold pulse Interrupt:

1. Enable the required channel interrupts of the DMA engine by setting 1 to the appropriate bit in the [CPDMA_RX_INTMASK_SET](#) register.
2. The receive threshold interrupt(s) to be routed to RX_THRESH_PULSE is selected by setting one or more bits in the receive threshold interrupt enable register [WR_CO_RX_THRESH_EN](#). The masked interrupt status can be read in the address location of RX_THRESH_STAT bit in the [WR_CO_RX_THRESH_STAT](#) register.

The RX_THRESH_PULSE is asserted when enabled when the channel's associated free buffer count RX *n*_FREEBUFFER is less than or equal to the corresponding RX *n*_PENDTHRESH register.

Upon reception of an interrupt, software should perform the following:

1. Read the RX_THRESH_STAT bit address location to determine which channel(s) caused the interrupt.
2. Process the received packets in order to add more buffers to any channel that is below the threshold value.

3. Write the CPSW_3G completion pointer(s).
4. Write the value 0h to the [CPDMA_EOI_VECTOR](#) register.

The threshold pulse interrupt is an immediate interrupt intended to indicate that software should immediately process packets to preclude an overrun condition from occurring for the particular channels.

To disable the interrupt:

1. The eight channel receive threshold interrupts may be individually disabled by writing to 1 the appropriate bit in the [CPDMA_RX_INTMASK_CLEAR](#) register.
2. The receive threshold pulse interrupt could be disabled by clearing to 0 the corresponding bits in the [WR_C0_RX_THRESH_EN](#) register.

The software could still poll for the [CPDMA_RX_INTSTAT_RAW](#) and [CPDMA_RX_INTSTAT_MASKED](#) registers, if the corresponding interrupts are enabled.

24.11.4.5.4 Miscellaneous Pulse Interrupt (MISC_PULSE)

The MISC_PULSE interrupt is an immediate pulse interrupt selected from the miscellaneous interrupts (SPF2_PEND, SPF1_PEND, EVNT_PEND, STAT_PEND, HOST_PEND, MDIO_LINKINT, MDIO_USERINT).

The miscellaneous interrupt(s) is selected by setting one or more bits in the miscellaneous interrupt enable register ([WR_C0_MISC_EN](#)).

Upon reception of an interrupt, software should perform the following:

- Read the MISC_STAT bit address location to determine the source of the interrupt.
- Process the interrupt.
- Write the value 3h to the [CPDMA_EOI_VECTOR](#) register.

NOTE: The [WR_C0_MISC_STAT](#) register's MDIO_LINKINT and MDIO_USERINT bitfields represent the Port0/Phy0 status. MDIO_LINKINT[1] and MDIO_USERINT[1] are not provided in [WR_C0_MISC_STAT](#) register. As such, MDIO Link and User interrupts can only be generated for Port0. For Port1, software must poll the status, visible in the [MDIO_LINKINTMASKED](#) or [MDIO_USERINTMASKED](#) registers.

24.11.4.5.4.1 EVNT_PEND(CPTS_PEND) Interrupt

See [Section 24.11.4.10](#), *Common Platform Time Sync (CPTS)* for more details on this interrupt.

24.11.4.5.4.2 Statistics Interrupt

The statistics level interrupt (STAT_PEND) will be asserted, if enabled when any statistics value is greater than or equal to 8000 0000h. The statistics interrupt is cleared by writing to decrement all statistics values greater than 8000 0000h (such that their new values are less than 8000 0000h). The raw and masked statistics interrupt status may be read by reading the [CPDMA_TX_INTSTAT_RAW](#) and [CPDMA_TX_INTSTAT_MASKED](#) registers, respectively.

The Statistics interrupt is enabled by setting to 1 the STAT_INT_MASK bit in the [CPDMA_DMA_INTMASK_SET](#) register

24.11.4.5.4.3 Host Error interrupt

The host error interrupt (HOST_PEND) will be asserted, if enabled when a host error is detected during transmit or receive CPDMA transactions. The host error interrupt is intended for software debug, and is cleared by a warm reset or a system reset. The raw and masked host interrupt status can be read by reading the [CPDMA_DMA_INTSTAT_RAW](#) and [CPDMA_DMA_INTSTAT_MASKED](#) registers, respectively.

The transmit host error conditions are:

- SOP error

- OWNERSHIP bit not set in SOP buffer
- next buffer descriptor pointer without EOP cleared to 0
- buffer pointer cleared to 0
- buffer length cleared to 0
- packet length error

The receive host error conditions are:

- OWNERSHIP bit not set in input buffer
- Zero buffer pointer
- Zero buffer Length on non-SOP descriptor
- SOP buffer length not greater than offset

The HOST_PEND is enabled by setting to 1 the HOST_ERR_INTMASK in the [CPDMA_DMA_INTMASK_SET](#) register. The host error interrupt is disabled by setting to 1 the appropriate bit in the [CPDMA_DMA_INTMASK_CLEAR](#) register.

24.11.4.5.4.4 MDIO Interrupts

[MDIO_LINKINT](#) is set if there is a change in the link state of the PHY corresponding to the address in the PHYADR_MON field of the MDIO_USERPHYSEL_n register and the corresponding LINKINT_ENABLE bit is set. The [MDIO_LINKINT](#) event is also captured in the [MDIO_LINKINTMASKED](#) register. When the GO bit in the MDIO_USERACCESS_n register transitions from 1 to 0, indicating the completion of a user access, and the corresponding USERINTMASKSET bit in the [MDIO_USERINTMASKSET](#) register is set, the MDIO_USERINT signal is asserted. The MDIO_USERINT event is also captured in the [MDIO_USERINTMASKED](#) register.

24.11.4.5.5 Interrupt Pacing

RX_PULSE and TX_PULSE interrupts can be paced. The RX_THRESH_PULSE and MISC_PULSE interrupts are not paced. The Interrupt pacing feature limits the number of interrupts that occur during a given period of time. For heavily loaded systems in which interrupts can occur at a very high rate (for example, 148,800 packets per second for Ethernet), the performance benefit is significant due to minimizing the overhead associated with servicing each interrupt. Interrupt pacing increases the processor cache hit ratio by minimizing the number of times that large interrupt service routines are moved to and from the processor instruction cache.

Each RX_PULSE and TX_PULSE interrupt contains an interrupt pacing sub-block (six total). Each sub-block is disabled by default allowing the selected interrupt inputs to pass through unaffected. The interrupt pacing module counts the number of interrupts that occur over a 1 ms interval of time. At the end of each 1 ms interval, the current number of interrupts is compared with a target number of interrupts (specified by the associated maximum number of interrupts register). Based on the results of the comparison, the length of time during which interrupts are blocked is dynamically adjusted. The 1 ms interval is derived from a 4 μ s pulse that is created from a prescale counter whose value is set in the INT_PRESCALE field in the [WR_INT_CONTROL](#) register. The INT_PRESCALE value should be written with the number of ICLK periods in 4 μ s. The pacing timer determines the interval during which interrupts are blocked and decrements every 4 μ s. It is reloaded each time a zero count is reached. The value loaded into the pacing timer is calculated by hardware every 1 ms according to the following algorithm:

```
if (intr_count > 2*intr_max)
    pace_timer = 255;
else if (intr_count > 1.5*intr_max)
    pace_timer = last_pace_timer*2 + 1;
else if (intr_count > 1.0*intr_max)
    pace_timer = last_pace_timer + 1;
else if (intr_count > 0.5*intr_max)
    pace_timer = last_pace_timer - 1;
else if (intr_count != 0)
    pace_timer = last_pace_timer/2;
else
    pace_timer = 0;
```


If the rate of interrupt inputs is much less than the target interrupt rate specified in the associated maximum interrupts register, then the interrupt is not blocked. If the interrupt rate is greater than the target rate, the interrupt will be "paced" at the rate specified in the interrupt maximum register. The [WR_CO_RX_IMAX/WR_CO_TX_IMAX](#) register should be written with a value between 2 and 63 inclusive, indicating the target number of interrupts per millisecond.

24.11.4.6 Reset Isolation

Reset isolation for the GMAC_SW allows the switch function to remain active in during all device resets except for POR pin reset and ICEPICK COLD reset. Packet traffic to/from the GMAC_SW host will be flushed/dropped, but the ethernet switch will remain operational for all traffic between external devices on the switch even though the device is undergoing a device reset. Pin mux configuration for ethernet related I/O and reference clocks needed by the GMAC_SW to be active is controlled by a protected control module bit.

24.11.4.6.1 Reset Isolation Functional Description

The device has two modes of operation concerning the reset of the GMAC_SW Ethernet switch. The mode is controlled by the GMAC_RESET_ISOLATION_ENABLE bit in the Control Module. This bit defaults to 0. Any modification of this bit first requires writing an unlock pattern to lock register in device control module. After modification, the bit should again be locked by writing appropriate value to the lock register. Writes to the GMAC_RESET_ISOLATION_ENABLE bit and to the lock register must all be supervisor mode writes.

GMAC_RESET_ISOLATION_ENABLE = 0 (disabled)

1. This is the default state of the bit after control module reset.
2. Upon any device level resets, the entire GMAC_SW, DPLL_GMAC, L3/L4 interconnect, control module (including all pin mux control and the GMAC_RESET_ISOLATION_ENABLE bit itself) are immediately reset.

GMAC_RESET_ISOLATION_ENABLE = 1 (enabled)

1. This mode is selected when the GMAC_RESET_ISOLATION_ENABLE bit is set to 1 by software.
2. Upon any device reset source other than porz pin or ICEPICK cold (that is, this includes software global cold, any watchdog reset, warm resetn pin, ICEPICK warm, software global warm or security violation), the following is true:
 1. The CPSW_3GSS_R is put into "isolate" mode and non-switch related portions of the subsystem are reset.
 2. The 50 MHz and 125 MHz reference clocks to the GMAC_SW Ethernet Subsystem remain active throughout the entire reset condition.
 3. The control for pin multiplexing for all of the signals maintain their current configuration throughout the entire reset condition.
 4. The reset-isolated logic inside GMAC_SW Ethernet Subsystem maintains the switch functionality
3. Upon any cold reset sources, the entire GMAC_SW Ethernet Subsystem, DPLL_GMAC, control module (including all pin mux control and the GMAC_RESET_ISOLATION_ENABLE bit itself) are reset.

For more details on the register configuration, see the Control Module CTRL_CORE_CONTROL_IO_2 register in [Section 18.5, Control Module Register Manual](#).

24.11.4.7 Software Reset

The CPSW_3G software reset register ([CPSW_SOFT_RESET](#)), CPSW_3GSS software reset register ([WR_SOFT_RESET](#)) and the three submodule software reset registers enable the CPSW_3GSS to be reset by software.

There are three CPSW_3G submodules that contain software reset registers (CPGMAC_SL1, CPGMAC_SL2 ([SL_SOFT_RESET](#)), and CPDMA ([CPDMA_SOFT_RESET](#))). Each of the three submodules may be individually commanded to be reset by software.

For the CPDMA, the reset state is entered at packet boundaries, at which time the CPDMA reset occurs. The CPGMAC_SL soft reset is immediate. Submodule reset status is determined by reading or polling the submodule reset bit. If the submodule reset bit is read as a one, then the reset process has not yet completed. The submodule soft reset process could take up to 2ms each. The reset has completed if the submodule reset bit is read as a zero.

After all three submodules (in any order) have been reset and a read of each submodule reset bit indicates that the reset process is complete, the CPSW_3G software reset register bit may be written to complete the CPSW_3G module software reset operation. The CPSW_3G software reset bit controls the reset of the FIFO's, the statistics submodule, and the address lookup engine (ALE). The CPSW_3G software reset is immediate and will be indicated by reading a zero from the soft reset bit.

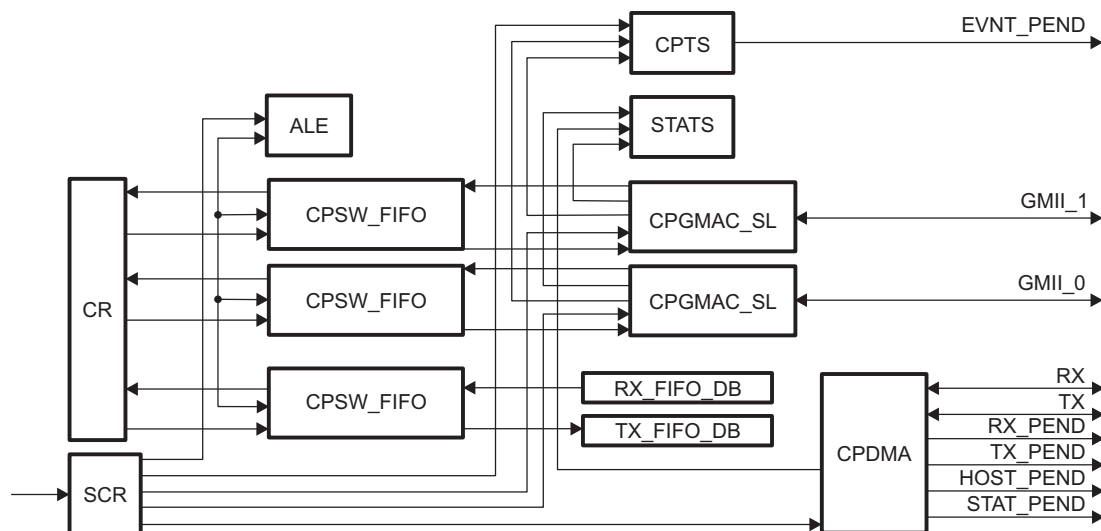
The CPSW_3GSS software reset bit controls the reset of the INT, REGS, and CPPI. The CPSW_3GSS software reset is immediate and will be indicated by reading a zero from the soft reset bit.

24.11.4.8 CPSW_3G

The CPSW_3G G/MII interfaces are compliant to the IEEE Std 802.3 Specification.

The CPSW_3G contains two CPGMAC_SL interfaces (ports 1 and 2), one CPPI interface Host Port (port 0), Common Platform Time Sync (CPTS), ALE Engine and CPDMA. A top-level block diagram of the CPSW_3G is shown in [Figure 24-198](#).

Figure 24-198. CPSW_3G Block Diagram



24.11.4.8.1 CPDMA RX and TX Interfaces

The CPDMA submodule is a CPPI compliant packet DMA transfer controller. The CPPI interface is port 0.

24.11.4.8.1.1 Functional Operation

After reset, initialization, and configuration the host may initiate transmit operations. Transmit operations are initiated by host writes to the appropriate transmit channel head descriptor pointer contained in the STATERAM block. The transmit DMA controller then fetches the first packet in the packet chain from memory in accordance with CPPI protocol. The DMA controller writes the packet into the external transmit FIFO in 64-byte bursts (maximum).

Receive operations are initiated by host writes to the appropriate receive channel head descriptor pointer after host initialization and configuration. The receive DMA controller writes the receive packet data to external memory in accordance with CPPI protocol. For a detailed description of buffer descriptors, see [Section 24.11.4.11](#), *CPPI Buffer Descriptors*.

24.11.4.8.1.2 Receive DMA Interface

The receive DMA is an eight channel CPPI compliant interface. Each channel has a single queue for frame reception.

24.11.4.8.1.2.1 Receive DMA Host Configuration

To configure the RX DMA for operation the software must perform the following:

1. Initialize the receive addresses.
2. Initialize the RX_HDP registers to 0.
3. Enable the desired receive interrupts in the [CPDMA_RX_INTMASK_SET](#) register.
4. Write the [CPDMA_RX_BUFFER_OFFSET](#) register value.
5. Setup the receive channel(s) buffer descriptors in host memory as required by CPPI
6. Enable the RX DMA controller by setting the RX_EN bit in the [CPDMA_RX_CONTROL](#) register.

24.11.4.8.1.2.2 Receive Channel Teardown

The host commands a receive channel teardown by writing the channel number to the [CPDMA_RX_TEARDOWN](#) register. When a teardown command is issued to an enabled receive channel the following will occur:

- Any current frame in reception will complete normally.
- The teardown_complete bit will be set in the next buffer descriptor in the chain (if there is one).
- The channel head descriptor pointer will be cleared to 0.
- A receive interrupt for the channel will be issued to the host.
- The software should acknowledge a teardown interrupt with a FFFF FFFCh Acknowledge value.

Channel teardown may be commanded on any channel at any time. The host is informed of the teardown completion by a set teardown complete buffer descriptor bit. The port does not clear any channel enables due to a teardown command. A teardown command to an inactive channel issues an interrupt that software should acknowledge with a FFFF FFFCh acknowledge value (note that there is no buffer descriptor in this case). Software may read the interrupt acknowledge location to determine if the interrupt was due to a commanded teardown. The read value will be FFFF FFFCh if the interrupt was due to a teardown command.

24.11.4.8.1.3 Transmit DMA Interface

The transmit DMA is an eight channel CPPI compliant interface. Priority between the eight queues may be either fixed or round robin as selected by the TX_PTYPE bit in the [CPDMA_DMACONTROL](#) register. If the priority type is fixed, then channel 7 has the highest priority and channel 0 has the lowest priority. Round robin priority proceeds from channel 0 to channel 7.

24.11.4.8.1.3.1 Transmit DMA Host Configuration

To configure the TX DMA for operation the software must do the following:

1. Initialize the TX_HDP registers to 0.
2. Enable the desired transmit interrupts in the [CPDMA_TX_INTMASK_SET](#) register.
3. Setup the transmit channel(s) buffer descriptors in host memory as defined in CPPI.
4. Configure and enable the transmit operation as desired in the [CPDMA_TX_CONTROL](#) register.
5. Write the appropriate TX_HDP registers with the appropriate values to start transmit operations.

24.11.4.8.1.3.2 Transmit Channel Teardown

The host commands a transmit channel teardown by writing the channel number to the [CPDMA_TX_TEARDOWN](#) register. When a teardown command is issued to an enabled transmit channel the following will occur:

- Any frame currently in transmission will complete normally

- The teardown complete bit will be set in the next sop buffer descriptor (if there is one).
- The channel head descriptor pointer will be cleared to 0.
- An interrupt will be issued to inform the host of the channel teardown.
- The software should acknowledge a teardown interrupt with a FFFF FFFCh acknowledge value

Channel teardown may be commanded on any channel at any time. The host is informed of the teardown completion by the set teardown complete buffer descriptor bit. The port does not clear any channel enables due to a teardown command. A teardown command to an inactive channel issues an interrupt that software should acknowledge with a FFFF FFFCh acknowledge value (note that there is no buffer descriptor in this case). Software may read the interrupt acknowledge location to determine if the interrupt was due to a commanded teardown. The read value will be FFFF FFFCh if the interrupt was due to a teardown command.

24.11.4.8.1.4 Transmit Rate Limiting

Transmit operations can be configured to rate limit the transmit data for each transmit priority. Rate limiting is enabled for a channel when the TX_RLIM bit associated with that channel is set in the [CPDMA_DMACONTROL](#) register. Rate limited channels must be the highest priority channels. For example, if two rate limited channels are required then TX_RLIM should be set to 11000000b with the MSB corresponding to channel 7. When any channels are configured to be rate-limiting, the priority type must be fixed for transmit. Round-robin priority type is not allowed when rate-limiting. Each of the eight transmit priorities has an associated register to control the rate at which the priority is allowed to send data (TX_PRI(0..7)_RATE) when the channel is rate-limiting. Each priority has a send count (PRI(0..7)_SEND_CNT) and an idle count (PRI(0..7)_IDLE_CNT). The transfer rate includes the inter-packet gap (12 bytes) and the preamble (8 bytes). The rate in Mbits/second that each priority is allowed to send is controlled by the equation:

$$\text{Priority Transfer rate in Mbit/s} = ((\text{PRI_IDLE_CNT} / (\text{PRI_IDLE_CNT} + \text{PRI_SEND_CNT})) \times \text{frequency} \times 32$$

Where *frequency* is the CPDMA interface clock (MAIN_CLK) frequency.

24.11.4.8.1.5 Command IDLE

The CMD_IDLE bit in the [CPDMA_DMACONTROL](#) register allows CPDMA operation to be suspended. When the idle state is commanded, the CPDMA will stop processing receive and transmit frames at the next frame boundary. Any frame currently in reception or transmission will be completed normally without suspension. For transmission, any complete or partial frame in the tx cell FIFO will be transmitted. For receive, frames that are detected by the CPDMA after the suspend state is entered are ignored. No statistics will be kept for ignored frames. Commanded idle is similar in operation to emulation control and clock stop.

24.11.4.8.2 Address Lookup Engine (ALE)

The address lookup engine (ALE) processes all received packets to determine which port(s) if any that the packet should be forwarded to. The ALE uses the incoming packet received port number, destination address, source address, length/type, and VLAN information to determine how the packet should be forwarded. The ALE outputs the port mask to the switch fabric that indicates the port(s) the packet should be forwarded to. The ALE is enabled when the ENABLE_ALE bit in the [ALE_CONTROL](#) register is set. All packets are dropped when the ENABLE_ALE bit is cleared to 0.

In normal operation, the CPGMAC_SL modules are configured to issue an abort, instead of an end of packet, at the end of a packet that contains an error (runt, frag, oversize, jabber, crc, alignment, code etc.) or at the end of a mac control packet. However, when the [SL_MACCONTROL](#) configuration bit(s) RX_CEF_EN, RX_CSF_EN, or RX_CMF_EN are set, error frames, short frames or mac control frames have a normal end of packet instead of an abort at the end of the packet. When the ALE receives a packet that contains errors (due to a set header error bit), or a mac control frame and does not receive an abort, the packet will be forwarded only to the host port (port 0). No ALE learning occurs on packets with errors or mac control frames. Learning is based on source address and lookup is based on destination address.

The ALE may be configured to operate in bypass mode by setting the ALE_BYPASS bit in the [ALE_CONTROL](#) register. When in bypass mode, all CPGMAC_SL received packets are forwarded only to the host port (port 0). Packets from the two ports can be on separate RX DMA channels by configuring the [P0_CPDMA_RX_CH_MAP](#) register. In bypass mode, the ALE processes host port transmit packets the same as in normal mode. In general, packets would be directed by the host in bypass mode.

The ALE may be configured to operate in OUI deny mode by setting the ENABLE_OUI_DENY bit in the [ALE_CONTROL](#) register. When in OUI deny mode, a packet with a non-matching OUI source address will be dropped unless the destination address matches a multicast table entry with the super bit set. Broadcast packets will be dropped unless the broadcast address is entered into the table with the super bit set. Unicast packets will be dropped unless the unicast address is in the table with block and secure both set (supervisory unicast packet).

Multicast supervisory packets are designated by the super bit in the table entry. Unicast supervisory packets are indicated when block and secure are both set. Supervisory packets are not dropped due to rate limiting, OUI, or VLAN processing.

24.11.4.8.2.1 Address Table Entry

The ALE table contains 1024 entries. Each table entry represents a free entry, an address, a VLAN, an address/VLAN pair, or an OUI address. Software should ensure that there are not double address entries in the table. The double entry used would be indeterminate. Reserved table bits must be written with zeroes.

Source Address learning occurs for packets with a unicast, multicast or broadcast destination address and a unicast or multicast (including broadcast) source address. Multicast source addresses have the group bit (bit 40) cleared before ALE processing begins, changing the multicast source address to a unicast source address. A multicast address of all ones is the broadcast address which may be added to the table. A learned unicast source address is added to the table with the following control bits:

Table 24-1256. Learned Address Control Bits

Bit(s)	Value
unicast_type	11
Block	0
Secure	0

If a received packet has a source address that is equal to the destination address then the following occurs:

- The address is learned if the address is not found in the table.
- The address is updated if the address is found.
- The packet is dropped.

Table Entry Type

00 - Free Entry

01 - Address Entry : unicast or multicast determined by destination **address bit 40**.

10 - VLAN entry

11 - VLAN Address Entry : unicast or multicast determined by **address bit 40**.

24.11.4.8.2.1.1 Free Table Entry

Table 24-1257. Free (Unused) Address Table Entry Bit Values

71:62	61:60	59:0
Reserved	ENTRY_TYPE(00)	Reserved

24.11.4.8.2.1.2 Multicast Address Table Entry

Table 24-1258. Multicast Address Table Entry Bit Values

71:69	68:66	65	64	63:62	61:60	59:48	47:0
Reserved	PORT_MASK	SUPER	Reserved	MCAST_FWD_STATE	ENTRY_TYPE(01)	Reserved	MULTICAST_ADDRESS

Supervisory Packet (SUPER)

When set, this field indicates that the packet with a matching multicast destination address is a supervisory packet.

0: Non-supervisory packet

1: Supervisory packet

Port Mask(2:0) (PORT_MASK)

This 3-bit field is the port bit mask that is returned with a found multicast destination address. There may be multiple bits set indicating that the multicast packet may be forwarded to multiple ports (but not the receiving port).

Multicast Forward State (MCAST_FWD_STATE)

Indicates the port state(s) required for the received port on a destination address lookup in order for the multicast packet to be forwarded to the transmit port(s). A transmit port must be in the Forwarding state in order to forward the packet. If the transmit PORT_MASK has multiple set bits then each forward decision is independent of the other transmit port(s) forward decision.

00 - Forwarding

01 - Blocking/Forwarding/Learning

10 - Forwarding/Learning

11 - Forwarding

The forward state test returns a true value if both the RX and TX ports are in the required state.

Table Entry Type (ENTRY_TYPE)

Address entry type. Unicast or multicast determined by address bit 40.

01: Address entry. Unicast or multicast determined by address bit 40.

Packet Address (MULTICAST_ADDRESS)

This is the 48-bit packet MAC address. For an OUI address, only the upper 24-bits of the address are used in the source or destination address lookup. Otherwise, all 48-bits are used in the lookup.

24.11.4.8.2.1.3 VLAN/Multicast Address Table Entry

Table 24-1259. VLAN/Multicast Address Table Entry Bit Values

71:69	68:66	65	64	63:62	61:60	59:48	47:0
Reserved	PORT_MASK	SUPER	Reserved	MCAST_FWD_STATE	ENTRY_TYPE(11)	VLAN_ID	MULTICAST_ADDRESS

Supervisory Packet (SUPER)

When set, this field indicates that the packet with a matching multicast destination address is a supervisory packet.

0: Non-supervisory packet

1: Supervisory packet

Port Mask(2:0) (PORT_MASK)

This 3-bit field is the port bit mask that is returned with a found multicast destination address. There may be multiple bits set indicating that the multicast packet may be forwarded to multiple ports (but not the receiving port).

Multicast Forward State (MCAST_FWD_STATE)

Indicates the port state(s) required for the received port on a destination address lookup in order for the multicast packet to be forwarded to the transmit port(s). A transmit port must be in the Forwarding state in order to forward the packet. If the transmit PORT_MASK has multiple set bits then each forward decision is independent of the other transmit port(s) forward decision.

00 - Forwarding

01 - Blocking/Forwarding/Learning

10 - Forwarding/Learning

11 - Forwarding

The forward state test returns a true value if both the RX and TX ports are in the required state.

Table Entry Type (ENTRY_TYPE)

Address entry type. Unicast or multicast determined by address bit 40.

11: VLAN address entry. Unicast or multicast determined by address bit 40.

VLAN ID (VLAN_ID)

The unique identifier for VLAN identification. This is the 12-bit VLAN ID.

Packet Address (MULTICAST_ADDRESS)

This is the 48-bit packet MAC address. For an OUI address, only the upper 24-bits of the address are used in the source or destination address lookup. Otherwise, all 48-bits are used in the lookup.

24.11.4.8.2.1.4 Unicast Address Table Entry

Table 24-1260. Unicast Address Table Entry Bit Values

71:68	67:66	65	64	63:62	61:60	59:48	47:0
Reserved	PORT_NUMBER	BLOCK	SECURE	UNICAST_TYPE (00) or (X1)	ENTRY_TYPE(01)	Reserved	UNICAST_ADDRESS

Port Number (PORT_NUMBER)

This field indicates the port number (not port mask) that the packet with a unicast destination address may be forwarded to. Packets with unicast destination addresses are forwarded only to a single port (but not the receiving port).

Block (BLOCK)

The block bit indicates that a packet with a matching source or destination address should be dropped (block the address).

0 - Address is not blocked.

1 - Drop a packet with a matching source or destination address (secure must be zero)

If block and secure are both set, then they no longer mean block and secure. When both are set, the block and secure bits indicate that the packet is a unicast supervisory (super) packet and they determine the unicast forward state test criteria. If both bits are set then the packet is forwarded if the receive port is in the Forwarding/Blocking/Learning state. If both bits are not set then the packet is forwarded if the receive port is in the Forwarding state.

Secure (SECURE)

This bit indicates that a packet with a matching source address should be dropped if the received port number is not equal to the table entry port_number.

0 - Received port number is a don't care.

1 - Drop the packet if the received port is not the secure port for the source address and do not update the address (block must be zero)

Unicast Type (UNICAST_TYPE)

This field indicates the type of unicast address the table entry contains.

00 - Unicast address that is not ageable.

01 - Ageable unicast address that has not been touched.

10 - OUI address - lower 24-bits are don't cares (not ageable).

11 - Ageable unicast address that has been touched.

Table Entry Type (ENTRY_TYPE)

Address entry. Unicast or multicast determined by address bit 40.

01: Address entry. Unicast or multicast determined by address bit 40.

Packet Address (UNICAST_ADDRESS)

This is the 48-bit packet MAC address. All 48-bits are used in the lookup.

24.11.4.8.2.1.5 OUI Unicast Address Table Entry

Table 24-1261. OUI Unicast Address Table Entry Bit Values

71:64	63:62	61:60	59:48	47:24	23:0
Reserved	UNICAST_TYPE(10)	ENTRY_TYPE(01)	Reserved	UNICAST_OUI	Reserved

Unicast Type (UNICAST_TYPE)

This field indicates the type of unicast address the table entry contains.

00 - Unicast address that is not ageable.

01 - Ageable unicast address that has not been touched.

10 - OUI address - lower 24-bits are don't cares (not ageable).

11 - Ageable unicast address that has been touched.

Table Entry Type (ENTRY_TYPE)

Address entry. Unicast or multicast determined by address bit 40.

01: Address entry. Unicast or multicast determined by address bit 40.

Packet Address (UNICAST_OUI)

For an OUI address, only the upper 24-bits of the address are used in the source or destination address lookup.

24.11.4.8.2.1.6 VLAN/Unicast Address Table Entry

Table 24-1262. Unicast Address Table Entry Bit Values

71:68	67:66	65	64	63:62	61:60	59:48	47:0
Reserved	PORT_NUMBER	BLOCK	SECURE	UNICAST_TYPE (00) or (X1)	ENTRY_TYPE (11)	VLAN_ID	UNICAST_ADDRESS

Port Number (PORT_NUMBER)

This field indicates the port number (not port mask) that the packet with a unicast destination address may be forwarded to. Packets with unicast destination addresses are forwarded only to a single port (but not the receiving port).]

Block (BLOCK)

The block bit indicates that a packet with a matching source or destination address should be dropped (block the address).

0 - Address is not blocked.

1 - Drop a packet with a matching source or destination address (secure must be zero)

If block and secure are both set, then they no longer mean block and secure. When both are set, the block and secure bits indicate that the packet is a unicast supervisory (super) packet and they determine the unicast forward state test criteria. If both bits are set then the packet is forwarded if the receive port is in the Forwarding/Blocking/Learning state. If both bits are not set then the packet is forwarded if the receive port is in the Forwarding state.

Secure (SECURE)

This bit indicates that a packet with a matching source address should be dropped if the received port number is not equal to the table entry PORT_NUMBER.

0 - Received port number is a don't care.

1 - Drop the packet if the received port is not the secure port for the source address and do not update the address (block must be zero)

Unicast Type (UNICAST_TYPE)

This field indicates the type of unicast address the table entry contains.

00 - Unicast address that is not ageable.

01 - Ageable unicast address that has not been touched.

10 - OUI address - lower 24-bits are don't cares (not ageable).

11 - Ageable unicast address that has been touched.

Table Entry Type (ENTRY_TYPE)

Address entry. Unicast or multicast determined by address bit 40.

11: VLAN address entry. Unicast or multicast determined by address bit 40.

VLAN ID (VLAN_ID)

The unique identifier for VLAN identification. This is the 12-bit VLAN ID.

Packet Address (UNICAST_ADDRESS)

This is the 48-bit packet MAC address. All 48-bits are used in the lookup.

24.11.4.8.2.1.7 VLAN Table Entry

Table 24-1263. VLAN Table Entry

71:62	61:60	59:48	47:27	26:24	23:19	18:16	15:11	10:8	7:3	2:0
Reserved	ENTRY_TYPE(10)	VLAN_ID	Reserved	FORCE_UNTAGGED_EGRESS	Reserved	REG_MCAS_T_FLOOD_MASK	Reserved	UNREG_MCAST_FLOOD_MASK	Reserved	VLAN_MEMBER_LIST

Table Entry Type (ENTRY_TYPE)

10: VLAN entry

VLAN ID (VLAN_ID)

The unique identifier for VLAN identification. This is the 12-bit VLAN ID.

Force Untagged Packet Egress (FORCE_UNTAGGED_EGRESS)

This field causes the packet VLAN tag to be removed on egress (except on port 0).

Registered Multicast Flood Mask (REG_MCAST_FLOOD_MASK)

Mask [port 2-1-0] used for multicast when the multicast address is found.

Unregistered Multicast Flood Mask (UNREG_MCAST_FLOOD_MASK)

Mask [port 2-1-0] used for multicast when the multicast address is not found.

VLAN Member List (VLAN_MEMBER_LIST)

This 3-bit field indicates which port(s) are members of the associated VLAN.

24.11.4.8.2.2 Packet Forwarding Processes

There are four processes that an incoming received packet may go through to determine packet forwarding. The processes are Ingress Filtering, VLAN_Aware Lookup, VLAN_Unaware Lookup, and Egress.

Packet processing begins in the Ingress Filtering process. Each port has an associated packet forwarding state that can be one of four values (Disabled, Blocked, Learning, or Forwarding). The default state for all ports is Disabled. The host sets the packet forwarding state for each port.

In the packet ingress process (receive packet process), there is a forward state test for unicast destination addresses and a forward state test for multicast addresses. The multicast forward state test indicates the port states required for the receiving port in order for the multicast packet to be forwarded to the transmit port(s). A transmit port must be in the Forwarding state for the packet to be forwarded for transmission. The mcast_fwd_state indicates the required port state for the receiving port as indicated in the preceding table. The unicast forward state test indicates the port state required for the receiving port in order to forward the unicast packet. The transmit port must be in the Forwarding state in order to forward the packet. The block and secure bits determine the unicast forward state test criteria. If both bits are set then the packet is forwarded if the receive port is in the Forwarding/Blocking/Learning state. If both bits are not set then the packet is forwarded if the receive port is in the Forwarding state. The transmit port must be in the Forwarding state regardless. The forward state test used in the ingress process is determined by the destination address packet type (multicast/unicast).

In general, packets received with errors are dropped by the address lookup engine without learning, updating, or touching the address. The error condition and the abort are indicated by the CPGMAC_SL to the ALE. Packets with errors may be passed to the host (not aborted) by a CPGMAC_SL port, if the port has the RX_CMF_EN, RX_CEF_EN, or RX_CSF_EN bit(s) set. Error packets that are passed to the host by the CPGMAC_SL are considered to be bypass packets by the ALE and are sent only to the host. Error packets do not learn, update, or touch addresses regardless of whether they are aborted or sent to the host. Packets with errors received by the host are forwarded as normal.

The following control bits are in the [SL_MACCONTROL](#) register:

- [22] RX_CEF_EN - enables frames that are fragments, long, jabber, CRC, code, and alignment errors to be forwarded
- [23] RX_CSF_EN - enables short frames to be forwarded
- [24] RX_CMF_EN - enables MAC control frames to be forwarded.

24.11.4.8.2.3 Learning Process

The learning process is applied to each receive packet that is not aborted. The learning process is a concurrent process with the packet forwarding process.

24.11.4.8.2.4 VLAN Aware Mode

The CPSW_3G is in VLAN aware mode when the VLAN_AWARE bit is set in the [CPSW_CONTROL](#) register. In VLAN aware mode, ports 0 receive packets (out of the CPSW_3G) may or may not be VLAN encapsulated depending on the RX_VLAN_ENCAP bit in the [CPSW_CONTROL](#) register. Port 0 receive packet data is never modified. VLAN is not removed regardless of the force untagged egress bit for Port 0. VLAN encapsulated receive packets have a 32-bit VLAN header encapsulation word added to the packet data. VLAN encapsulated packets are specified by a set rx_vlan_encap bit in the packet buffer descriptor.

Port 0 transmit packets are never VLAN encapsulated (encapsulation is not allowed).

In VLAN aware mode, transmitted packet data is changed depending on the packet type (pkt_type), packet priority (pkt_pri), and VLAN information.

24.11.4.8.2.5 VLAN Unaware Mode

The CPSW_3G is in VLAN unaware mode when the VLAN_AWARE bit is cleared to 0 in the [CPSW_CONTROL](#) register. Port 0 receive packets (out of the CPSW_3G) may or may not be VLAN encapsulated depending on the RX_VLAN_ENCAP bit in the [CPSW_CONTROL](#) register. Port 0 transmit packets are never VLAN encapsulated.

24.11.4.8.3 Packet Priority Handling

Packets are received on three ports, two are CPGMAC_SL Ethernet ports and the third port is the CPPI host port. Received packets have a received packet priority (0 to 7, with 7 being the highest priority).

The received packet priority is the port priority for untagged packets, and the actual packet priority for priority tagged and VLAN tagged packets. The received packet priority is mapped through the receive ports associated packet priority to header packet priority mapping register to obtain the header packet priority (the CPDMA RX and TX nomenclature is reversed from the CPGMAC_SL nomenclature).

The header packet priority is mapped through the header priority to switch priority mapping register to obtain the hardware switch priority (0 to 3, with 3 being the highest priority). The header packet priority is then used as the actual transmit packet priority if the VLAN information is to be sent on egress.

24.11.4.8.4 FIFO Memory Control

Each of the three CPSW_3G ports has an identical associated FIFO. Each FIFO contains a single logical receive queue and four logical transmit queues (priority 0 through 3). Each FIFO memory contains 20,480 bytes (20k) total organized as 2560 by 64-bit words contained in a single memory instance. The FIFO memory is used for the associated port transmit and receive queues. The TX_MAX_BLKs field in the FIFOs associated Px_MAX_BLKs register determines the maximum number of 1k FIFO memory blocks to be allocated to the four logical transmit queues (transmit total). The RX_MAX_BLKs field in the FIFO's associated Px_MAX_BLKs register determines the maximum number of 1k memory blocks to be allocated to the logical receive queue. The TX_MAX_BLKs value plus the RX_MAX_BLKs value must sum to 20 (the total number of blocks in the FIFO). If the sum were less than 20, then some memory blocks would be unused. The default is 17 (decimal) transmit blocks and three receive blocks. The FIFOs follow the naming convention of the Ethernet ports. Host Port is Port0 and External Ports are Port1 and Port2.

24.11.4.8.5 FIFO Transmit Queue Control

There are four transmit queues in each transmit FIFO. Software has some flexibility in determining how packets are loaded into the queues and on how packet priorities are selected for transmission (how packets are removed and transmitted from queues). All ports on the switch have identical FIFO's. For the purposes of the below the transmit FIFO is switch egress even though the port 0 transmit FIFO is connected to the CPDMA receive (also switch egress). The CPDMA nomenclature is reversed from the CPGMAC_SL nomenclature due to legacy reasons.

24.11.4.8.5.1 Normal Priority Mode

When operating in normal mode, lower priority frames are dropped before higher priority frames. The intention is to give preference to higher priority frames. Priority 3 is the highest priority and is allowed to fill the FIFO. Priority 2 will drop packets if the packet is going to take space in the last 2k available. Priority 1 will drop packets if the packet is going to take space in the last 4k available. Priority 0 will drop packets if the packet is going to take space in the last 6k available. If fewer than 4 priorities are to be implemented then the priorities should be mapped such that the highest priorities are used. For example, if two priorities are going to be used then all packets should be mapped to priorities 3 and 2 and priorities 1 and 0 should be unused. Priority escalation may be used in normal priority mode if desired. Normal priority mode is configured as described below:

- Select normal priority mode by setting TX_IN_SEL = 00 for all ports (default value in P0/1/2_TX_IN_CTL)
- Configure priority mapping to use only the highest priorities if less than 4 priorities are used. Refer to [Section 24.11.4.8.3, Packet Priority Handling](#).

24.11.4.8.5.2 Dual MAC Mode

When operating in dual MAC mode the intention is to transfer packets between ports 0 and 1 and ports 0 and 2, but not between ports 1 and 2. Each CPGMAC_SL appears as a single MAC with no bridging between MAC's. Each CPGMAC_SL has at least one unique (not the same) mac address.

Dual MAC mode is configured as described below:

- Set the ALE_VLAN_AWARE bit in the [ALE_CONTROL](#) register. This bit configures the ALE to process in VLAN aware mode. The CPSW_3G VLAN aware bit (VLAN_AWARE in [CPSW_CONTROL](#)) determines how packets VLAN's are processed on CPGMAC_SL egress and does not affect how the ALE processes packets or the packet destination. The CPSW_3G VLAN aware bit may be set or not as required (must be set, if VLAN's are to exit the switch).
- Configure the Port 1 to Port 0 VLAN
 - Add a VLAN Table Entry with ports 0 and 1 as members (clear the flood masks).
 - Add a VLAN/Unicast Address Table Entry with the Port1/0 VLAN and a port number of 0. Packets received on port 1 with this unicast address will be sent only to port 0 (egress). If multiple mac addresses are desired for this port then multiple entries of this type may be configured.
- Configure the Port 2 to Port 0 VLAN
 - Add a VLAN Table Entry with ports 0 and 2 as members (clear the flood masks).
 - Add a VLAN/Unicast Address Table Entry with the Port2/0 VLAN and a port number of 0. Packets received on port 2 with this unicast address will be sent only to port 0 (egress). If multiple mac addresses are desired for this port then multiple entries of this type may be configured.
- Packets from the host (port 0) to ports 1 and 2 should be directed. If directed packets are not desired then VLAN with addresses can be added for both destination ports.
- Select the dual mac mode on the port 0 FIFO by setting TX_IN_SEL = 01 in [P0_TX_IN_CTL](#). The intention of this mode is to allow packets from both ethernet ports to be written into the FIFO without one port starving the other port.
- The priority levels may be configured such that packets received on port 1 egress on one CPDMA RX channel while packets received on port 2 egress on a different CPDMA RX channel.

24.11.4.8.5.3 Rate Limit Mode

Rate-limit mode is intended to allow some CPDMA transmit (switch ingress) channels and some CPGMAC_SL FIFO priorities (switch egress) to be rate-limited. Non rate-limited traffic (bulk traffic) is allowed on non rate-limited channels and FIFO priorities. The bulk traffic does not impact the rate-limited traffic. Rate-limited traffic must be configured to be sent to rate-limited queues (via packet priority handling). The allocated rates for rate-limited traffic must not be oversubscribed. For example, if port 1 is sending 15% rate limited traffic to port 2 priority 3, and port 0 is also sending 10% rate-limited traffic to

port 2 priority 3, then the port 2 priority 3 egress rate must be configured to be 25% plus a percent or two for margin. The switch must be configured to allow some percentage of non rate-limited traffic. Non rate-limited traffic must be configured to be sent to non rate-limited queues. No packets from the host should be dropped, but non rate-limited traffic received on an ethernet port can be dropped. Rate-limited mode is configured as shown below:

1. Set TX_IN_SEL = 10 in P1/2_TX_IN_CTL to enable ports 1 and 2 transmit FIFO inputs to be configured for rate-limiting queues. Enabling a queue to be rate-limiting with this field affects only the packet being loaded into the FIFO, it does not configure the transmit for queue shaping.
2. Configure the number of rate-limited queues for port 1 and 2 transmit FIFO's by setting the TX_RATE_EN field in P1/2_TX_IN_CTL. Rate limited queues must be the highest number. For example, if there are two rate limited queues then 1100 would be written to this field for priorities 3 and 2. This field enables the FIFO to allow rate-limited traffic into rate-limited queues while discriminating against non rate-limited queues.
3. Set P1_PRIN_SHAPE_EN and P2_PRIN_SHAPE_EN in the CPSW_PTYPE register. These bits determine which queues actually shape the output data stream. In general, the same priorities that are set in TX_RATE_EN are set in these bits as well, but the FIFO input and output enable bits are separate to allow rate-limiting from the host to non shaped channels if desired. When queue shaping is not enabled for a queue then packets are selected for egress based on priority. When queue shaping is enabled then packets are selected for egress based on queue percentages. If shaping is required on a single queue then it must be priority 3 (priorities 2, 1 and 0 are strict priority). If shaping is required on two queues then it must be on priorities 2 and 3 (priorities 1 and 0 are strict priority). If shaping is required on three queues then it must be priorities 3, 2, and 1 (priority 0 would then get the leftovers). Priority shaping follows the requirements in the IEEE P802.1Qav/D6.0 specification. Priority shaping is not compatible with priority escalation (escalation must be disabled).
4. P0_TX_IN_CTL[17:16] TX_IN_SEL should be set to 00, so Port 0 egress (CPDMA RX) is not rate-limited.
5. The CPDMA is configured for rate-limited transmit (switch ingress) channels by setting the highest bits of the TX_RLIM field in the CPDMA_DMACONTROL register. If there are two rate-limited channels, then TX_RLIM = 11000000 (the rate limited channels must be the highest priorities). Also, the TX_PTYPE bit in the CPDMA_DMACONTROL register must be set (fixed priority mode). Rate-limited channels must go to rate-limited FIFO queues, and the FIFO queue rate must not be oversubscribed.

24.11.4.8.6 Audio Video Bridging

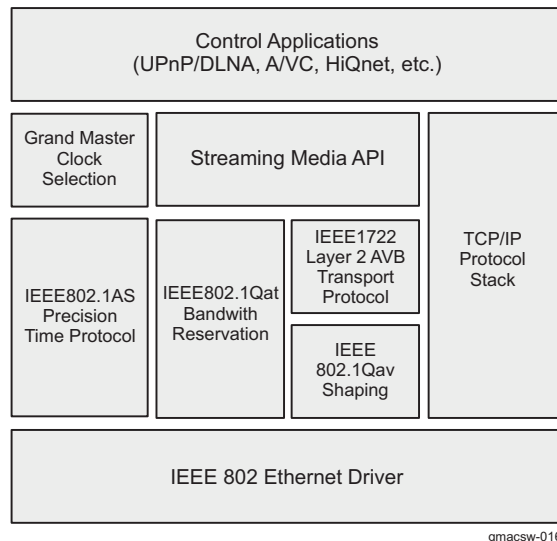
Audio Video Bridging is an ongoing project of IEEE 802.1 concerned with enabling low-latency streaming of time-sensitive audiovisual data over networks. Devices are designated as talkers (transmitters), bridges, or listeners (receivers). It is suggested that the maximum latency could be 2 ms over 7 hops for Class A devices and 20 ms over 7 hops for Class B devices. A hop is essentially a single local area network stage in the journey of a packet. Every time a bridge is encountered between one network section and another a hop is involved. One of the performance goals is that AVB streams will not use more than 75 percent of a link's bandwidth, leaving the remaining capacity for non-AVB streams.

The goal of developing AVB is simply--extend Ethernet's data-networking capabilities to the realm of reliable real-time audio/video networking.

An "Audio Video Bridging" network is one that implements a set of protocols being developed by the IEEE 802.1 Audio/Video Bridging Task Group. There are four primary differences between the proposed Audio Video Bridging architecture and existing 802 architectures (from now on the term "AVB" will be used instead of "Audio Video Bridging"):

1. Precise synchronization - IEEE 802.1AS: "*Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks*". "a.k.a Precision Time Protocol (PTP)."
2. Traffic shaping for media streams - IEEE 802.1Qav: "*Virtual Bridged Local Area Networks: Forwarding and Queuing for Time-Sensitive Streams*."
3. Admission controls - IEEE 802.1Qat: "*Virtual Bridged Local Area Networks - Amendment 9: Stream Reservation Protocol (SRP)*."
4. Identification of non-participating devices - IEEE 802.1BA: "Audio/Video Bridging (AVB) Systems"

Figure 24-199. The Network Static with AVB



The following sections describe the media transport protocols that work within the AVB framework.

24.11.4.8.6.1 IEEE 802.1AS: Timing and Synchronization for Time-Sensitive Applications in Bridged Local Area Networks (Precision Time Protocol (PTP))

The protocol defined by 802.1AS automatically selects a device to be the master clock, and then distributes this clock throughout the bridged LAN / IP subnet to all other network devices using link-specific transmit/receive time-stamping. However, we only use a two-step solution only on transmit. That is, we do not modify a packet with the timestamp on the way out. The timestamp packet is sent out and then a separate message with the timestamp is sent by the host afterward. Receive can be one or two step.

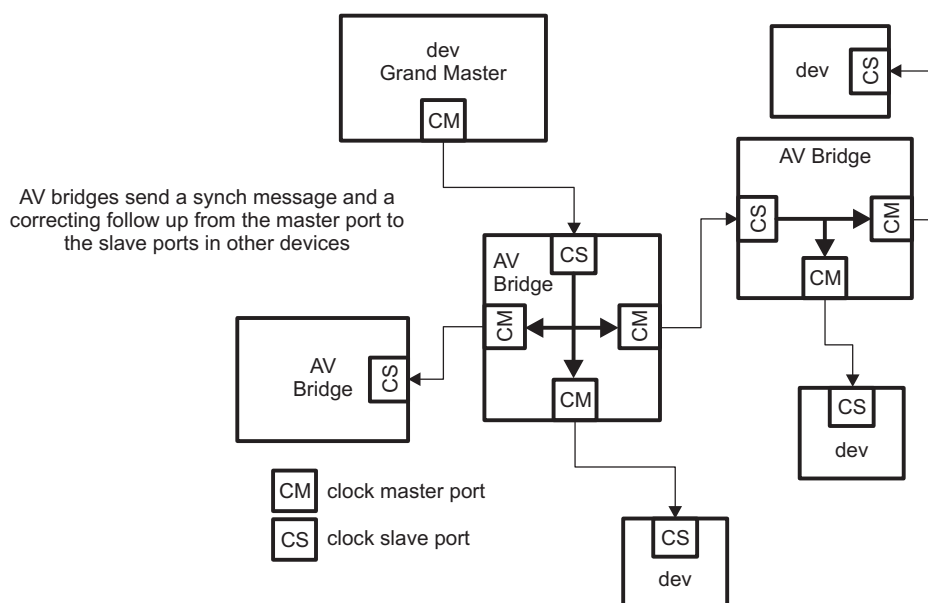
NOTE: The 802.1AS-distributed clock is not used as a media clock. Rather, the shared 802.1AS clock reference is used to regenerate the media clock at the listener/renderer. Such a reference removes the need to force the latency of the network to be constant, or compute long running averages in order to estimate the actual media rate of the transmitter in the presence of substantial network jitter. IEEE 802.1AS is based on the ratified IEEE 1588 standard.

Based on IEEE 1588:2002, PTP devices exchange standard Ethernet messages that synchronize network nodes to a common time reference by defining clock master selection and negotiation algorithms, link delay measurement and compensation, and clock rate matching and adjustment mechanisms.

Designed as a simplified profile of IEEE 1588, a primary difference between 1588 and IEEE 802.1AS is that PTP is a layer 2--in other words, a non-IP routable protocol. Like IEEE 1588, PTP defines an automatic method for negotiating the network clock master, the Best Master Clock Algorithm (BMCA). PTP nodes can be assigned one of eight priority levels, presumably based on clock quality. BMCA defines the underlying negotiation and signaling mechanism whose purpose is to identify the AVB LAN Grandmaster. Once a Grandmaster has been selected, synchronization automatically begins.

At the core of 802.1AS synchronization is time-stamping. In short, during PTP message ingress/egress from the 802.1AS-capable MAC, the PTP Ether type triggers the sampling of the value of a local real-time counter (RTC). Slave nodes compare the value of their RTC against the PTP Grandmaster and, by use of link delay measurement and compensation techniques, match their RTC value to the time of the AVB LAN PTP domain. After network time throughout the AVB LAN has converged, periodic SYNC and FOLLOW_UP messages provide the information that enables the PTP rate matching adjustment algorithms. The result is all PTP nodes are then synchronized to the same "Wall Clock" time. PTP assures 1- μ s accuracy over seven network hops.

Figure 24-200. AVB Network & PTP Clock Entities



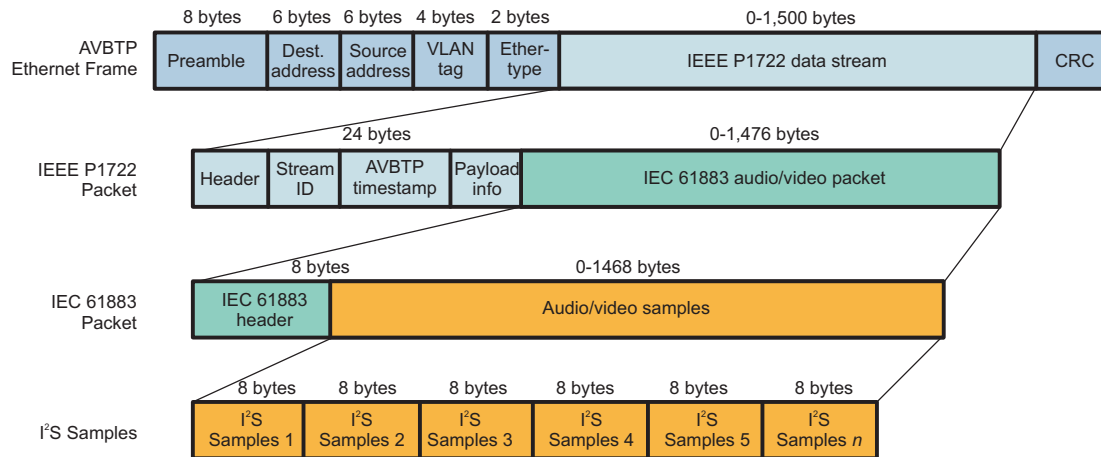
The media transport protocols that work within the AVB framework are:

24.11.4.8.6.1.1 IEEE 1722: "Layer 2 Transport Protocol for Time-Sensitive Streams"

AVBTP or 1722 sits above the IEEE 802.1 AVB plumbing and below the application layer. It acts as the conduit between an Ethernet MAC and a streaming application. AVBTP abstracts the underlying network transmission channel to enable the virtual connection of distributed audio and video CODECs over reliable Ethernet networks. A complete AVBTP Ethernet packet is shown in [Figure 24-201](#) and illustrates how IEC 61883-6 AM824 uncompressed audio samples are encapsulated in an Ethernet frame.

Figure 24-201. IEEE 1722 Packets

IEEE 1722 Packet Construction



1722 or AVBTP Presentation Time and Synchronization:

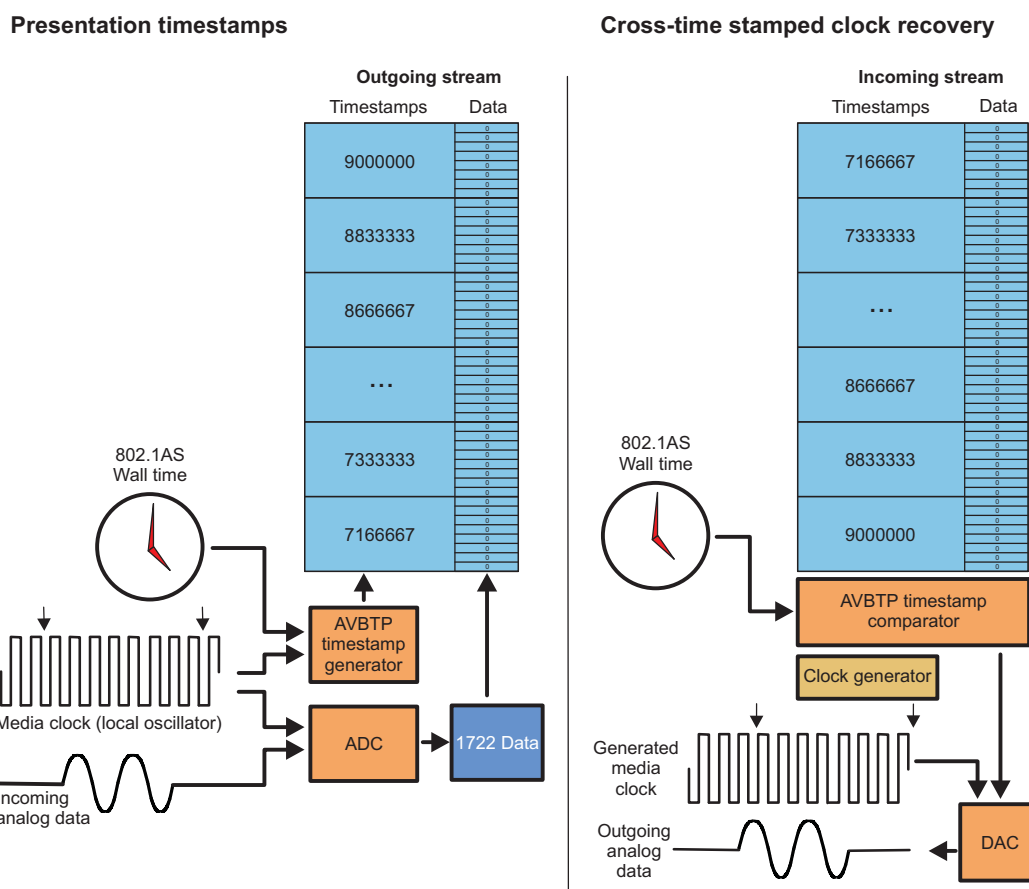
Synchronization in an AVB network starts with the Precision Time Protocol but ends with synchronized media clocks. PTP is responsible for synchronizing all nodes in an AVB network to identical wall clock time; not for synchronizing media clocks. In other words, PTP does not actually transport synchronized media clocks but instead provides a low-level building block crucial for managing a distributed media synchronization system.

A crucial benefit of this approach is coexistence of multiple, independent media clock domains on an AVB network. Unrelated audio and video streams can simultaneously exist in the same LAN.

AVBTP assumes that AVB node media clocks are clocked by free-running oscillators. It is also assumed that the node's internal concept of wall clock time has been synchronized to the PTP Grandmaster. AVBTP media clock sources embed "AVBTP Presentation Timestamps" in AVBTP streaming packets. [Figure 24-202](#) illustrates the relationship between PTP network time and AVBTP Presentation Timestamps.

24.11.4.8.6.1.2 IEEE 1733: Extends RTCP for RTP Streaming over AVB-supported Networks

This standard specifies the protocol, data encapsulations, connection management and presentation time procedures used to ensure interoperability between audio and video based end stations that use standard networking services provided by all IEEE 802 networks meeting QoS requirements for time-sensitive applications by leveraging the Real-time Transport Protocol (RTP) family of protocols and IEEE 802.1 Audio/Video Bridging (AVB) protocols.



The requirements for end stations that participate as "talkers" i.e., sources of time-sensitive streams are different from the requirements that apply to "listeners", the destination station(s) for the streams.

Talker Behavior

In order for Talker-originated data streams to make use of the credit-based shaper behavior in Bridges, it is a requirement for a Talker to use the priorities that the Bridges in the network recognize as being associated with SR classes exclusively for transmitting stream data.

It is also necessary for the Talker and the Bridges in the path to the Listener(s), to have a common view of the bandwidth required in order to transmit the Talker's streams, and for that bandwidth to be reserved along the path to the Listener(s). This latter requirement can be met by means of stream reservation mechanisms, such as defined in SRP, or by other management means.

End stations that are Talkers shall exhibit transmission behavior for frames that are part of "time-sensitive streams" that is consistent with the operation of the credit-based shaper algorithm, both in terms of the way they transmit frames that are part of an individual data stream, and in terms of the way they transmit stream data frames from a Port.

In effect, the queuing model for a Talker Port (and a Listener port), and for given priorities, can be considered to look like [Figure 24-203](#).

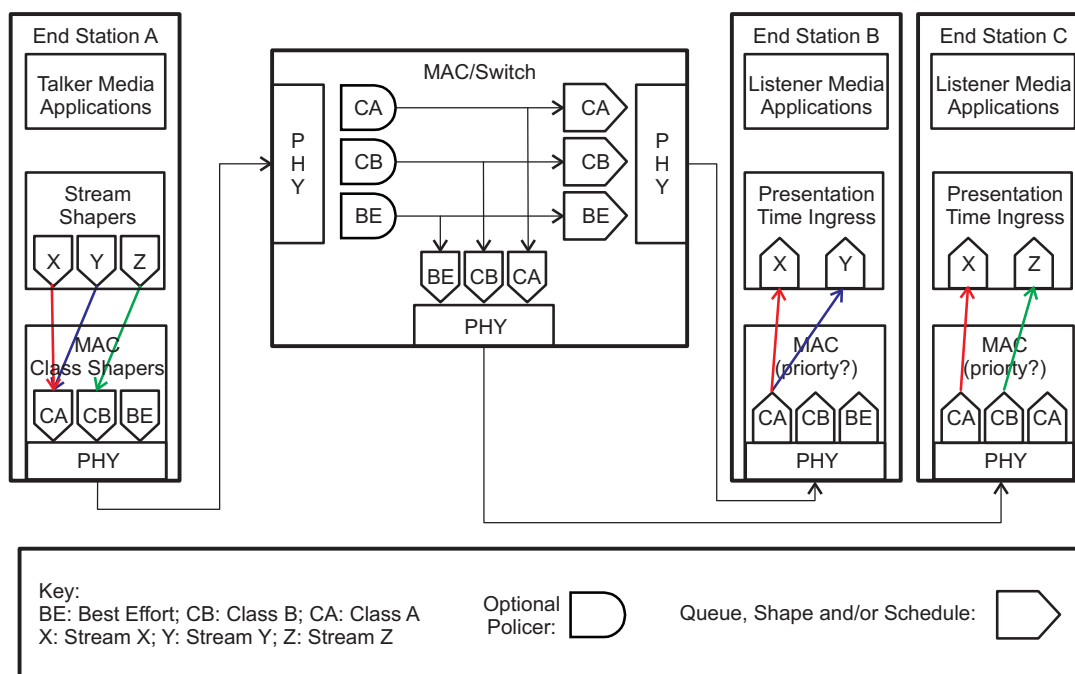
Listener Behavior

The primary requirement for a listener station is that it is capable of buffering the amount of data that could be transmitted for a stream during a time period equivalent to the accumulated maximum jitter that could be experienced by stream data frames in transmission between Talker and Listener.

From the point of view of the specification of the forwarding and queuing requirements for time-sensitive streams, it is assumed that the listener will assess the buffering required for a stream as part of the stream bandwidth reservation mechanisms employed by the implementation.

The credit-based shaper's operation details are beyond the scope of this document.

Figure 24-203. AV Stream Queuing/Policing



24.11.4.8.6.2.1 Configuring the Device for 802.1Qav Operation:

There is no dedicated register-set to be configured for the time-sensitive stream handling. The list of functional features of CPSW_3G that will have to be configured are:

- DESCRIPTORS and CHANNEL CONFIGURATIONS:
 - CPPI TX and RX descriptors
 - VLAN and Priority tags

Table 24-1264. Example of TX Configuration

TX DMA CHANNEL	Packet Priority	Switch Queue Priority
7	7	3
6	5	2
5	3	1
4	1	0

Table 24-1265. Example of RX Configuration

RX DMA CHANNEL	Packet Priority	Switch Queue Priority
0	7	0
0	5	0
0	3	0
0	1	0

- ALE Configuration:
 - ALE in VLAN-ware mode, Non-ALE in bypass mode.

Table 24-1266. Example of Rate-limit Configurations

Register	Value	Description
CPSW_PTYPE	0x0006 0000	For Port1 -- 2 highest priority channels.
P1_TX_IN_CTL[23:20] TX_RATE_EN	0b1100	2 highest priority channels are rate limited
P1_TX_IN_CTL[17:16] TX_IN_SEL	0b10	Rate limit mode
P1_SEND_PERCENT	0x14 3E00	20% PRI7, 62% PRI5
CPDMA_DMACONTROL	0xC001	Chan7, Chan6 are Rate Limited. Round-robin selection of DMA channel.
CPDMA_TX_PRI7_RATE	0x1 0013	200 Mbps
CPDMA_TX_PRI6_RATE	0x1 0005	~600 Mbps

24.11.4.8.7 Ethernet MAC Sliver (CPGMAC_SL)

The CPGMAC_SL peripheral shall be compliant to the IEEE Std 802.3 Specification. Half-duplex mode is supported in 10/100 Mbps mode, but not in 1000 Mbps (gigabit) mode.

Features:

- Synchronous 10/100/1000 Mbit operation
- G/MII Interface
- Hardware Error handling including CRC
- Full-Duplex Gigabit operation (half-duplex gigabit is not supported)
- EtherStats and 802.3Stats RMON statistics gathering support for external statistics collection module
- Transmit CRC generation selectable on a per channel basis
- Emulation Support
- VLAN Aware Mode Support
- Hardware flow control

- Programmable Inter Packet Gap (IPG).

24.11.4.8.7.1 G/MII Media Independent Interface

The following sections cover operation of the Media Independent Interface in 10/100/1000 Mbps modes. An IEEE 802.3 compliant Ethernet MAC controls the interface.

24.11.4.8.7.1.1 Data Reception

24.11.4.8.7.1.1.1 Receive Control

Data received from the PHY is interpreted and output. Interpretation involves detection and removal of the preamble and start of frame delimiter, extraction of the address and frame length, data handling, error checking and reporting, cyclic redundancy checking (CRC), and statistics control signal generation.

24.11.4.8.7.1.1.2 Receive Inter-Frame Interval

The 802.3 required inter-packet gap (IPG) is 24 GMII clocks (96 bit times) for 10/100 Mbit modes, and 12 GMII clocks (96 bit times) for 1000 Mbit mode. However, the MAC can tolerate a reduced IPG (2 GMII clocks in 10/100 mode and 5 GMII clocks in 1000 mode) with a correct preamble and start frame delimiter.

This interval between frames must comprise (in the following order):

- An Inter-Packet Gap (IPG).
- A seven octet preamble (all octets 0x55).
- A one octet start frame delimiter (0x5D).

24.11.4.8.7.1.2 Data Transmission

The Gigabit Ethernet Mac Sliver (GMII) passes data to the PHY when enabled. Data is synchronized to the transmit clock rate. The smallest frame that can be sent is two bytes of data with four bytes of CRC (6 byte frame).

24.11.4.8.7.1.2.1 Transmit Control

A jam sequence is output if a collision is detected on a transmit packet. If the collision was late (after the first 64 bytes have been transmitted) the collision is ignored. If the collision is not late, the controller will back off before retrying the frame transmission. When operating in full duplex mode the carrier sense (CRS) and collision sensing modes are disabled.

24.11.4.8.7.1.2.2 CRC Insertion

The MAC generates and appends a 32-bit Ethernet CRC onto the transmitted data, if the transmit packet header PASS_CRC bit is 0. For the CPMAC_SL generated CRC case, a CRC at the end of the input packet data is not allowed.

If the header word PASS_CRC bit is set, then the last four bytes of the TX data are transmitted as the frame CRC. The four CRC data bytes should be the last four bytes of the frame and should be included in the packet byte count value. The MAC performs no error checking on the outgoing CRC when the PASS_CRC bit is set.

24.11.4.8.7.1.2.3 MTXER

The MTXER signal is only used for EEE. If an underflow condition occurs on a transmitted frame, the frame CRC will be inverted to indicate the error to the network. Underflow is a hardware error.

24.11.4.8.7.1.2.4 Adaptive Performance Optimization (APO)

The Ethernet MAC port incorporates Adaptive Performance Optimization (APO) logic that may be enabled by setting the TX_PACE bit in the [SL_MACCONTROL](#) register. Transmission pacing to enhance performance is enabled when set. Adaptive performance pacing introduces delays into the normal transmission of frames, delaying transmission attempts between stations, reducing the probability of collisions occurring during heavy traffic (as indicated by frame deferrals and collisions) thereby increasing the chance of successful transmission.

When a frame is deferred, suffers a single collision, multiple collisions or excessive collisions, the pacing counter is loaded with an initial value of 31. When a frame is transmitted successfully (without experiencing a deferral, single collision, multiple collision or excessive collision) the pacing counter is decremented by one, down to zero.

With pacing enabled, a new frame is permitted to immediately (after one IPG) attempt transmission only if the pacing counter is zero. If the pacing counter is non zero, the frame is delayed by the pacing delay, a delay of approximately four inter-packet gap delays. APO only affects the IPG preceding the first attempt at transmitting a frame. It does not affect the back-off algorithm for re-transmitted frames.

24.11.4.8.7.1.2.5 Inter-Packet-Gap Enforcement

The measurement reference for the IPG of 96 bit times is changed depending on frame traffic conditions. If a frame is successfully transmitted without collision, and MCRS is de-asserted within approximately 48 bit times of MTXEN being de-asserted, then 96 bit times is measured from MTXEN. If the frame suffered a collision, or if MCRS is not de-asserted until more than approximately 48 bit times after MTXEN is de-asserted, then 96 bit times (approximately, but not less) is measured from MCRS.

The transmit IPG can be shortened by eight bit times when enabled and triggered. The TX_SHORT_GAP_EN bit in the [SL_MACCONTROL](#) register enables the TX_SHORT_GAP input to determine whether the transmit IPG is shorted by eight bit times.

24.11.4.8.7.1.2.6 Back Off

The Gigabit Ethernet Mac Sliver (GMII) implements the 802.3 binary exponential back-off algorithm.

24.11.4.8.7.1.2.7 Programmable Transmit Inter-Packet Gap

The transmit inter-packet gap (IPG) is programmable through the [SL_TX_GAP](#) register. The default value is decimal 12. The transmit IPG may be increased to the maximum value of 1FFh. Increasing the IPG is not compatible with transmit pacing. The short gap feature will override the increased gap value, so the short gap feature may not be compatible with an increased IPG.

24.11.4.8.7.1.2.8 Speed, Duplex and Pause Frame Support Negotiation

The CPMAC_SL can operate in half duplex or full duplex in 10/100 Mbit modes, and can operate in full duplex only in 1000 Mbit mode. Pause frame support is included in 10/100/1000 Mbit modes as configured by the host.

24.11.4.8.7.2 RMII Interface

The CPRMII peripheral is compliant to the RMII specification document.

24.11.4.8.7.2.1 Features

- Source Synchronous 10/100 Mbit operation
- Full and Half Duplex support

24.11.4.8.7.2.2 RMII Receive (RX)

The CPRMII receive (RX) interface converts the input data from the external RMII PHY (or switch) into the required MII (CPGMAC) signals. The carrier sense and collision signals are determined from the RMII input data stream and transmit inputs as defined in the RMII specification.

An asserted RMRXER on any di-bit in the received packet will cause an MRXER assertion to the CPGMAC during the packet. In 10Mbps mode, the error is not required to be duplicated on 10 successive clocks. Any di-bit which has an asserted RMII_RXER during any of the 10 replications of the data will cause the error to be propagated.

Any received packet that ends with an improper nibble boundary aligned RMCERSDV toggle will issue an MRXER during the packet to the CPGMAC. Also, a change in speed or duplex mode during packet operations will cause packet corruption.

The CPRMII can accept receive packets with shortened preambles, but 0x55 followed by a 0x5D is the shortest preamble that will be recognized (1 preamble byte with the start of frame byte). At least one byte of preamble with the start of frame indicator is required to begin a packet. An asserted RMCERSDV without at least a single correct preamble byte followed by the start of frame indicator will be ignored.

24.11.4.8.7.2.3 RMII Transmit (TX)

The CPRMII transmit (TX) interface converts the GMAC_SW MII input data into the RMII transmit format. The data is then output to the external RMII PHY.

The GMAC_SW does not source the transmit error (MII TXERR) signal. Any transmit frame from the CPGMAC with an error (underrun) will be indicated as an error by an error CRC. Transmit error is assumed to be de-asserted at all times and is not an input into the CPRMII module. Zeroes are output on RMTXD[1:0] for each clock that RMTXEN is de-asserted.

24.11.4.8.7.3 RGMII Interface

The CPRGMII peripheral is compliant to the RGMII specification document.

24.11.4.8.7.3.1 RGMII Features

- Supports 1000/100/10 Mbps speed
- In SR1.1, Internal TXC delay on transmit is always enabled
- In SR2.x, Internal TXC delay on transmit can be enabled or disabled using bits [26] RGMII2_ID_MODE_N and [25] RGMII1_ID_MODE_N of the [CTRL_CORE_SMA_SW_1](#) register.
- MII mode is not supported

24.11.4.8.7.3.2 RGMII Receive (RX)

The CPRGMII receive (RX) interface converts the source synchronous DDR input data from the external RGMII PHY into the required G/MII (CPGMAC) signals.

24.11.4.8.7.3.3 In-Band Mode of Operation

The CPRGMII is operating in the in-band mode of operation when the EXT_EN bit of the [SL_MACCONTROL](#) register is set to 1. The link status, duplexity, and speed are determined from the RGMII input data stream RXD[3:0] when RX_CTL is deasserted, as defined in the RGMII specification. The PHY might need to be configured beforehand to output in-band data. The in-band data is indicated as shown in [Table 24-1267](#).

Table 24-1267. In-Band Data

RXD3	RXD[2:1]		RXD0
Duplex status: 0: half-duplex 1: full-duplex	Link Speed: 00: 10-Mbps mode 01: 100-Mbps mode 10: 1000-Mbps mode 11: reserved	RXC_CLK Speed: 2.5 MHz 25 MHz 125 MHz reserved	Link Status: 0: Link is down 1: Link is up

24.11.4.8.7.3.4 Forced Mode of Operation

The CPRGMII is operating in the forced mode of operation when the EXT_EN bit of the [SL_MACCONTROL](#) register is set to 0. In the forced mode of operation, the in-band data is ignored if present. The link status is forced high, and the duplexity and speed are determined from the [SL_MACCONTROL](#)[0] FULLDUPLEX and [7] GIG bits. If GIG = 1, then operation is gigabit mode. If the GIG is 0, the operation is 100 Mbps mode.

24.11.4.8.7.3.5 RGMII Transmit (TX)

The CPRGMII transmit (TX) interface converts the CPGMAC G/MII input data into the DDR RGMII format. The DDR data is then output to the external PHY.

The CPGMAC does not source the transmit error (TXERR) signal. Any transmit frame from the CPGMAC with an error (underrun) will be indicated as an error by an error CRC. Transmit error is assumed to be de-asserted at all times and is not an input into the CPRGMII module.

The TXD[7:0] data bus uses only the lower nibble. The CPRGMII will output the lower nibble twice in 10/100 mode to avoid unnecessary signal switching.

Packets will be precluded from transmission through the CPRGMII module for 4096 transmit clocks after the rising edge of RGMII_LINK. Packet transmission will begin on the first TX_CTL rising edge after the 4096 transmit clock count has expired.

24.11.4.8.7.4 Frame Classification

Received frames are proper (good) frames if they are between 64 and RX_MAXLEN in length (inclusive) and contain no errors (code/align/CRC).

Received frames are long frames if their frame count exceeds the value in the [SL_RX_MAXLEN](#) register. The [SL_RX_MAXLEN](#) register reset (default) value is 1518 (decimal). Long received frames are either oversized or jabber frames. Long frames with no errors are oversized frames. Long frames with CRC, code, or alignment errors are jabber frames.

Received frames are short frames if their frame count is less than 64 bytes. Short frames that contain no errors are undersized frames. Short frames with CRC, code, or alignment errors are fragment frames.

A received long packet will always contain RX_MAXLEN number of bytes transferred to memory (if RX_CEF_EN = 1). An example with RX_MAXLEN = 1518 is:

- If the frame length is 1518, then the packet is not a long packet and there will be 1518 bytes transferred to memory.
- If the frame length is 1519, there will be 1518 bytes transferred to memory. The last three bytes will be the first three CRC bytes.
- If the frame length is 1520, there will be 1518 bytes transferred to memory. The last two bytes will be the first two CRC bytes.
- If the frame length is 1521, there will be 1518 bytes transferred to memory. The last byte will be the first CRC byte.

If the frame length is 1522, there will be 1518 bytes transferred to memory. The last byte will be the last data byte.

24.11.4.8.8 Embedded Memories

Table 24-1268. Embedded Memories

Memory Type Description	Number of Instances	
Single-port 2560 × 64 RAM	3	(Packet FIFO's)
Single-port 64-word × 1152-bit RAM	1	(ALE)
Single-port 2048-word × 32-bit RAM	1	(CPPI)

24.11.4.8.9 Flow Control

There are two types of switch flow control: CPPI port flow control and Ethernet port flow control. The CPPI and Ethernet port naming conventions for data flow into and out of the switch are reversed. For the CPPI port (port 0), transmit operations move packets from external memory into the switch and then out to either or both Ethernet transmit ports (ports 1 and 2). CPPI receive operations move packets that were received on either or both Ethernet receive ports to external memory.

24.11.4.8.9.1 CPPI Port Flow Control

The CPPI port has flow control available for transmit (switch ingress). CPPI receive operations (switch egress) do not require flow control. CPPI Transmit flow control is initiated when enabled and triggered. CPPI transmit flow control is enabled by setting the P0_FLOW_EN bit in the [CPSW_FLOW_CONTROL](#) register. CPPI transmit flow control is enabled by default on reset because host packets should not be dropped in any mode of operation.

24.11.4.8.9.2 Ethernet Port Flow Control

The Ethernet ports have flow control available for transmit and receive. Transmit flow control stops the Ethernet port from transmitting packets to the wire (switch egress) in response to a received pause frame. Transmit flow control does not depend on FIFO usage.

The ethernet ports have flow control available for receive operations (packet ingress). Ethernet port receive flow control is initiated when enabled and triggered. Packets received on an ethernet port can be sent to the other ethernet port or the CPPI port (or both). Each destination port can trigger the receive ethernet port flow control. An ethernet destination port triggers another ethernet receive flow control when the destination port is full.

When a packet is received on an ethernet port interface with enabled flow control the below occurs:

- The packet will be sent to all ports that currently have room to take the entire packet.
- The packet will be retried until successful to all ports that indicate they don't have room for the packet.

The flow control trigger to the CPGMAC_SL will be asserted until the packet has been sent, and there is room in the logical receive FIFO for packet runout from another flow control trigger (RX_PKT_CNT = 0). Ethernet port receive flow control is disabled by default on reset. Ethernet port receive flow control requires that the RX_FLOW_EN bit in the associated CPGMAC_SL be set to 1. When receive flow control is enabled on a port, the port's associated FIFO block allocation must be adjusted. The port RX allocation must increase from the default three blocks to accommodate the flow control runout. A corresponding decrease in the TX block allocation is required. If a sending port ignores a pause frame then packets may overrun on receive (and be dropped) but will not be dropped on transmit. If flow control is disabled for G/MII ports, then any packets that are dropped are dropped on transmit and not on receive.

24.11.4.8.9.2.1 Receive Flow Control

When enabled and triggered, receive flow control is initiated to limit the CPGMAC_SL from further frame reception. Half-duplex mode receive flow control is collision based while full duplex mode issues 802.3X pause frames. In either case, receive flow control prevents frame reception by issuing the flow control appropriate for the current mode of operation. Receive flow control is enabled by the RX_FLOW_EN bit in the [SL_MACCONTROL](#) register. Receive flow control is triggered (when enabled) when the RX_FLOW_TRIGGER input is asserted. The CPGMAC_SL is configured for collision or IEEE 802.3X flow control via the FULLDUPLEX bit in the [SL_MACCONTROL](#) register.

24.11.4.8.9.2.1.1 Collision Based Receive Buffer Flow Control

Collision-based receive buffer flow control provides a means of preventing frame reception when the port is operating in half-duplex mode (FULLDUPLEX is cleared in [SL_MACCONTROL](#)). When receive flow control is enabled and triggered, the port will generate collisions for received frames. The jam sequence transmitted will be the twelve byte sequence C3.C3.C3.C3.C3.C3.C3.C3.C3.C3.C3.C3 (hex). The jam sequence will begin no later than approximately as the source address starts to be received. Note that these forced collisions will not be limited to a maximum of 16 consecutive collisions, and are independent of the normal back-off algorithm. Receive flow control does not depend on the value of the incoming frame destination address. A collision will be generated for any incoming packet, regardless of the destination address.

24.11.4.8.9.2.1.2 IEEE 802.3X Based Receive Flow Control

IEEE 802.3x based receive flow control provides a means of preventing frame reception when the port is operating in full-duplex mode (FULLDUPLEX is set in [SL_MACCONTROL](#)). When receive flow control is enabled and triggered, the port will transmit a pause frame to request that the sending station stop transmitting for the period indicated within the transmitted pause frame.

The CPGMAC_SL will transmit a pause frame to the reserved multicast address at the first available opportunity (immediately if currently idle, or following the completion of the frame currently being transmitted). The pause frame will contain the maximum possible value for the pause time (FFFFh). The MAC will count the receive pause frame time (decrements FF00h down to 0) and retransmit an outgoing pause frame if the count reaches zero. When the flow control request is removed, the MAC will transmit a pause frame with a zero pause time to cancel the pause request.

Note that transmitted pause frames are only a request to the other end station to stop transmitting. Frames that are received during the pause interval will be received normally (provided the RX FIFO is not full).

Pause frames will be transmitted if enabled and triggered regardless of whether or not the port is observing the pause time period from an incoming pause frame.

The CPGMAC_SL will transmit pause frames as:

- The 48-bit reserved multicast destination address 01.80.C2.00.00.01.
- The 48-bit source address - from SL_SA[47:0] input.
- The 16-bit length/type field containing the value 88.08
- The 16-bit pause opcode equal to 00.01
- The 16-bit pause time value FF.FF. A pause-quantum is 512 bit-times. Pause frames sent to cancel a pause request will have a pause time value of 00.00.
- Zero padding to 64-byte data length (The CPGMAC_SL will transmit only 64 byte pause frames).
- The 32-bit frame-check sequence (CRC word).

All quantities above are hexadecimal and are transmitted most-significant byte first. The least-significant bit is transferred first in each byte.

If RX_FLOW_EN is cleared to 0 while the pause time is nonzero, then the pause time will be cleared to 0 and a zero count pause frame will be sent.

24.11.4.8.9.2.2 Transmit Flow Control

Incoming pause frames are acted upon, when enabled, to prevent the CPGMAC_SL from transmitting any further frames. Incoming pause frames are only acted upon when the FULLDUPLEX and TX_FLOW_EN bits in the [SL_MACCONTROL](#) register are set. Pause frames are not acted upon in half-duplex mode. Pause frame action will be taken if enabled, but normally the frame will be filtered and not transferred to memory. MAC control frames will be transferred to memory if the RX_CMF_EN (copy MAC frames) bit in the [SL_MACCONTROL](#) register is set. The TX_FLOW_EN and FULLDUPLEX bits effect whether or not MAC control frames are acted upon, but they have no effect upon whether or not MAC control frames are transferred to memory or filtered.

Pause frames are a subset of MAC Control Frames with an opcode field = 0001h. Incoming pause frames will only be acted upon by the port if:

- TX_FLOW_EN is set in [SL_MACCONTROL](#) register, and
- the frame's length is 64 to [SL_RX_MAXLEN](#) bytes inclusive, and
- the frame contains no CRC error or align/code errors.

The pause time value from valid frames will be extracted from the two bytes following the opcode. The pause time will be loaded into the port's transmit pause timer and the transmit pause time period will begin.

If a valid pause frame is received during the transmit pause time period of a previous transmit pause frame then:

- if the destination address is not equal to the reserved multicast address or any enabled or disabled unicast address, then the transmit pause timer will immediately expire, or
- if the new pause time value is zero then the transmit pause timer will immediately expire, else
- the port transmit pause timer will immediately be set to the new pause frame pause time value. (Any remaining pause time from the previous pause frame will be discarded).

If TX_FLOW_EN in [SL_MACCONTROL](#) register is cleared, then the pause-timer will immediately expire.

The port will not start the transmission of a new data frame any sooner than 512-bit times after a pause frame with a non-zero pause time has finished being received (MRXDV going inactive). No transmission will begin until the pause timer has expired (the port may transmit pause frames in order to initiate outgoing flow control). Any frame already in transmission when a pause frame is received will be completed and unaffected.

Incoming pause frames consist of the below:

- A 48-bit destination address equal to:
 - The reserved multicast destination address 01.80.C2.00.00.01
 - , or the CPGMAC_SL SL_SA [47:0] input.
- The 48-bit source address of the transmitting device.
- The 16-bit length/type field containing the value 88.08
- The 16-bit pause opcode equal to 00.01
- The 16-bit pause_time. A pause-quantum is 512 bit-times.
- Padding to 64-byte data length.
- The 32-bit frame-check sequence (CRC word).

All quantities above are hexadecimal and are transmitted most-significant byte first. The least-significant bit is transferred first in each byte.

The padding is required to make up the frame to a minimum of 64 bytes. The standard allows pause frames longer than 64 bytes to be discarded or interpreted as valid pause frames. The CPGMAC_SL will recognize any pause frame between 64 bytes and RX_MAXLEN bytes in length.

24.11.4.8.10 Short Gap

The port 1 (and port 2) transmit inter-packet gap (IPG) may be shortened by eight bit times when enabled and triggered. The TX_SHORT_GAP_EN bit in the [SL_MACCONTROL](#) register enables the gap to be shortened when triggered. The condition is triggered when the port 1 (port 2) transmit FIFO has a user defined number of FIFO blocks used. The port 1 transmit FIFO blocks used determines if the port 1 gap is shortened, and the port 2 transmit FIFO blocks used determines if the port 2 gap is shortened. The [CPSW_GAP_THRESH](#) register value determines the port 1 short gap threshold, and the [CPSW_GAP_THRESH](#) register value determines the port 2 short gap threshold.

24.11.4.8.11 Switch Latency

The CPSW_3G is a store and forward switch. The switch latency is defined as the amount of time between the end of packet reception of the received packet to the start of the output packet transmit.

Table 24-1269. Switch Latency

Mode	Latency
Gig (1000)	880 ns
100	1.3 μ s
10	6.5 μ s

24.11.4.8.12 Emulation Control

The emulation control input (EMUSUSP) and submodule emulation control registers allow CPSW_3G operation to be completely or partially suspended. There are three CPSW_3G submodules that contain emulation control registers (CPGMAC_SL1, CPGMAC_SL2, and CPDMA). The submodule emulation control registers must be accessed to facilitate CPSW_3G emulation control. The CPSW_3G module enters the emulation suspend state if all three submodules are configured for emulation suspend and the emulation suspend input is asserted. A partial emulation suspend state is entered if one or two submodules is configured for emulation suspend and the emulation suspend input is asserted. Emulation suspend occurs at packet boundaries. The emulation control feature is implemented for compatibility with other peripherals.

CPGMAC_SL Emulation Control

The emulation control input (TBEMUSUP) and register bits (SOFT and FREE bits in the [SL_EMCONTROL](#) register) allow CPGMAC_SL operation to be suspended. When the emulation suspend state is entered, the CPGMAC_SL will stop processing receive and transmit frames at the next frame boundary. Any frame currently in reception or transmission will be completed normally without suspension. For receive, frames that are detected by the CPGMAC_SL after the suspend state is entered are ignored. Emulation control is implemented for compatibility with other peripherals.

CPDMA Emulation Control

The emulation control input (TBEMUSUP) and register bits (SOFT and FREE bits in the [CPDMA_EMCONTROL](#) register) allow CPDMA operation to be suspended. When the emulation suspend state is entered, the CPDMA will stop processing receive and transmit frames at the next frame boundary. Any frame currently in reception or transmission will be completed normally without suspension. For transmission, any complete or partial frame in the tx cell FIFO will be transmitted. For receive, frames that are detected by the CPDMA after the suspend state is entered are ignored. No statistics will be kept for ignored frames. Emulation control is implemented for compatibility with other peripherals.

[Table 24-1270](#) shows the operations of the emulation control input and register bits.

Table 24-1270. Emulation Control Input

EMUSUSP	SOFT	FREE	Description
0	X	X	Normal Operation
1	0	0	Normal Operation
1	1	0	Emulation Suspend
1	X	1	Normal Operation

24.11.4.8.13 FIFO Loopback

FIFO loopback mode is entered when the FIFO_LOOPBACK bit in the [CPSW_CONTROL](#) register is set. FIFO loopback mode causes packets received on a port to be turned around and transmitted back on the same port. Port 0 receive is fixed on channel 0 in FIFO loopback mode. The RXSOFOVERRUN statistic is incremented for each packet sent in FIFO loopback mode. Packets sent in with errors are returned with errors (they are not dropped). FIFO loopback is intended as a simple mechanism for test purposes. FIFO loopback should be performed in full duplex mode only.

24.11.4.8.14 Device Level Ring (DLR) Support

Device Level Ring (DLR) support is enabled by setting the DLR_EN bit in the [CPSW_CONTROL](#) register. When enabled, incoming received DLR packets are detected and sent to queue 3 (highest priority) of the egress port(s). If the host port is the egress port for a DLR packet then the packet is sent on the CPDMA Rx channel selected by the P0_DLR_CPDMA_CH field in the [P0_CONTROL](#) register. The supervisor node MAC address feature is supported with the dlr_unicast bit in the unicast address table entry. When set, the dlr_unicast bit causes a packet with the matching destination address to be flooded to the vlan_member_list minus the receive port and minus the host port (the port_number field in the unicast address table entry is a don't care). Matching dlr_unicast packets are flooded regardless of whether the packet is a DLR packet or not. The EN_P0_UNI_FLOOD bit in the [ALE_CONTROL](#) register has no effect on DLR unicast packets. Packets are determined to be DLR packets as shown below:

1. DLR is enabled (DLR_EN is set in the [CPSW_CONTROL](#)).
2. One of the sequences below are true.
 1. The first packet ltype matches [CPSW_VLAN_LTYPE](#)[15:0] VLAN_LTYPE1 and Px_CONTROL[20] Px_VLAN_LTYPE1_EN is set and the second packet ltype matches [CPSW_DLR_LTYPE](#)[15:0] DLR_LTYPE.
 2. The first packet ltype matches [CPSW_VLAN_LTYPE](#)[31:16] VLAN_LTYPE2 and Px_CONTROL[21] Px_VLAN_LTYPE2_EN is set and the second packet ltype matches [CPSW_DLR_LTYPE](#)[15:0] DLR_LTYPE.
 3. The first packet ltype matches [CPSW_VLAN_LTYPE](#)[15:0] VLAN_LTYPE1 and Px_CONTROL[20] Px_VLAN_LTYPE1_EN is set and the second packet ltype matches [CPSW_VLAN_LTYPE](#)[31:16] VLAN_LTYPE2 and Px_CONTROL[21] Px_VLAN_LTYPE2_EN is set and the third packet ltype matches [CPSW_DLR_LTYPE](#)[15:0] DLR_LTYPE.

24.11.4.8.15 Energy Efficient Ethernet Support (802.3az)

NOTE: CM_GMAC_CLKSTCTRL[1:0]CLKTRCTRL bit field must not be programmed for SW_SLEEP or HW_AUTO.

Energy Efficient Ethernet (EEE) allows the PRCM to turn off the module clock during inactive periods as determined by network and host traffic. The module can then be awakened by host queued transmit packet(s) or by a port's external Ethernet PHY. EEE operations are configured as shown below:

1. The 10-bit EEE clock pre-scale value is written to the [CPSW_EEE_PRESCALE](#) register. The pre-scaler is used to clock all EEE-related counters
2. The port Idle to LPI count values (Px_IDLE2LPI) are written with the desired values
3. The port LPI to Wake count values (Px_LPI2WAKE) are written with the desired values
4. The EEE_EN bit is set in the switch [CPSW_CONTROL](#) register
5. Set the [WR_CONTROL](#)[8] SS_EEE_EN bit to enable energy efficient operations at subsystem level.

EEE operation can begin after configuration. The host allows (through PRCM) the CPSW_3G to enter a low power state by asserting the CLKSTOP_REQ signal. There are no requirements on host queues or traffic in order for the host to assert or de-assert CLKSTOP_REQ to the CPSW_3G.

Each ethernet port has a transmit and a receive LPI (low power indicate) state. The receive LPI state is entered when the port's corresponding PHY indicates the LPI state via the CPSW_3G GMII interface. The PHY indicates LPI by asserting MRXER with a MRXD[7:0] value of 0x01 while MRXDV is deasserted (inter-packet gap). The Ethernet transmit port indicates LPI after the Px_IDLE2LPI value has been counted (the transmit port has gone idle for the configured amount of time). If another packet is received for transmit during the count then the count is restarted. When the transmit port has been idle for the Idle to LPI time, the transmit port enters the LPI state and indicates LPI to the associated PHY. The LPI is indicated to the external PHY by an asserted MTXER with a MTXD[7:0] while MTXEN is deasserted (inter-packet gap). The CPDMA LPI state includes transmit and receive. The CPDMA LPI state is entered when the CPDMA transmit and receive have both been idle for the Idle to LPI time ([P0_IDLE2LPI](#)). The Idle to LPI time value for all ports must be large relative to the switch latency to ensure that the count is not able to complete between successive packets.

NOTE: The procedure above is described for the GMII interfaces at the CPSW_3G boundary. External PHY signaling has the following conditions:

- GMII interface is used only in MII mode. Therefore, only MRXD[3:0] and MTXD[3:0] are pinned out
- RGMII is a DDR interface. TXEN and TXER are the sampled values of TX_CTL at the rising and the falling TXC_CLK edges, respectively. RXDV and RXER are the sampled values of RX_CTL at the rising and the falling RXC_CLK edges, respectively
- In RMII mode, EEE is not supported.

When all transmit and receive ports are in the LPI state (CPSW LPI state), the CLKSTOP_ACK signal is asserted, and the PRCM is allowed to stop the module clock. When CLKSTOP_ACK is asserted, the clock may be turned on and off as desired by the host. The host is allowed to restart the clock, perform slave read/write operations to the CPSW_3G memory address space, and then turn off the clock again while CLKSTOP_ACK is asserted.

The software can remove and disable from re-entering the CPSW LPI state by restarting the module clock and then de-asserting CLKSTOP_REQ. The host may queue transmit packets at any time including without regard to the CPSW_3G LPI state (the clock must be restarted in order to write the CPSW_3G slave address space as described above). Host writes to transmit head descriptor pointers will cause the CLKSTOP_WAKEUP signal to be asserted if the CPSW_3G is in the low power state (if CLKSTOP_ACK is asserted).

The external Ethernet PHY's can also wakeup the PRCM by removing the Ethernet receive LPI indication. If the module is in the CPSW Idle state with CLKSTOP_ACK asserted and the receive LPI indication is removed, the CLKSTOP_WAKEUP signal will be asynchronously asserted. On wakeup, the PRCM restarts the clock and de-assert the CLKSTOP_REQ signal. The CLKSTOP_WAKEUP signal will be synchronously deasserted with CLKSTOP_ACK. Upon the deassertion of CLKSTOP_REQ, the Ethernet ports will count the Px_LPI2WAKE time for each port at which time the port is available for transmit. Upon the de-assertion of CLKSTOP_REQ, the CPDMA transmit will count the P0_LPI2WAKE count at which time the CPDMA will begin to send any packets that the host has queued (switch ingress). The wait time on CPDMA transmit is included to preclude the host from filling up the Ethernet port transmit FIFO's while the Ethernet ports are in the LPI to wake time. There is no LPI to wake time on CPDMA receive (switch egress).

24.11.4.8.16 CPSW_3G Network Statistics

The CPSW_3G has a set of statistics that record events associated with frame traffic on selected switch ports. The statistics values are cleared to zero 38 clocks after the rising edge of GMAC_RST. When one or more port enable (Pn_STAT_EN) bits in the CPSW_STAT_PORT_EN register are set, all statistics registers are write to decrement. The value written will be subtracted from the register value with the result being stored in the register. If a value greater than the statistics value is written, then zero will be written to the register (writing 0xFFFF FFFF clears a statistics location). When all port enable bits are cleared to zero, all statistics registers are read/write (normal write direct, so writing 0x0000 0000 clears a statistics location). All write accesses must be 32-bit accesses. In the below statistics descriptions, "the port" refers to any enabled port (with a corresponding set Pn_STAT_EN bit).

The statistics interrupt (STAT_PEND) will be issued if enabled when any statistics value is greater than or equal to 0x8000 0000. The statistics interrupt is removed by writing to decrement any statistics value greater than 0x8000 0000. The statistics are mapped into internal memory space and are 32-bits wide. All statistics rollover from 0xFFFF FFFF to 0x0000 0000.

See [Section 24.11.6.5, STATS Registers](#) for description of every network statistic.

[Table 24-1271](#) and [Table 24-1272](#) summarize network statistics.

Table 24-1271. Rx Statistics Summary

Rx Statistic	Frame /Oct	Rx/Rx +Tx	Frame Type					Frame Size (bytes)								Event				
			MAC control		Data ⁽¹⁾			<64	64	65-127	128-255	256-511	512-1023	1024-rx_maxlen	>rx_maxlen	Flow Coll. ⁽²⁾	CRC Error	Align/Code	Overrun	Addr. Disc.
			Pause frame	Non-pause ⁽³⁾	Multicast	Broadcast	Unicast													
Good Rx Frames	F	R	(y ⁽⁴⁾	y	y	y	y)	n	(y	y	y	y	y	y)	n	- ⁽⁵⁾	n	n	-	n
Broadcast Rx Frames	F	R	(% ⁽⁶⁾	%	n	y)	n	n	(y	y	y	y	y	y)	n	-	n	n	-	n
Multicast Rx Frames	F	R	(%	%	y)	n	n	n	(y	y	y	y	y	y)	n	-	n	n	-	n
Pause Rx Frames	F	R	y	n	n	n	n	n	(y	y	y	y	y	y)	n	-	n	n	-	-
Rx CRC Errors	F	R	(y	y	y	y	y)	n	(y	y	y	y	y	y)	n	-	y	n	-	n
Rx Align/Code Errors	F	R	(y	y	y	y	y)	n	(y	y	y	y	y	y)	n	-	-	y	-	n
Oversized Rx Frames	F	R	(y	y	y	y	y)	n	n	n	n	n	n	n	y	-	n	n	-	n
Rx Jabbers	F	R	(y	y	y	y	y)	n	n	n	n	n	n	n	y	-	(y	y)	-	n
Undersized Rx Frames	F	R	n	n	(y	y	y)	y	n	n	n	n	n	n	n	-	n	n	-	n
Rx Fragments	F	R	n	n	(y	y	y)	y ⁽⁷⁾	n	n	n	n	n	n	n	-	(y	y)	-	-
Rx Overruns ⁽⁸⁾	F	R	(y	y	y	y	y)	(y	y	y	y	y	y	y	y)	-	-	-	y	n
64octet Frames	F	R+T ⁽⁹⁾	(y	y	y	y	y)	n	y	n	n	n	n	n	n	-	-	-	-	n
65-127octet Frames	F	R+T	(y	y	y	y	y)	n	n	y	n	n	n	n	n	-	-	-	-	n
128-255octet Frames	F	R+T	(y	y	y	y	y)	n	n	n	y	n	n	n	n	-	-	-	-	n
256-511octet Frames	F	R+T	(y	y	y	y	y)	n	n	n	n	y	n	n	n	-	-	-	-	n
512-1023octet Frames	F	R+T	(y	y	y	y	y)	n	n	n	n	n	y	n	n	-	-	-	-	n
1024-UPoctet Frames	F	R+T	(y	y	y	y	y)	n	n	n	n	n	n	y	n	-	-	-	-	n
Rx Octets	O	R	(y	y	y	y	y)	n	(y	y	y	y	y	y)	n	-	n	n	-	n
Net Octets	O	R+T	(y	y	y	y	y)	(y	y	y	y	y	y	y	y)	y)	-	-	-	-

⁽¹⁾ The multicast, broadcast and unicast columns in the table refer to non-MAC Control/non-pause frames (i.e. data frames).

⁽²⁾ Flow coll. are half-duplex collisions forced by the MAC to achieve flow-control. A collision will be forced during the first 8 bytes so should not show in frame fragments. Some of the '-'s in this column might in reality be 'n's.

⁽³⁾ The non-pause column refers to all MAC control frames (for example, frames with length/type=88.08) with opcodes other than 0x0001. The pauseframe column refers to MAC frames with the opcode=0x0001.

⁽⁴⁾ "AND" is assumed horizontally across the table between all conditions which form the statistic (marked y or n) except where (y|y), meaning "OR" is indicated. Parentheses are significant.

⁽⁵⁾ "-" indicates conditions which are ignored in the formations of the statistic.

⁽⁶⁾ "%" If either a MAC control frame or pause frame has a multicast or broadcast destination address then the appropriate statistics will be updated.

⁽⁷⁾ "y^" Frame fragments are not counted if less than 8 bytes.

⁽⁸⁾ The rx_overruns stat is for RX_MOF_OVERRUNS and RX_SOF_OVERRUNS added together.

⁽⁹⁾ Statistics marked "R+T" are formed by summing the Rx and Tx statistics, each of which is formed independently.

Table 24-1272. Tx Statistics Summary

Tx Statistic ⁽¹⁾	Frame/Octet	Tx/Rx+Tx	Frame Type					Frame Size (bytes)							Event									
			MAC control ⁽²⁾		Data			64	65-127	128-255	256-511	512-1023	1024-1535	>1535	CRC Error	Collision Type					No Carrier	Queued	Deferred	Underrun
			Pause-MAC	Any-CPU	Multicast	Broadcast	Unicast									Flow ⁽³⁾	1	2-15	16	Late				
Good Tx Frames	F	T	(y ⁽⁴⁾	y	y	y	y	(y	y	y	y	y	y	y	-(⁽⁵⁾	-	-	-	n	n	n	-	-	n
Broadcast Tx Frames	F	T	n	(% ⁽⁶⁾	n	y	n	(y	y	y	y	y	y	y	-	-	-	-	n	n	n	-	-	n
Multicast Tx Frames	F	T	(y	%	y	n	n	(y	y	y	y	y	y	y	-	-	-	-	n	n	n	-	-	n
Pause Tx Frames	F	T	y	n	n	n	n	y	n	n	n	n	n	n	-	-	-	-	-	-	-	-	-	-
Collisions	F	T	n	(y	y	y	y	(y	y	y	y	y	y	y	-	(+ ⁽⁷⁾	+	+	+	+	n	-	-	-
Single Collision Tx Frames	F	T	n	(y	y	y	y	(y	y	y	y	y	y	y	-	-	y	n	n	n	n	-	-	-
Multiple Collision Tx Frames	F	T	n	(y	y	y	y	(y	y	y	y	y	y	y	-	-	n	y	n	n	n	-	-	-
Excessive Collisions	F	T	n	(y	y	y	y	(y	y	y	y	y	y	y	-	-	n	n	y	n	n	-	-	-
Late Collisions	F	T	n	(y	y	y	y	n	(y	y	y	y	y	y	-	-	-	-	-	y	-	-	-	-
Deferred Tx Frames	F	T	n	(y	y	y	y	(y	y	y	y	y	y	y	-	-	n	n	n	n	n	-	y	n
Carrier Sense Errors	F	T	(y	y	y	y	y	(y	y	y	y	y	y	y	-	-	-	-	-	-	y	-	-	-
64octet Frames	F	R+T ⁽⁸⁾	(y	y	y	y	y	y	n	n	n	n	n	n	-	-	-	-	n	n	n	-	-	-
65-127octet Frames	F	R+T	(y	y	y	y	y	n	y	n	n	n	n	n	-	-	-	-	n	n	n	-	-	-
128-255octet Frames	F	R+T	(y	y	y	y	y	n	n	y	n	n	n	n	-	-	-	-	n	n	n	-	-	-

⁽¹⁾ When the transmit Tx FIFO is drained due to the MAC being disabled or link being lost, then the frames being purged will not appear in the Tx statistics.

⁽²⁾ Pause (MAC) frames are issued in the MAC as perfect (no CRC error) 64 byte frames in full duplex only, so they cannot collide.

⁽³⁾ The flow collision type is for half-duplex collisions forced by the MAC to achieve flow control. Some of the '-'s in this column might in reality be 'n's. To prevent double-counting, Net Octets are unaffected by the jam sequence – the 'received' bytes, however, are counted. (See [Table 24-1271](#).)

⁽⁴⁾ "AND" is assumed horizontally across the table between all conditions which form the statistic (marked y or n) except where (y|y), meaning "OR" is indicated. Parentheses are significant.

⁽⁵⁾ "-" indicates conditions which are ignored in the formations of the statistic.

⁽⁶⁾ "%" If a CPU sourced MAC control frame has a multicast or broadcast destination address then the appropriate statistics will be updated.

⁽⁷⁾ "+" indicates collisions which are "summed" (i.e. every collision is counted in the Collisions statistic). Jam sequences used for halfduplex flow control are also counted.

⁽⁸⁾ Statistics marked "R+T" are formed by summing the Rx and Tx statistics, each of which is formed independently.

Table 24-1272. Tx Statistics Summary (continued)

Tx Statistic ⁽¹⁾	Frame/Octet	Tx/Rx+Tx	Frame Type					Frame Size (bytes)							Event									
			MAC control ⁽²⁾			Data		64	65-127	128-255	256-511	512-1023	1024-1535	>1535	CRC Error	Collision Type					No Carrier	Queued	Deferred	Underrun
			Pause-MAC	Any-CPU	Multicast	Broadcast	Unicast									Flow ⁽³⁾	1	2-15	16	Late				
256-511octet Frames	F	R+T	(y	y	y	y	y)	n	n	n	y	n	n	n	-	-	-	-	n	n	n	-	-	-
512-1023octet Frames	F	R+T	(y	y	y	y	y)	n	n	n	n	y	n	n	-	-	-	-	n	n	n	-	-	-
1024-UPoctet Frames	F	R+T	(y	y	y	y	y)	n	n	n	n	n	y	y	-	-	-	-	n	n	n	-	-	-
Tx Octets	O	T	(y	y	y	y	y)	(y	y	y	y	y	y	y)	-	-	-	-	n	n	n	-	-	n
Net Octets	O	R+T	(y	y	y	y	y)	(y	y	y	y	y	y	y)	-	-	\$ ⁽⁹⁾	\$	\$	\$	\$	-	-	-

⁽⁹⁾ "\$" Every byte written on the wire during each retry attempt is also counted in addition to frames which experience no collisions or carrier loss.

24.11.4.9 Static Packet Filter (SPF)

24.11.4.9.1 SPF Overview

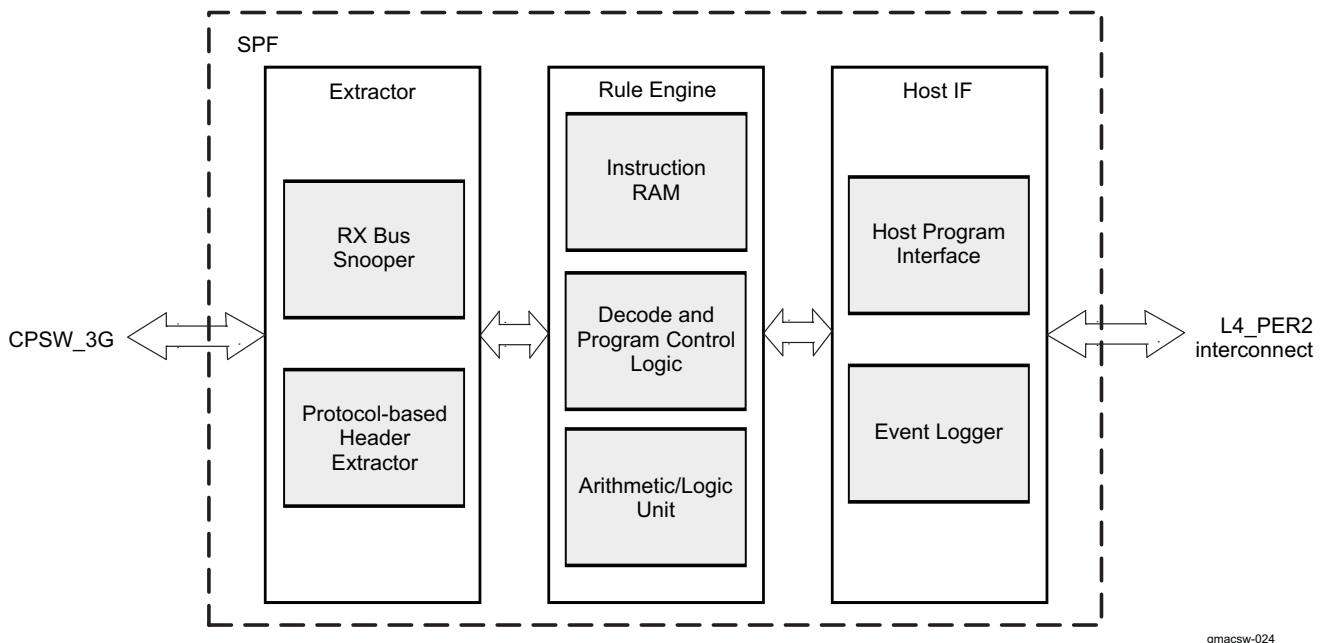
The Static Packet Filter (SPF) provides protection against some of the most common forms of Denial of Service (DoS) attacks that exploit the Layer 3 and Layer 4 functions of the network stack running on a system. The DoS attacks that are mitigated by the SPF module are:

- ARP Flood
- Fragggle
- Illegal Fragment Offset
- Illegal TCP Options
- Jolt2
- Jolt
- Land
- Micro Fragment
- Null Scan
- Ping Flood
- Short ICMP Packet
- Smurf
- SSPIing
- SYN Fragment
- TCP SYN Flood
- Xmas Scan

24.11.4.9.2 SPF Functional Description

24.11.4.9.2.1 SPF Block Diagram

[Figure 24-204](#) shows the block diagram of the Static Packet Filter module. There are three main components of the static packet filter – packet header extractor to decode packet formats, rule engine to check for malformed protocol header fields and a host interface to provide control and configuration access to a host processor and to keep a log of filtered packets in system memory.

Figure 24-204. SPF Block Diagram


The packet parser interfaces to the three-port Ethernet switch (CPSW_3G). The CPSW_3G signals are snooped for receive traffic and the packet contents are provided to the packet header extractor. The parser can decode various network packet formats and determine the location of the header corresponding to each of the specified protocols and store the values in internal registers. The packet parser can decode VLAN, PPPoE, IP, TCP, UDP and ICMP frame formats and determine the location of each protocol header in a frame. The location of each of these headers is used by the rule engine for performing checks against preprogrammed conditions.

The rule engine monitors information in the header fields and upon detection of an abnormal combination of values in these packet header fields, issues an instruction to drop the packet to the external RFIFO interface. The rule engine can also monitor the receive rate of a particular class of packets and can limit the number of packets that actually pass through the system. The operation of rule engine is programmable and that gives flexibility to perform a range of different checks on the contents of packet to determine whether or not it should be accepted.

The event logger captures the activity in the packet filter. In addition, based upon the settings programmed by the host software, the event logger writes detailed information about any frames that have been dropped thus far. This information is written to a part of the system memory as configured by the host software.

24.11.4.9.2.2 Interrupts

The Static Packet Filter has one interrupt that is used to inform the host about excessive number of logged packet drops. The addresses of the instructions (or drop codes) in the instruction memory that cause packet to be dropped are associated with a threshold. When the threshold is met, the packet drop is logged. Each time a packet drop is logged, it is counted. When the number of logged records reaches the value specified by the [SPF_INTCNT](#) register, the host is interrupted. The threshold settings can be controlled by host software to limit the frequency of interrupts. Writing a zero to [SPF_INTCNT](#) disables the interrupt in this scenario. Whenever interrupt is enabled and is triggered, it can be cleared by writing one to either Raw register or Masked register.

Interrupts are controlled by the following registers:

1. Raw register [SPF_INT_RAW](#): Holds the Raw state of interrupt, that is, without mask. The hardware interrupt is latched in this register and is only cleared when a one is written to either Raw register or Masked register. Writing a zero has no effect on this register.
2. Masked register [SPF_INT_MASKED](#): This is the actual interrupt given to the system. It is the result of

bitwise AND operation of Raw and Mask registers. Once the interrupt is sensed by the system, this register should be cleared by writing one to this register or to Raw register during ISR execution. Writing zero has no effect on this register

3. Mask set register [SPF_MASK_SET](#): Writing one to this register enables the interrupt. By default, the interrupt is disabled and needs to be enabled by writing a one to this register. Writing a zero has no effect on this register. Writing to this register also set clear register.
4. Mask clear register [SPF_MASK_CLR](#): Writing one to this register disables interrupt. Writing a zero has no effect on this register.

24.11.4.9.2.3 Protocol Header Extractor

The extractor module has a protocol aware state machine. It decodes the header field in the current protocol header to determine the encapsulating protocol type and extracts various parameters used for software logging. The extractor module also provides the offset to octets in the packet, where Layer 3 and Layer 4 protocol headers start, to the rule engine. These protocol headers correspond to the location of IP and TCP/UDP/ICMP headers from the beginning of the packet.

Ethernet packets with VLAN, PPPoE, IP, IP Options, and ICMP/TCP/UDP protocol are supported by the extractor. When unknown protocols are detected, the extractor skips any additional packet processing. Some examples of how packet parsing is done by the Extractor are given below.

- Ethernet – VLAN - PPPoE – IP – IP Options –TCP/UDP/ICMP – Payload
- Ethernet – VLAN – PPPoE – IP – IP options – Unknown – Payload
- Ethernet – VLAN – PPPoE – IP – TCP/UDP/ICMP – Payload
- Ethernet – VLAN – PPPoE – IP – Unknown – Payload
- Ethernet – VLAN – PPPoE – Fragmented IP – Unknown – Payload
- Ethernet – VLAN – IP – IP Options –TCP/UDP/ICMP – Payload
- Ethernet – VLAN – IP – IP options – Unknown – Payload
- Ethernet – VLAN – IP – TCP/UDP/ICMP – Payload
- Ethernet – VLAN – IP – Unknown – Payload
- Ethernet – VLAN – Fragmented IP – Unknown – Payload
- Ethernet – IP – IP Options –TCP/UDP/ICMP – Payload
- Ethernet – IP – IP options – Unknown – Payload
- Ethernet – IP – TCP/UDP/ICMP – Payload
- Ethernet – IP – Unknown – Payload
- Ethernet – Fragmented IP – Unknown – Payload
- Ethernet – Unknown

The flow of Extractor state machine flow is as follows.

1. Wait for start of packet and proceed to step 2 if start of packet detected.
2. Read Ethernet header Length/Type field (at offset 13). Go to step 3 if next header is VLAN, to step 4 if it is PPP header, to step 5 if it is IP header and to step 8 otherwise.
3. Allow VLAN header to pass and read the Length/Type field. Go to step 4 if next header is PPP, to step 5 if it is IP header and to step 8 otherwise.
4. Read the protocol type in PPP header. Go to step 5 if protocol is IP and to step 8 for unknown protocol
5. Extract source IP address and destination IP address from the IP header. Determine if the packet has IP options or if the packet is fragmented. Go to step 6 if the packet has IP options and to step 8 if the packet is fragmented. If there are no options and the next protocol header is TCP/UDP, go to step 7. Otherwise, skip processing and go to step 8 because the packet has unrecognized protocol header
6. Skip IP options and determines the next protocol header. Go to step 7 if next header is TCP/UDP/ICMP. In case the packet contains an IP fragment or if the protocol in the next header is unknown, skip processing and go to step 8.
7. The packet has TCP/UDP/ICMP header. Extract TCP/UDP source and destination port numbers for logging or ICMP type and code fields and go to step 8.

8. Wait for end of frame and go to step 1 at end of frame.

In case the packet is aborted (indicated by address 0x10 on VBUSP with request and write ready asserted), the extractor state machine flushes its current state, goes back to the idle state and waits for next packet.

The Extractor provides information to rule engine about location of Layer 3 and Layer 4 protocols. When this information is provided to rule engine, the Base Register 1 and Base register 2 are loaded with Layer 3 start offset and Layer 4 start offset respectively. Until the extractor completes decoding of these protocol headers and sends the offset values to rule engine, the rule engine does not execute any instruction that operates on fields in these protocol headers.

Extractor module extracts IP protocol, Source IP address, Destination IP address, TCP/UDP source port, TCP/UDP destination port and ICMP type/code from incoming packet and provides this information to the Host logger module for software reporting. In case IP header or TCP/UDP header does not contain the extractable fields then the corresponding field is logged as zero.

In case packet has unknown Layer 4 protocol or IP packet is fragmented such that the packet does not contain TCP/UDP/ICMP header then TCP/UDP source/destination ports or ICMP type/code fields are logged as zeros.

24.11.4.9.2.4 Programmable Rule Engine

The rule engine is a micro-coded machine that is programmed by the host software. The rule engine is programmed to evaluate various expressions involving the header fields of different protocols that the incoming packet belongs to. These expressions are coded into the instructions that are stored in an internal RAM and are used to check whether a packet is mal-formed or is potentially a DoS attack packet. The rule engine executes the instructions for each incoming packet and makes a decision about accepting or rejecting the packet.

Since each packet can have multiple distinct protocol headers, the location of the headers can be different from packet to packet. The rule engine gets information about the location of the protocol headers from the packet header extractor. The packet header extractor can decode several different protocol types. The octet number at which a particular protocol header is stored is loaded into SPF Base Registers. An application may require SPF to filter packets based on protocols that are not decoded by the extractor. In such cases, SPF allows bypassing the header extractor and then the rule engine can be programmed to analyze packets and figure out the location of each header.

The Rule Engine instructions are programmed before the SPF module is enabled. The instructions cannot be overwritten while the rule engine is processing packets. To modify the contents of code RAM during operation, SPF must be disabled temporarily and then new instructions can be loaded.

Once SPF is enabled, the rule engine starts to fetch instructions one at a time. For each instruction, the operands are obtained from either the packet octets that are being received, from internal registers or from the immediate values inside the instruction itself. If the octet that is needed for execution has not yet been received, then the execution stalls until the required octet is received. In case the operand specified in the instruction refers to a packet octet that has already gone by and is not available in the packet buffer, then the execution stalls until the end of packet. An instruction is executed only when all the required operands are available. Based on the instruction execution results, the packet may immediately be dropped or the results of the evaluated expressions may be stored for future use. In addition, the current instruction can also cause the rule engine to skip a specified number of instructions (immediately following the current instruction) and resume from another location in the instruction RAM.

The rule engine operates on multiple operands and performs multiple tasks in each clock cycle. In each cycle, it can perform one arithmetic and/or logical operation on two pairs of operands. The operands are masked with a 32-bit mask that is generated from the information provided in the instruction. The mask allows for operations that involve variable sized operands. Each operation generates a 32-bit number and a flag bit. The 32-bit number is typically either the sum or the output of bit-wise logical operation. The flag is a single bit result of a comparison operation. The result of each operation can be saved in the internal registers if a specified condition is satisfied. Similarly, depending on the result of the operation, the rule

engine can jump to another location in the instruction memory. The program can instruct the rule engine to either perform two save operations, two conditional jumps or one jump and one save. The conditions must be mutually exclusive to prevent unspecified operation. In addition to providing an alternate path for execution, the instruction can cause the rule engine to accept or reject the current packet and exit from the program until the next packet is received.

When the execution of an instruction does not result in a decision to accept or reject the packet, the rule engine progresses to the next instruction. Instructions are executed until a final decision is made. If a packet is aborted in the middle at the network interface, the rule engine aborts execution and clears all base registers, program counter, octet counter and packet buffer. It is then ready to process next packet.

From a hardware perspective, the rule engine contains a buffer to store packet octets, several internal registers for temporary storage, instruction decoding logic and control circuitry. A description of each of the hardware resources in the rule engine follows.

24.11.4.9.2.4.1 Internal Registers

Several different types of registers are provided in the rule engine. The description of each of these is provided below.

- **Base Registers (B0-B4):** There are four hardware base registers (B1-B4) that store the location of various protocol headers. The B0 register is not a hardware register and it always means a reference to the first octet in a packet. Each base register is eight bits wide and is used to reference octets in a packet. The base registers can point to any octet in the packet upto 255 octets. These registers are readable and writable by the rule engine. In addition, base registers B1 and B2 are loaded with data provided by the extractor when the rule engine is not running in extractor bypass mode ([SPF_CONTROL\[2\] SPF_EXT_BYPASS](#)). The rule engine can write to base registers irrespective of whether extractor bypass is enabled. When an operand needs to use contents of B1-B4 registers, the instruction only executes if the specified base register was loaded at least once after the beginning of current packet. If the specified base register was not loaded during the current packet, then the operand referenced by this base register cannot be extracted from the packet and will cause the rule engine to stall.
- **Constant Registers (C0-C7):** The rule engine can refer to any of eight 32-bit constant values. These are programmed by the host ([SPF_CONSTj](#), j = 0 to 7) and the rule engine only has read access to these registers. Each of these registers can be changed at any time by the host software. However, changing the value of these registers is not recommended as unpredictable behavior may occur if contents are changed while the rule engine is executing instructions that use the C0-C7 values.
- **General Purpose 32-bit registers (R0-R7):** There are eight 32-bit registers that can be read/written by the rule engine. These are general purpose registers and can be used as temporary storage. In addition, when the rule engine is required to provide logging information, R4-R7 are used to store information that will be written to memory.
- **Rate limit registers (L0-L3):** Four 8-bit rate limit registers are used by rule engine to count specific types of packets that are to be rate limited. These registers are counters that are loaded with programmed threshold values ([SPF_RATELIMITi](#), i = 0 to 3) at the end of time interval determined by the clock pre-scaler ([SPF_PRESCALE](#)).
- **General Purpose 1-bit registers (T0-T3):** Four 1-bit registers are provided to store comparison results by the rule engine. These registers can be used to store 1-bit output from ALUs. In addition, the logical OR and logical AND of the flags can also be stored in T0-T3 registers. The 32-bit ALU results may also be stored in T0-T3 registers but only the LSB will be stored.

The rule engine instructions refer to these internal registers during execution. More details about instruction encoding are in subsequent sections.

24.11.4.9.2.4.2 Packet Buffer

When the rule engine is processing a packet, it stored a snapshot of latest 32 octets in an internal buffer. This buffer allows evaluation of expressions on packet octets even after the packet octets have gone by on the external receive VBUSP network interface.

At the start of packet, the first eight octets received are loaded into the first eight locations of the packet buffer. When the next eight octets arrive, these are loaded into the first eight locations and the existing octets are moved to the following eight locations. This process is continued until the packet buffer is full. When packet buffer is full and additional octets arrive, the oldest eight octets are shifted out and are no longer available to the rule engine. Thus, the packet buffer provides a snapshot of most recent octets received.

When the rule engine encounters instructions that reference packet octets which have not yet been received then the rule engine stalls until the required octets are available. Similarly, instructions which require octets that have already been shifted out of the packet buffer will cause rule engine to stall. And since these octets will never be available, the rule engine will not execute any further instructions for the current packet. It will return to the first instruction when the packet ends.

Figure 24-205. Packet Octets as Stored in the Packet Buffer

RFIFO VBUSP	63:56	55:48	47:40	39:32	31:24	23:16	15:8	7:0
	↓	↓	↓	↓	↓	↓	↓	↓
Clock 0	8	7	6	5	4	3	2	1
Clock 8	16	15	14	13	12	11	10	9
	8	7	6	5	4	3	2	1
Clock 16	24	23	22	21	20	19	18	17
	16	15	14	13	12	11	10	9
	8	7	6	5	4	3	2	1
Clock 24	32	31	30	29	28	27	26	25
	24	23	22	21	20	19	18	17
	16	15	14	13	12	11	10	9
	8	7	6	5	4	3	2	1
Clock 40	48	47	46	45	44	43	42	41
	40	39	38	37	36	35	34	33
	32	31	30	29	28	27	26	25
	24	23	22	21	20	19	18	17

gmascw-025

The contents of a packet, when used as one or more operands in an instruction, are always fetched from the packet buffer. The Base registers (B0-B4) in conjunction with an offset and bits are used to specify which octets are to be used as operands.

24.11.4.9.2.5 Intrusion Event Logger

The activity of the static packet filter is logged by the intrusion event logger. Based on the configuration, the event logger writes specific fields of the incoming packet that violate a particular rule to the system memory for software diagnostics.

The logging module allows controlling the frequency at which events are logged to the memory. A set of nine counters are provided and, of these, eight can be mapped to any address of the rule engine's instruction RAM using map registers. This mapping logically associates the log information with the instruction that caused the packet to be dropped. The 9th counter is not associated with any particular instruction and it can be used to count the packets that were dropped at instructions not mapped to any of the other eight counters.

For each of the nine counters, there is a register to store the minimum number of attacks that must be detected before any information is logged to memory. Only when this threshold is met and logging is enabled, the host interface module writes log data in system memory. The memory area used to log is specified by the host in the [SPF_LOG_BEGIN](#) address and [SPF_LOG_END](#) address registers.

The event logger continues to write data to memory until it runs out of space and the log overwrite is disabled. With each update of the log, the [SPF_LOG_HWPTR](#) is updated. This information can be used by the host to determine how much data has been logged. The software in turn keeps the [SPF_LOG_SWPTR](#) updated to inform the SPF about the next address from where information will be read by the host software. It is required that software program correct value of [SPF_LOG_END](#) address to enable hardware to determine roll-over location. The SPF considers all space before the [SPF_LOG_SWPTR](#) as available for logging. In case [SPF_CONTROL\[9\]](#) [SPF_LOGOW_EN](#) control bit is set, SPF ignores [SPF_LOG_SWPTR](#) and logs data irrespective of software read log status. Note that the memory space allocated for logging is a multiple of four 32-bit words and end address [SPF_LOG_END](#) should be loaded with byte address of the next byte following the last log entry. For example if 16 bytes space is allocated for SPF logging from address 0x1400_0000, then [SPF_LOG_BEGIN](#) should be 0x1400_0000 and [SPF_LOG_END](#) should be 0x1400_0010.

The host software must set [SPF_CONTROL\[8\]](#) [SPF_LOG_EN](#) bit to activate logging. The setting of [SPF_LOG_EN](#) bit has no effect on the threshold based counters and they are always active. In addition to the log counters, SPF has one master drop count register ([SPF_DROPCNT](#)) which tracks the total number of packets dropped thus far. This counter does not roll over and must be cleared for re-run by the host processor once it reaches the maximum value.

The format in which packet information is written in the memory by the event logger is shown in [Table 24-1273](#) and [Table 24-1274](#).

Table 24-1273. Format for TCP/UDP Packets

Drop Code	-	-	Protocol
Source IP Address			
Destination IP Address			
Source TCP/UDP Port		Destination TCP/UDP Port	

Table 24-1274. Format for ICMP Packets

Drop Code	-	-	Protocol
Source IP Address			
Destination IP Address			
Type	Code	Checksum	

Each entry logged to memory has a drop code associated with it. The drop code is the address in instruction memory that actually triggered the drop. Up to eight drop codes can be monitored in this manner. In addition to drop code, the protocol, IP addresses and source/destination ports associated with the dropped packet are recorded.

Log data can be supplied by either Extractor module or by Rule engine. The [SPF_CONTROL\[3\]](#) [SPF_RULE_LOG](#) bit must be set to use log data from Rule Engine and cleared to use data supplied by the Extractor.

When logging is done through the Rule Engine, contents of internal registers R4-R7 are written to memory. The format of packet information stored can be programmed in the rule engine except for the drop code field which is static and cannot be changed. The format in which rule engine information is written in the memory by the event logger is shown in [Table 24-1275](#).

Table 24-1275. Rule Engine Format

Drop Code	Register 4[23:0]
	Register 5[31:0]
	Register 6[31:0]

Table 24-1275. Rule Engine Format (continued)

Register 7[31:0]

The rule engine programming must ensure that these registers contain all required information that is to be recorded before the packet drop instruction is executed. As soon as the drop instruction is executed, the logging module starts to send data from the registers R4, R5, R6 and R7 to the system memory and is not possible to modify the contents of these registers.

24.11.4.9.2.6 Rate Limiter

The static packet filter provides a way to limit the rate of specific types of incoming packets. The SPF module provides four rate limit registers (L0-L3) that are used in conjunction with a clock prescaler ([SPF_PRESCALE](#)) and rule engine instructions. The clock prescaler is a clock divider and every count down to zero and the subsequent roll-over indicates end of a time interval. At every such event, each of the four rate limit registers is loaded with a preset value ([SPF_RATELIMIT_i](#), $i = 0$ to 3) that is programmable by the host processor. When packets are received, the rule engine can identify packets of particular types and execute a limit operation. The limit operation specifies a condition (described in [Table 24-1293](#)) and a limit register. If the condition evaluates to true, the packet is dropped if the rate limit register is zero. If it is non-zero, then the rate limit register is decremented by one.

Thus, by using the prescaler and the limit registers, it is possible to control the rate of specific type of packets. By controlling the value of prescale counter (common to all rate limit registers) and the rate limit thresholds, the granularity of the rate can be modified. A lower value of prescale counter will cause frequent reloads of the rate limit registers and will allow rate control over small time intervals. Keeping the prescale counter at extremely small values may cause the rate limiter to be ineffective because the limit registers will be reloaded too frequently to count down to zero. Conversely, a higher value of prescale counter will cause less frequent reloads of the rate limit registers and the rate control will be over longer time intervals. The reload value of each 8-bit rate limit register and the prescale register are programmed by the host processor before SPF is enabled. A reload value of 0xFF can be used to disable any of the rate limiters at run-time without changing the firmware. Both the prescale and limit registers can be modified during run-time.

24.11.4.9.2.7 Rule Engine Instruction Set Architecture

24.11.4.9.2.7.1 Instruction Format

The design supports 64 deep instruction memory. Each of the instructions is 78 bit wide and is stored in a RAM internal to the SPF module. Each instruction can have up to four operands, two arithmetic/logical operations and two conditional jump/save actions based on the outcome of the operations.

The operand field is 52 bits wide and it can have up to four operands. The arithmetic and logical operation codes are four bits each. The operation codes are nine bits each.

Table 24-1276. Instruction Format

Bits	Field Name	Description
77:26	OPERAND	Up to four operands are specified in this field. The source of the operands can be the packet that is being received at the network interface, one of the internal register values or an 8/16/32 bit operand specified within the instruction.
25:22	FUNCTION0	This field specifies a single or dual operand Arithmetic/Logic Function. This function is applied to one or both of first two operands specified in the instruction.
21:18	FUNCTION1	This field also specifies a single or dual operand Arithmetic/Logic Function. This function is applied to one or both of the third and fourth operands specified in the instruction.

Table 24-1276. Instruction Format (continued)

Bits	Field Name	Description
17:9	OPERATION0	A Save/Jump/Limit/Nop operation is specified in this function. Save operation code includes the source and destination information. The save source is the output of functions and the destination is one of the internal registers where data should be saved. Jump operation code has information about a condition and a destination. The jump occurs when the condition, which is based on the result of function0 and function1, is true. The Jump destination controls the flow of instruction execution. Jump to location 1 results in the packet being dropped. Rule engine goes back to initial instruction and waits for next packet. The event logger writes information to memory. Jump to location zero results in the packet being accepted. The rule engine goes back to initial instruction and waits for next packet. Limit operation code has information about a condition and a rate limit register. The Limit operation either causes the packet to be dropped or results in the specified rate limit register to be decremented by one.
8:0	OPERATION1	This field specifies a second save/jump operation. The format for this field is same as for Operation0.

24.11.4.9.2.7.2 Operand Field

The operand field specifies up to four operands. Each operand is obtained from the incoming packet data, from an internal register or as an immediate value specified in the instruction itself. In addition, there is a bit mask associated with each operand. The mask values are encoded in the instruction itself and the mask for each operand is applied before it is used by the Arithmetic and Logic Unit.

The operands are input to the two 32-bit Arithmetic and Logic units. For all calculations, a mask is used with each operand. The mask is a 32-bit number that is generated from a 5-bit code provided in the instruction or from an immediate value in the instruction. The 5-bit mask code specifies how many bits (from the LSB side) will be input to the ALU. An immediate mask must be used when the mask is not a continuous sequence of 1's. Whenever an immediate mask value is specified, the mask value specified by the 'Bits' fields is ignored.

Each operand is at least 13-bits long and the encoding for each operand types is shown in [Table 24-1277](#) to [Table 24-1284](#).

Table 24-1277. Packet Data Operand

Description

This type of operand is derived from the packet itself. The encoding specifies a number of bits (up to 32) to be extracted from a location in the packet indicated by the selected base register and offset. The format of 13-bit operand code is shown below. Internally, a 32-bit number is obtained from the packet. Then, a mask is created from the bits[4:0] field and bitwise ANDed with the 32- bits extracted from the packet.

12	11	10	9	8	7	6	5	4	3	2	1	0
Base[2:0]			Offset[4:0]					Bits[4:0]				

Bits	Field Name	Description
12:10	Base[2:0]	The Base field selects one of the Base registers B0, B1, B2, B3 or B4. The operand comprises of (Bits+1) bits extracted from the packet. The offset from where the operand octets are extracted is determined by the sum of the value of specified base register (Base 0 to Base 4) and the specified offset. Base 0 is start of packet.
9:5	Offset[4:0]	The value of the selected base register plus the specified offset is the octet location in the packet from where the specified number of bits will be picked.
4:0	Bits[4:0]	The number of bits to be used as operand is specified by Bits+1. When Bits[4:0] is zero, only one bit is used as operand. When Bits[4:0] is 31, then 32 bits are used as operand.

Table 24-1278. Constant Operand

Description					One of the eight constants programmed by the host software can be used as operand. The bits field specifies the mask to be applied to the selected 32-bit constant.							
12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	C[2:0]			Bits[4:0]				

Bits	Field Name	Description
7:5	C[2:0]	This field selects one of the constants (C0-C7).
4:0	Bits[4:0]	The number of bits to be used as operand is specified by Bits+1. When Bits[4:0] is zero, the least significant bit is used as operand. When Bits[4:0] is 2, then two LSBs are used as operand and so on.

Table 24-1279. 32-bit Register Operand

Description					One of eight 32-bit registers is used as operand and bits[4:0] field specifies the bitmask to be applied to that register.							
12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	1	R[2:0]			Bits[4:0]				

Bits	Field Name	Description
7:5	R[2:0]	This field selects one of the 32-bits registers (R0-R7).
4:0	Bits[4:0]	The number of bits to be used as operand is specified by Bits+1. When Bits[4:0] is zero, the least significant bit is used as operand. When Bits[4:0] is 2, then two LSBs are used as operand and so on.

Table 24-1280. 1-bit Register Operand

Description					One of the four 1-bit registers is used as operand.							
12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	T[1:0]		0	0	0	0	0
Bits	Field Name				Description							
6:5	T[1:0]				This field selects one of the four 1-bits registers (T0-T3).							

Table 24-1281. Base Register Operand

Description					One of the base registers is used. The 2-bit codes are 00, 01, 10 and 11 for B1 to B4 respectively.							
12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	B[1:0]		0	0	Bits[2:0]		
Bits	Field Name				Description							
6:5	B[1:0]				This field selects one of the four base registers (B1-B4). Note that B0 is a virtual register which is always zero and specifies the start of packet.							
2:0	Bits[2:0]				The number of bits to be used as operand is specified by Bits+1. When Bits[4:0] is zero, the least significant bit is used as operand. When Bits[4:0] is 2, then two LSBs are used as operand and so on.							

Table 24-1282. End of Packet Operand

Description	The End-of-Packet (EOP) operand can be used to detect the end of current packet. Use of this operand will stall the CPU until the end of packet is reached. The end of packet operand will always be equal to one when the CPU detects end of packet. It will be zero until packet data is being received and will also stall the CPU. The recommended use of this operand is in the last instruction where a check may be made for size of current packet. Note that this operand only tracks the end of a complete successfully received packet by the RFIFO. The EOP operand does not detect abort of a packet.														
--------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	0	Bits[4:0]				

Bits	Field Name	Description
4:0	Bits[4:0]	The number of bits specifies the mask. However, these bits are inconsequential for EOP operand. Irrespective of the mask specified by bits[4:0], the EOP operand is always equal to 0x1 in the execution cycle of the instruction it is used.

Table 24-1283. Octet Count Operand

Description	The Octet count operand is used to determine the number of bytes that have been received by rule engine.														
--------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	1	1	Bits[4:0]				

Bits	Field Name	Description
4:0	Bits[4:0]	The number of bits specifies the mask that is applied to octet counter value. The recommend value for this field is 0x1F.

Table 24-1284. Immediate Data Operands

Description	The immediate data operands can be bigger than 13 bits as shown in the codes below. Each of such operands is directly sent to the ALU without applying any addition bit mask. The immediate data operands can be 8, 16 or 32 bits wide.														
--------------------	---	--	--	--	--	--	--	--	--	--	--	--	--	--	--

12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	Data[7:0]							

20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	Data[15:0]															

3	3	3	3	3	3	3	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	9	8	7	6	5	4	3	2	1	0			
6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0										
1	1	1	1	0	Data[31:0]																															

Table 24-1285. Immediate Operand Masks

Description	The operand fields can also be used to specify bit masks. These are used when only specific bits from an operand are to be extracted. There is provision for 8-bit and 16-bit masks that can be encoded into an instruction's operand field.														
--------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	Mask[7:0]							

20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	Mask[15:0]															

When the operand field is all-ones it does not specify any operand and is considered zero.

The tables below list the combinations in which operands may be programmed in an instruction. In this list, Pkt/Reg operand is an operand consisting of octets from the packet or the contents of one of the internal registers; mask specifies the value that is used as a mask instead of the mask generated from the Bits field in the instruction; immediate operand (immediate[7:0], immediate[15:0] and immediate[31:0]) is an 8/16/32 bit number that is used as an operand. The format of Pkt/Reg operand is as shown previously. The decoding of operands for each ALU is dependent upon the type of operation specified in the ALU and the format of the operand field. When there are four operands in the operand field, the first and second operands are used in ALU0 and the rest by ALU1. When ALU0 needs only one operand, then the remaining operands are fed to ALU1. The order of decoding operands is designed to first provide operands to ALU0 from the more significant side of operand field and then to ALU1. If the number of operands specified in instruction does not match with the number of operands required by the ALUs, unspecified behavior can occur. For an instruction to execute, both ALUs must have valid data. If any of the ALUs uses a packet data operand which is not yet available then the rule engine stalls until data is available. During the stall phase, the instruction is decoded every cycle and all required operands must be available simultaneously for the instruction to be executed. In addition, if an instruction refers to packet octets that are no longer available, the rule engine stalls until the end of packet.

Table 24-1286. Operand Field With Four Operands

Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]
---------------------------------------	--	--	--

Table 24-1287. Operand Field With Three Operands

Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Reserved
Pkt/Reg Operand[12:0]	11000 Mask[7:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]
Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt/Reg Operand[12:0]	11000 Mask[7:0]	Pkt / Reg / Imm [7:0] Operand[12:0]
Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Pkt/Reg Operand[12:0]	11000 Mask[7:0]
11101 Immediate[15:0] 11111		Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt / Reg / Imm[7:0] Operand[12:0]
Pkt / Reg / Imm[7:0] Operand[12:0]	11101 Immediate[15:0] 11111		Pkt / Reg / Imm[7:0] Operand[12:0]

Table 24-1288. Operand Field With Two Operands

Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt / Reg / Imm [7:0] Operand[12:0]	Reserved	Reserved
Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt/Reg Operand[12:0]	11000 Mask[7:0]	Reserved
Pkt/Reg Operand[12:0]	11000 Mask[7:0]	Pkt / Reg / Imm[7:0] Operand[12:0]	Reserved

Pkt/Reg Operand[12:0]	11000 Mask[7:0]	Pkt/Reg Operand[12:0]	11000 Mask[7:0]
11101 Immediate[15:0] 11111	Pkt/Reg Operand[12:0]	11000 Mask[7:0]	
Pkt / Reg / Imm[7:0] Operand[12:0]	11101 Immediate[15:0] 11111	Reserved	
Pkt/Reg Operand[12:0]	11000 Mask[7:0]	11101 Immediate[15:0] 11111	
11101 Immediate[15:0] 11111	11101 Immediate[15:0] 11111		
11101 Immediate[15:0] 11111	Pkt / Reg / Imm[7:0] Operand[12:0]	Reserved	
Pkt/Reg Operand[12:0]	11001 Mask[15:0] 11111	Pkt / Reg / Imm[7:0] Operand[12:0]	
Pkt / Reg / Imm[7:0] Operand[12:0]	Pkt/Reg Operand[12:0]	11001 Mask[15:0] 11111	
Pkt/Reg Operand[12:0]	1101 Mask[15:0] 01 Immediate[15:0]		
Pkt / Reg / Imm[7:0] Operand[12:0]	11110 Immediate[31:0] 11		
11110 Immediate[31:0] 11	Pkt / Reg / Imm[7:0] Operand[12:0]		

Table 24-1289. Operand Field With One Operand

Pkt / Reg / Imm[7:0] Operand[12:0]	Reserved	Reserved	Reserved
Pkt/Reg Operand[12:0]	11000 Mask[7:0]	Reserved	Reserved
Pkt/Reg Operand[12:0]	11001 Mask[15:0] 11111	Reserved	
11101 Immediate[15:0] 11111	Reserved	Reserved	
11110 Immediate[31:0] 11	Reserved		

The formats in which operands are specified in the operand field have certain restrictions:

- a mask cannot follow an immediate value
- a mask cannot follow a previous mask
- a reserved field should have all following fields reserved as well
- the first field cannot be a mask

24.11.4.9.2.7.3 Arithmetic/Logical Function Field

There can be two codes for arithmetic and logical functions in each instruction. Each of the operations can involve one or two operands.

The location of each function field in an instruction is listed in [Table 24-1290](#).

Table 24-1290. Arithmetic/Logical Instruction Field

Bits	Description
23:20	Single or Dual operand Arithmetic/Logic Function (function 0)
19:16	Single or Dual operand Arithmetic/Logic Function (function 1)

The bit codes for each arithmetic/logic function that can be performed on the operands of an instruction are listed in [Table 24-1291](#).

Table 24-1291. Arithmetic/Logical Operation Codes

Code	Description	Operands Required	32-bit result	1-bit flag result
0x0	Less Than	2	0	Set if true
0x1	Less Than Equal To	2	0	Set if true
0x2	Greater Than	2	0	Set if true
0x3	Greater Than Equal To	2	0	Set if true
0x4	Equal to Zero	1	0	Set if true
0x5	Not Equal to Zero	1	0	Set if true
0x6	Equal to One	1	0	Set if true
0x7	Not Equal to One	1	0	Set if true
0x8	Equal	2	0	Set if true
0x9	Not Equal	2	0	Set if true
0xA	Add	2	Sum	Set if overflow
0xB	Subtract	2	Difference	Set if output is negative
0xC	Bitwise AND	2	Bitwise AND	0
0xD	Bitwise OR	2	Bitwise OR	0
0xE	Result is same as Operand	1	Same as operand	0
0xF	No Function (NOF)	0	0	0

The ALU0 and ALU1 are assigned function0 and function1 respectively. In case only one function is to be performed in an instruction, function0 field should be used to do that. An instruction with no function for ALU0 and a valid function for ALU1 will result in unspecified behavior.

The output of each function is in the form of a 32-bit number and a single bit value. The single bit value is either the carry bit from an add/subtract operation or the Boolean result from a comparison function. The single bit output from each function is referred as the flag0 or flag1 result. The 32-bit number is the outcome of arithmetic or a bitwise logical operation and it is referred to as the word0 or word1 result.

24.11.4.9.2.7.4 Operation Field

The operation field specifies a limit, save or a jump operation. All of these operations involve two arguments that are specified within the instruction.

In case of limit operation, one of the arguments is a condition and the other argument is a limit register. In case of save operation, one of the arguments is source data and the other argument is a destination where the data will be saved. In jump operation, the first argument is a condition that must be true for the jump to occur and the second argument is a destination to which the program control will be moved to. The destination argument in a jump operation also specifies specific destinations that lead to the current packet being immediately rejected or accepted.

The location of operation field in an instruction is shown in [Table 24-1292](#).

Table 24-1292. Operation Fields

Bits	Description
17:9	Limit/Save/Jump Operation (operation 0) For limit operation, [17:16] is the operation field, [15:13] is the condition field and [12:9] is a limit register. For save operation, [17:16] is the operation field, [15:13] is the source field, and [12:9] is the destination field. For jump operation, [17:16] is the operation field, [15:13] is the condition field, and [12:9] is the destination field
8:0	Limit/Save/Jump Operation (operation 1) For limit operation, [8:7] is the operation field, [6:4] is the condition field and [3:0] is a limit register. For save operation, [8:7] is the operation field, [6:4] is the source field, and [3:0] is the destination field. For jump operation, [8:7] is the operation field, [6:4] is the condition field, and [3:0] is the destination field.

Table 24-1293. Limit Operation

Description	In a limit operation, the condition argument specifies a condition that must be true for the specified limit register to be decremented if non-zero or the packet to be dropped if the limit register is zero.
--------------------	--

8	7	6	5	4	3	2	1	0
0	0		Condition Code					Limit Register

Bits	Field Name	Description
6:4	Condition Code	It is a 3-bit code that specifies the condition that must be true for the operation to be executed. 0x0: Single bit flag (flag0 from function 0) 0x1: Single bit flag (flag1 from function 1) 0x2: Logical OR of flags (flag0 flag1) 0x3: Logical NOR of flags (!(flag0 flag1)) 0x4: Logical AND of flags (flag0 & flag1) 0x5: Logical NAND of flags (!(flag0 & flag1)) 0x6: XOR of flags (flag0 ^ flag1) 0x7: Unconditional Decrement/Limit
3:0	Limit Register	It is a 4-bit code that specifies the limit register that is to be decremented or checked for zero value. 0x0: Rate limit register L0 0x1: Rate limit register L1 0x2: Rate limit register L2 0x3: Rate limit register L3 0x4-0xF: Reserved

Table 24-1294. Save Operation

Description	The arguments for a save operation include a code for source data and a code for the destination. The source is either a single bit data or a 32-bit word. The single bit data is either a flag from the functions or a logical OR/AND of the flags. The 32-bit word is the result of the ALU arithmetic/logical functions.
--------------------	---

8	7	6	5	4	3	2	1	0
0	1		Source Data Code					Destination Register Code

Bits	Field Name	Description
6:4	Source Data Code	It is a 3-bit code that specifies the source data which is to be save in another register 0x0: Single bit flag (flag0 from function 0) 0x1: Single bit flag (flag1 from function 1) 0x2: Logical OR of flags (flag0 flag1) 0x3: Logical AND of flags (flag0 & flag1) 0x4: Output of function (word0) 0x5: Output of function (word1) 0x6-0x7: Reserved
3:0	Destination Register Code	It is a 4-bit code that specified the register in which source data is to be stored 0x0-0x3: Single Bit register (T0-T3) 0x4-0x7: Base Register (B1-B3) 0x8-0xF: 32-bit Register (R0-R7)

For some instructions, the rule engine extracts the operand from the current packet. To determine the packet bits to be used as operands, it refers to one or more base registers. Whenever an instruction causes a change in value of any of the base registers, it cause the rule engine to wait for one clock cycle before executing the next instruction. The rule engine is a pipelined processor and change in the value of a base register causes the prefetched operand values to become stale and these need to be fetched again. The clock cycle inserted allows rule-engine to fetch the data again before the instruction using this data is executed. Only the instructions that use packet data will be delayed by a clock cycle if the previous instruction caused a change in the base register that is referenced in the current instruction.

Table 24-1295. Jump Operation

Description		For jump operation, one of the operands is a condition that must evaluate to true for the jump operation to occur. The other operand is the destination to which the program counter will move to if the jump operation is executed. The condition in a jump operation is either one of the flags from the output of the arithmetic/logic unit or a logical AND/OR of the flags.							
8	7	6	5	4	3	2	1	0	
1	0	Condition Code				Destination Code			

Bits	Field Name	Description
6:4	Condition Code	Condition Code 0x0: Single bit flag (flag0 from function 0) 0x1: Single bit flag (flag1 from function 1) 0x2: Logical OR of flags (flag0 flag1) 0x3: Logical NOR of flags (!(flag0 flag1)) 0x4: Logical AND of flags (flag0 & flag1) 0x5: Logical NAND of flags (!(flag0 & flag1)) 0x6: XOR of flags (flag0 ^ flag1) 0x7: Unconditional Jump
3:0	Destination Code	Destination Code 0x0: Accept packet, return to instruction zero and wait for the next packet. 0x1: Reject packet and return to instruction zero. 0x2-0xF: Go to instruction at offset +2 ...+15

The jump operation results in a change of program execution flow. The jump destination determines the next instruction that will be executed by the rule engine. The destination is only specified as a positive offset to the current value of the program counter. The program counter can result in the rule engine skipping a given number of instructions instead of executing the next instruction. In addition, the destination field can instruct the rule engine to immediately reject or accept the current packet without any further checks. The rule engine then goes back to the first instruction and waits for a new packet to arrive before it executes any further instructions.

The rule engine is a pipelined processor. When it executes a jump operation, there is a delay of one clock cycle before the next instruction is executed.

The operations specified in the two operation fields are independent of each other. However, it is possible that conflicting operations are specified in the operation fields of an instruction. When conflicting operations are specified and conditions of both operations evaluate to true; then the execution flow is determined as described below:

1. If one of the ALUs has a limit/jump operation that will cause the packet to be dropped, then the packet will be dropped irrespective of what the other ALU indicates.
2. If case 1 above is false and one of the ALUs has a jump operation that will cause the packet to be accepted, then the packet will be accepted and the rule engine will return to idle state.
3. If case 1 and 2 above are false and one of the ALUs has a jump to offset with the respective condition true, the jump will be executed. If there is conflicting jump to offset information in the two ALUs, ALU0 is given priority.

Table 24-1296. No Operation

Description		If there is no Save, Jump or Limit operation specified in the operation field, it is interpreted as a no operation. There is no change in execution flow, no change in any register values or rate limit registers when a no operation is encountered.							
8	7	6	5	4	3	2	1	0	
1	1	Reserved			Reserved				

It is expected that the rule engine will be programmed such that a decision to accept or discard the current packet is made before all instructions have executed. If the rule engine reaches the last instruction and it does not result in a decision to accept/reject the packet, then the execution flow does not stop and all instructions from the first instruction are executed again until the end of packet is reached.

24.11.4.9.3 Programming Guide

24.11.4.9.3.1 Initialization Routine

The packet filter must be initialized in order for it to operate as specified. A typical order in which the module can be initialized is shown below:

1. Initialize the firmware by programming the internal memory and verify it (SPF_INSTR_W2, SPF_INSTR_W1, SPF_INSTR_W0, [SPF_INSTR_CTL](#))
2. Allocate memory for logs written by SPF and initialized the log space parameters ([SPF_LOG_BEGIN](#), [SPF_LOG_END](#), SPF_SW_PTR)
3. Program the log map registers to associate drop codes with the log thresholds. ([SPF_LOG_MAP0](#), [SPF_LOG_MAP1](#))
4. Program log thresholds ([SPF_LOG_THRESHk](#), where k = 0 to 8)
5. Program constant registers if required by the firmware ([SPF_CONSTj](#), where j = 0 to 7)
6. Program clock prescale counters and rate limit registers if required by the firmware ([SPF_PRESCALE](#), [SPF_RATELIMITi](#), where i = 0 to 3)
7. Program the interrupt frequency control register and interrupt mask register to setup interrupts ([SPF_INTCNT](#), [SPF_MASK_SET](#))
8. Program the control registers to enable SPF filter and logger ([SPF_CONTROL](#)).

24.11.4.9.3.2 Interrupt Service Routine

The interrupt service routine in SPF should be designed to process the data logs generated by SPF. It should trigger a higher layer application software that can analyze the data logs and determine if any remedial measures are required based on the information in the logs. At the minimum, the interrupt service routine must reprogram the software pointer for the logging machine to be able to continue logging.

The typical tasks performed following an interrupt are listed in the pseudo code below.

```
SPF_ISR {
    read log hardware pointer
    read log software pointer
    determine how many log entries are to be read
    read each log entry starting with the software pointer
    update software pointer to reflect the next unread entry
    clear interrupt
}
```

24.11.4.9.3.3 Rule Engine Example Program

Here is an example of how the rule engine can be programmed to detect packets that resemble Denial of Service traffic. The pseudo code is shown below.

```
IDLE:
    jump to ICMP if start_of_packet
```

```

ICMP:
if (protocol==ICMP)
limit_ICMP
if (fragmented packet)
drop and jump to IDLE
accept and jump to IDLE
else
jump to IP
IP:
if (source_ip==dest_ip)
drop and jump to IDLE
if (fragmented and (fragment_offset+ip_size)>2^16)
drop and jump to IDLE
accept the packet and jump to IDLE

```

24.11.4.10 Common Platform Time Sync (CPTS)

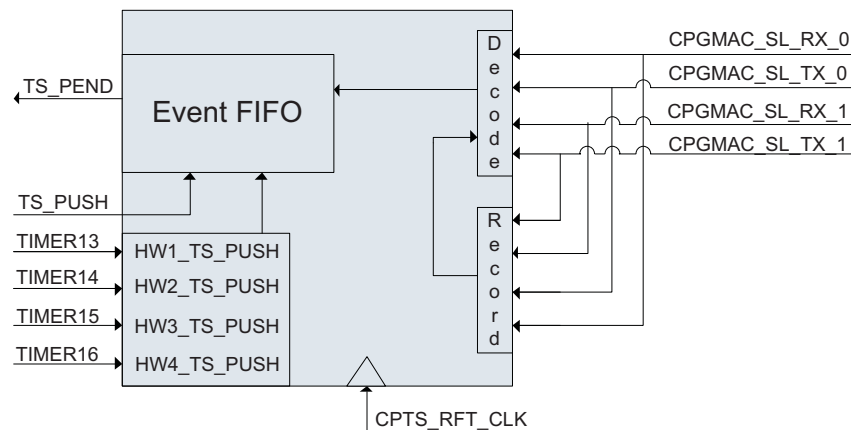
The Common Platform Time Sync (CPTS) module is used to facilitate host control of time sync operations. It enables compliance with the IEEE 1588-2008 standard for a precision clock synchronization protocol.

24.11.4.10.1 CPTS Architecture

Figure 24-206 shows the architecture of the CPTS module inside the GMAC_SW Ethernet Subsystem. Time stamp values for every packet transmitted or received on either port of the GMAC_SW are recorded. At the same time, each packet is decoded to determine if it is a valid time sync event. If so, an event is loaded into the Event FIFO for processing containing the recorded time stamp value when the packet was transmitted or received.

In addition, both hardware (HWx_TS_PUSH) and software (TS_PUSH) can be used to read the current time stamp value through the Event FIFO

Figure 24-206. CPTS Block Diagram



The reference clock used for the time stamp (CPTS_RFT_CLK) can be derived from several sources. See [Chapter 3, PRCM](#) for more details.

24.11.4.10.2 CPTS Initialization

The CPTS module should be configured as:

1. Reset the CPTS module.
2. Clear the CPTS_EN bit in the [CPTS_CONTROL](#) register.
3. Write the CLKSEL_RFT value in the CM_GMAC_GMAC_CLKCTRL register in the PRCM with the desired reference clock selection.
4. Set the CPTS_EN bit in the [CPTS_CONTROL](#) register.

5. If using interrupts and not polling, enable the interrupt by setting the TS_PEND_EN bit in the [CPTS_INT_ENABLE](#) register.

24.11.4.10.3 Time Stamp Value

The time stamp value is a 32-bit value that increments on each CPTS_RFT_CLK rising edge when CPTS_EN is set to 1. When CPTS_EN is cleared to 0, the time stamp value is reset to 0.

If more than 32-bits of time stamp are required by the application, the host software must maintain the necessary number of upper bits. The upper time stamp value should be incremented by the host when the rollover event is detected.

For test purposes, the time stamp can be written via the time stamp load function ([CPTS_TS_LOAD_VAL](#) and [CPTS_TS_LOAD_EN](#) registers).

24.11.4.10.4 Event FIFO

All time sync events are push onto the Event FIFO. There are 16 locations in the event FIFO with no overrun indication supported. Software must service the event FIFO in a timely manner to prevent FIFO overrun.

24.11.4.10.5 Time Sync Events

Time Sync events are 64-bit values that are pushed onto the event FIFO and read in two 32-bit reads. Two 32-bit registers, [CPTS_EVENT_HIGH](#) and [CPTS_EVENT_LOW](#) hold the data of a time sync event. There are five types of sync events:

- Time stamp push event
- Time stamp counter rollover event
- Time stamp counter half-rollover event
- Ethernet receive event
- Ethernet transmit event

24.11.4.10.5.1 Time Stamp Push Event

Software can obtain the current time stamp value (at the time of the write) by initiating a time stamp push event. The push event is initiated by setting the TS_PUSH bit of the [CPTS_TS_PUSH](#) register. The time stamp value is returned in the event, along with a time stamp push event code. Software should not push a second time stamp event on to the FIFO until the first time stamp value has been read from the event FIFO.

24.11.4.10.5.2 Time Stamp Counter Rollover Event

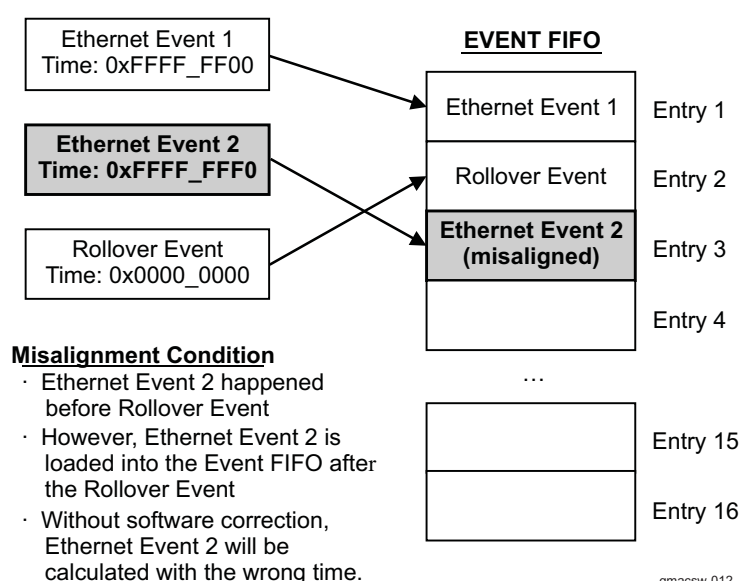
The CPTS module contains a 32-bit time stamp value. The counter upper bits are maintained by host software. The rollover event indicates to software that the time stamp counter has rolled over from FFFF FFFFh to 0000 0000h, and the software maintained upper count value should be incremented.

24.11.4.10.5.3 Time Stamp Counter Half-rollover Event

The CPTS includes a time stamp counter half-rollover event. The half-rollover event indicates to software that the time stamp value has incremented from 7FFF FFFFh to 8000 0000h. The half-rollover event is included to enable software to correct a misaligned event condition. The half-rollover event is included to enable software to determine the correct time for each event that contains a valid time stamp value, such as an Ethernet event. If an Ethernet event occurs around a counter rollover (full rollover), the rollover event could possibly be loaded into the event FIFO before the Ethernet event, even though the Ethernet event time was actually taken before the rollover. [Figure 24-207](#) shows a misalignment condition.

Host software must detect and correct for misaligned event conditions. For every event after a rollover and before a half-rollover, software must examine the time stamp most significant bit. If bit 31 of the time stamp value is low (0000 0000h through 7FFF FFFFh), then the event time stamp was taken after the rollover and no correction is required. If the value is high (8000 0000h through FFFF FFFFh), the time stamp value was taken before the rollover and a misalignment is detected. The misaligned case indicates to software that it must subtract one from the upper count value stored in software to calculate the correct time for the misaligned event. The misaligned event occurs only on the rollover boundary and not on the half-rollover event. Software only needs to check for misalignment from a rollover event to a half-rollover event.

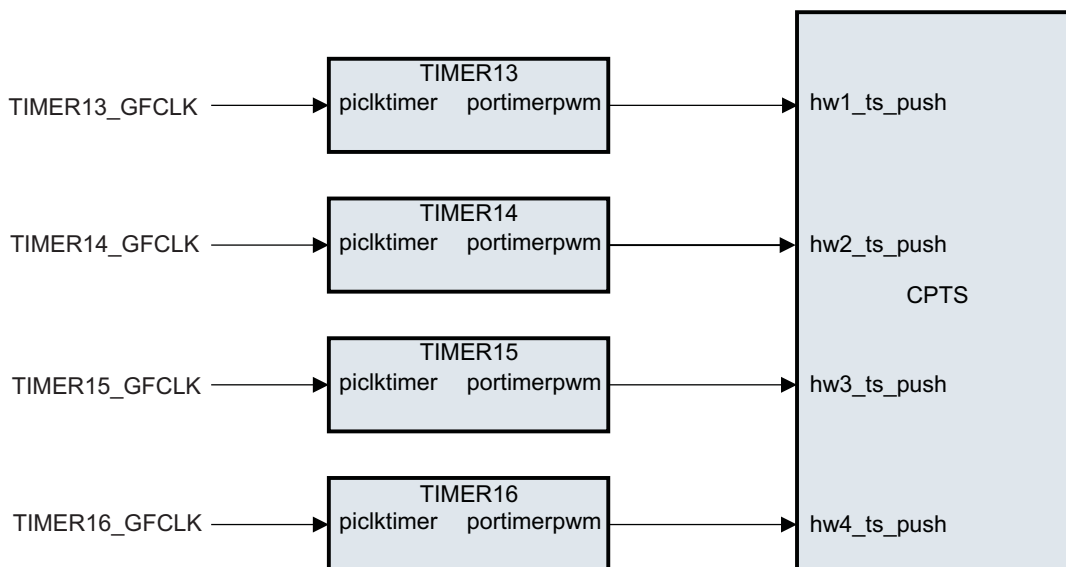
Figure 24-207. Event FIFO Misalignment Condition



24.11.4.10.5.4 Hardware Time Stamp Push Event

There are four hardware time stamp inputs (HW1/4_TS_PUSH) that can cause hardware time stamp push events to be loaded into the Event FIFO. Each hardware time stamp input is internally connected to the PORTIMERPWM output of each timer as shown in [Figure 24-208](#).

Figure 24-208. HW1/4_TSP_PUSH Connection



The event is loaded into the event FIFO on the rising edge of the timer, and the `PORT_NUMBER` field in the `CPTS_EVENT_HIGH` register indicates the hardware time stamp input that caused the event.

Each hardware time stamp input must be asserted for at least 10 periods of the selected RCLK clock. Each input can be enabled or disabled by setting the respective bits in the `CPTS_CONTROL` register.

Hardware time stamps are intended to be an extremely low frequency signals, such that the event FIFO does not overrun. Software must keep up with the event FIFO and ensure that there is no overrun, or events will be lost.

24.11.4.10.5.5 Ethernet Port Events

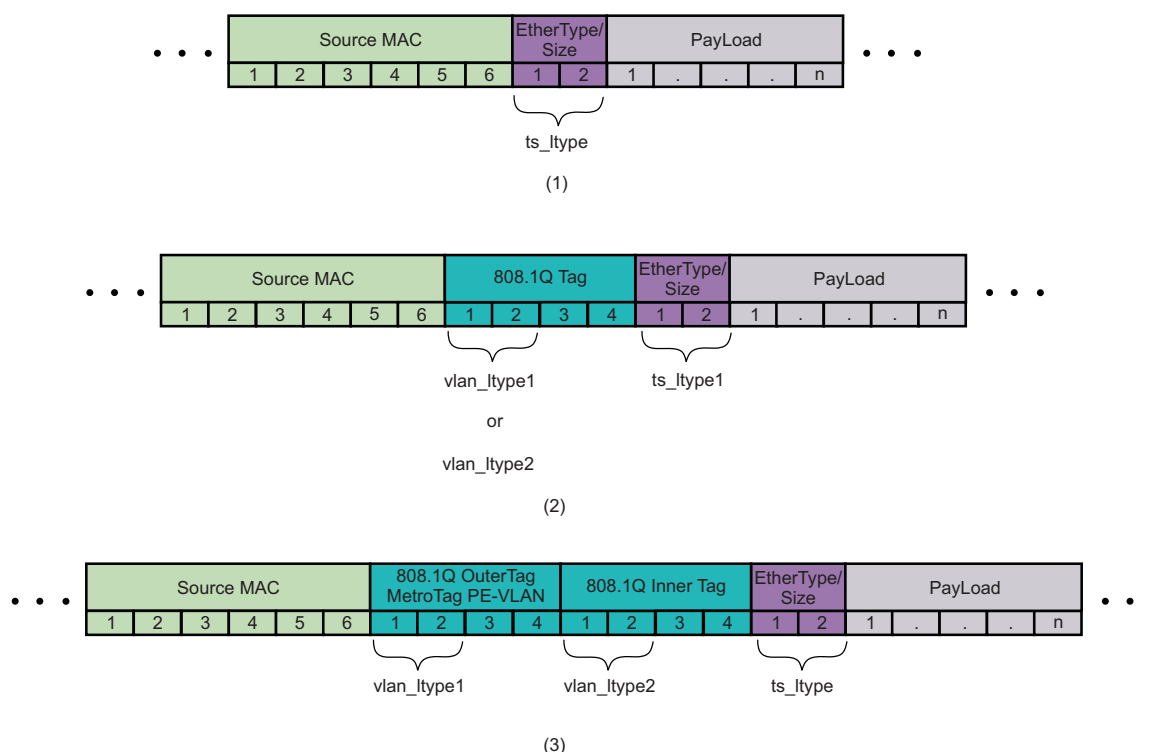
Packets transmitted or received on each Ethernet port can generate Ethernet Transmit Events or Ethernet Receive Events, respectively. The CPTS hardware will decode each packet to determine if it is a valid CPTS time sync event.

According to the IEEE 802.3 Ethernet standard, each Ethernet frame contains a 2-octet EtherType field to indicate which protocol is encapsulated in the Payload field, as shown in Figure 24-209. For standard time sync packets, this will contain the EtherType for the Precision Time Protocol (IEEE 1588), which is defined as 0x88F7. The CPTS hardware will compare this field to the `TS_LTYPE1` field or the `TS_LTYPE2` field (depending on which enable bit was set) in the `CPSW_TS_LTYPE` register, which should also be programmed to 88F7h.

When a virtual LAN is used, an additional 4-octet 802.1Q tag is inserted in the Ethernet frame before the EtherType field, as shown in Figure 24-209. To indicate to the CPTS hardware that a virtual LAN is in use, the `Pn_TS_VLAN_LTYPE1_EN` (or `Pn_TS_VLAN_LTYPE2_EN`) enable bit must be set in the `Pn_CONTROL` register. The EtherType for the 802.1Q tag is defined as 0x8100, and the CPTS hardware will compare this value to the `VLAN_LTYPE1` (or `VLAN_LTYPE2` depending on which enable bit was set) field in the `CPSW_VLAN_LTYPE` register, which should also be programmed to 0x8100.

When two stacked VLANs are used, two additional 4-octet 801.Q tags are inserted in the Ethernet frame before the EtherType field, as shown in Figure 24-209. In this case, both `VLAN_LTYPE1` and `VLAN_LTYPE2` must be enabled. The outer tag must match the value of the `VLAN_LTYPE1` field, and the inner tag must match the value of the `VLAN_LTYPE2` field.

Figure 24-209. Partial Ethernet-II Frames Showing Register Mapping of EtherTypes for a Simple Frame (1), a Single 1Q Tag Added (2), and Two 1Q Tags Added (3)



To enable transmit/receive event packets on a given Ethernet port, perform the following steps, where n is the port number:

- Set the [1] `Pn_TS_TX_EN` or the [0] `Pn_TS_RX_EN` bit in the `Pn_CONTROL` register to enable transmit/receive event packets
- Configure the register fields as follows:
 - Set the `Pn_CONTROL`[2] `Pn_TS_LTYPE1_EN` or [3] `Pn_TS_LTYPE2_EN` bit to 1
 - Set the `CPSW_TS_LTYPE`[15:0] `TS_LTYPE1` or [31:16] `TS_LTYPE2` (depending on which enable bit was set in 2(a)) field to 88F7h, which corresponds to the Precision Time Protocol (IEEE 1588) EtherType.
 - Set the `Pn_TS_SEQ_MTYPE`[21:16] `Pn_TS_SEQ_ID_OFFSET` field to 1Eh, which is the sequence ID offset in the common message header given in the IEEE 1588 specification.
- To enable support for VLAN tagging (IEEE 802.1Q):
 - Set the [20] `Pn_TS_VLAN_LTYPE1_EN` or [21] `Pn_TS_VLAN_LTYPE2_EN` bit in the `Pn_CONTROL` register.
 - Set the `CPSW_VLAN_LTYPE`[15:0] `VLAN_LTYPE1` or [31:16] `VLAN_LTYPE2` field (matching what was used in step 3a) to 8100h, which corresponds to the VLAN-tagged frame (IEEE 802.1Q) EtherType.
- To enable support for up to two stacked VLANs (IEEE 802.1ad):
 - Set the `Pn_TS_VLAN_LTYPE1_EN` and `Pn_TS_VLAN_LTYPE2_EN` bits in the `Pn_CONTROL` register.
 - Set the `VLAN_LTYPE1` field in the `CPSW_VLAN_LTYPE` register to match the EtherType of the outer tag.
 - Set the `VLAN_LTYPE2` field in the `CPSW_VLAN_LTYPE` register to 8100h, which corresponds to the VLAN-tagged frame (IEEE 802.1Q) EtherType of the inner tag.
- Set the `Pn_TS_SEQ_MTYPE`[15:0] `Pn_TS_MSG_TYPE_EN` field to choose which types of time stamp messages will push events onto the Event FIFO. Table 9-20 lists the message types defined in the

IEEE 1588-2008 specification. [Table 24-1297](#) lists the message types defined in the IEEE 1588-2008 specification.

Table 24-1297. Values of Message Type Field

Message Type	Value (hex)
Sync	0
Delay_Req	1
Pdelay_Req	2
Pdelay_Resp	3
Reserved	4-7
Follow_Up	8
Delay_Resp	9
Pdelay_Resp_Follow_Up	A
Announce	B
Signaling	C
Management	D
Reserved	E-F

Once a transmitted or received packet is determined to be a valid time sync packet, the Ethernet Transmit Event or Ethernet Receive Event is loaded onto the Event FIFO. The [CPTS_EVENT_HIGH](#) register contains the Message Type and Sequence ID values from the original time sync packet. The [CPTS_EVENT_LOW](#) register contains the time stamp value when the packet arrived at the corresponding port.

24.11.4.10.6 CPTS Interrupt Handling

When an event is push onto the Event FIFO, an interrupt can be generated to indicate to software that a time sync event occurred. The following steps should be taken to process time sync events using interrupts:

1. Enable the TS_PEND interrupt by setting the TS_PEND_EN bit of the [CPTS_INT_ENABLE](#) register.
2. Upon interrupt, read the [CPTS_EVENT_LOW](#) and [CPTS_EVENT_HIGH](#) registers values.
3. Set the EVENT_POP field (bit 0) of the [CPTS_EVENT_POP](#) register to pop the previously read value off of the event FIFO.
4. Process the interrupt as required by the application software.

Software has the option of processing more than a single event from the event FIFO in the interrupt service routine in the following way:

1. Enable the TS_PEND interrupt by setting the TS_PEND_EN bit of the [CPTS_INT_ENABLE](#) register.
2. Upon interrupt, read the [CPTS_EVENT_LOW](#) and [CPTS_EVENT_HIGH](#) registers values.
3. Set the EVENT_POP bit of the [CPTS_EVENT_POP](#) register to pop the previously read value off of the event FIFO.
4. Wait for an amount of time greater than four CPTS_RFT_CLK periods plus four MAIN_CLK periods.
5. Read the TS_PEND_RAW bit in the [CPTS_INTSTAT_RAW](#) register to determine if another valid event is in the event FIFO. If it is asserted, go to step 2; otherwise, go to step 6.
6. Process the interrupt(s) as required by the application software.

Software also has the option of disabling the interrupt and polling the TS_PEND_RAW bit of the [CPTS_INTSTAT_RAW](#) register to determine if a valid event is on the event FIFO.

24.11.4.11 CPPI Buffer Descriptors

The buffer descriptor is a central part of the GMAC_SW Ethernet Subsystem and is how the application software describes Ethernet packets to be sent and empty buffers to be filled with incoming packet data.

Host Software sends and receives network frames via the CPPI compliant host interface. The host interface includes module registers and host memory data structures. The host memory data structures are buffer descriptors and data buffers. Buffer descriptors are data structures that contain information about a single data buffer. Buffer descriptors may be linked together to describe frames or queues of frames for transmission of data and free buffer queues available for received data.

NOTE: The 8K bytes of Ethernet Subsystem CPPI RAM begin at address 0x4848 6000 and end at 0x4848 7FFF from the GMAC_SW perspective. The buffer descriptors programmed to access the CPPI RAM memory should use this address range.

24.11.4.11.1 TX Buffer Descriptors

A TX buffer descriptor (Table 24-1298) is a contiguous block of four 32-bit data words aligned on a 32-bit word boundary.

Table 24-1298. TX Buffer Descriptor Format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Next Descriptor Pointer																															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Buffer Pointer																															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Buffer Offset																Buffer Length															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SOP	EOP	OWNER	EOQ	TDOWNCMPLT	PASSCRC	RESERVED						TO_PORT_EN	RESERVED	TO_PORT	RESERVED						Packet Length										

24.11.4.11.1.1 CPPI TX Data Word 0

Next Descriptor Pointer

The next descriptor pointer points to the 32-bit word aligned memory address of the next buffer descriptor in the transmit queue. This pointer is used to create a linked list of buffer descriptors. If the value of this pointer is zero, then the current buffer is the last buffer in the queue. The software application must set this value prior to adding the descriptor to the active transmit list. This pointer is not altered by the EMAC. The value of pNext should never be altered once the descriptor is in an active transmit queue, unless its current value is NULL. If the pNext pointer is initially NULL, and more packets need to be queued for transmit, the software application may alter this pointer to point to a newly appended descriptor. The EMAC will use the new pointer value and proceed to the next descriptor unless the pNext value has already been read. In this latter case, the transmitter will halt on the transmit channel in question, and the software application may restart it at that time. The software can detect this case by checking for an end of queue (EOQ) condition flag on the updated packet descriptor when it is returned by the EMAC.

24.11.4.11.1.2 CPPI TX Data Word 1

Buffer Pointer

The byte aligned memory address of the buffer associated with the buffer descriptor. The host sets the `buffer_pointer`. The software application must set this value prior to adding the descriptor to the active transmit list. This pointer is not altered by the EMAC.

24.11.4.11.1.3 CPPI TX Data Word 2

Buffer Offset

Indicates how many unused bytes are at the start of the buffer. A value of 0000h indicates that no unused bytes are at the start of the buffer and that valid data begins on the first byte of the buffer. A value of 000Fh (decimal 15) indicates that the first 15 bytes of the buffer are to be ignored by the port and that valid buffer data starts on byte 16 of the buffer. The host sets the `Buffer_Offset` value (which may be zero to the buffer length minus 1). Valid only on SOP.

Buffer Length

Indicates how many valid data bytes are in the buffer. Unused or protocol specific bytes at the beginning of the buffer are not counted in the `Buffer Length` field. The host sets the `Buffer_Length`. The `Buffer_Length` must be greater than zero.

24.11.4.11.1.4 CPPI TX Data Word 3

Packet Length

Specifies the number of bytes in the entire packet. Offset bytes are not included. The sum of the `Buffer_Length` fields should equal the `Packet_Length`. Valid only on SOP. The packet length must be greater than zero. The packet data will be truncated to the packet length if the packet length is shorter than the sum of the packet buffer descriptor buffer lengths. A host error occurs if the packet length is greater than the sum of the packet buffer descriptor buffer lengths.

Start of Packet (SOP) Flag

When set, this flag indicates that the descriptor points to a packet buffer that is the start of a new packet. In the case of a single fragment packet, both the SOP and end of packet (EOP) flags are set. Otherwise, the descriptor pointing to the last packet buffer for the packet sets the EOP flag. This bit is set by the software application and is not altered by the EMAC.

0 - Not start of packet buffer.

1 - Start of packet buffer.

End of Packet (EOP) Flag

When set, this flag indicates that the descriptor points to a packet buffer that is last for a given packet. In the case of a single fragment packet, both the start of packet (SOP) and EOP flags are set. Otherwise, the descriptor pointing to the last packet buffer for the packet sets the EOP flag. This bit is set by the software application and is not altered by the EMAC.

0 - Not end of packet buffer.

1 - End of packet buffer.

Ownership (OWNER) Flag

When set this flag indicates that all the descriptors for the given packet (from SOP to EOP) are currently owned by the EMAC. This flag is set by the software application on the SOP packet descriptor before adding the descriptor to the transmit descriptor queue. For a single fragment packet, the SOP, EOP, and OWNER flags are all set. The OWNER flag is cleared by the EMAC once it is finished with all the descriptors for the given packet. Note that this flag is valid on SOP descriptors only.

0 - The packet is owned by the host

1 - The packet is owned by the port

End of Queue (EOQ) Flag

When set, this flag indicates that the descriptor in question was the last descriptor in the transmit queue for a given transmit channel, and that the transmitter has halted. This flag is initially cleared by the software application prior to adding the descriptor to the transmit queue. This bit is set by the EMAC when the EMAC identifies that a descriptor is the last for a given packet (the EOP flag is set), and there are no more descriptors in the transmit list (next descriptor pointer is NULL).

The software application can use this bit to detect when the EMAC transmitter for the corresponding channel has halted. This is useful when the application appends additional packet descriptors to a transmit queue list that is already owned by the EMAC. Note that this flag is valid on EOP descriptors only.

0 - The TX queue has more packets to transfer.

1 - The Descriptor buffer is the last buffer in the last packet in the queue.

Teardown Complete (TDOWNCMPLT) Flag

This flag is used when a transmit queue is being torn down, or aborted, instead of allowing it to be transmitted. This would happen under device driver reset or shutdown conditions. The EMAC sets this bit in the SOP descriptor of each packet as it is aborted from transmission. Note that this flag is valid on SOP descriptors only. Also note that only the first packet in an unsent list has the TDOWNCMPLT flag set. Subsequent descriptors are not processed by the EMAC.

0 - The port has not completed the teardown process.

1 - The port has completed the commanded teardown process.

Pass CRC (PASSCRC) Flag

This flag is set by the software application in the SOP packet descriptor before it adds the descriptor to the transmit queue. Setting this bit indicates to the EMAC that the 4 byte Ethernet CRC is already present in the packet data, and that the EMAC should not generate its own version of the CRC. When the CRC flag is cleared, the EMAC generates and appends the 4-byte CRC. The buffer length and packet length fields do not include the CRC bytes. When the CRC flag is set, the 4-byte CRC is supplied by the software application and is already appended to the end of the packet data. The buffer length and packet length fields include the CRC bytes, as they are part of the valid packet data. Note that this flag is valid on SOP descriptors only.

0 - The CRC is not included with the packet data and packet length.

1 - The CRC is included with the packet data and packet length.

TO_PORT

Port number to send the directed packet to. This field is set by the host. This field is valid on SOP. Directed packets go to the directed port, but an ALE lookup is performed to determine untagged egress in VLAN_AWARE mode.

1 - Send the packet to port 1 if TO_PORT_EN is asserted.

2 - Send the packet to port 2 if TO_PORT_EN is asserted.

TO_PORT_ENABLE

Indicates when set that the packet is a directed packet to be sent to the TO_PORT field port number. This field is set by the host. The packet is sent to one port only (index not mask). This bit is valid on SOP.

0 - not a directed packet

1 - directed packet

24.11.4.11.2 RX Buffer Descriptors

A RX buffer descriptor ([Table 24-1299](#)) is a contiguous block of four 32-bit data words aligned on a 32-bit word boundary.

Table 24-1299. RX Buffer Descriptor Format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Next Descriptor Pointer																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Buffer Pointer																															

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED								Buffer Offset								RESERVED								Buffer Length							

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
SOP	EOP	OWNER	EOQ	TDOWNCMPLT	PASSCRC	LONG	SHORT	MAC_CTL	OVERRUN	PKT_ERR	RX_VLAN_ENCAP	FROM_PORT	RESERVED								Packet Length											

24.11.4.11.2.1 CPPI RX Data Word 0

Next Descriptor Pointer

The 32-bit word aligned memory address of the next buffer descriptor in the RX queue. This is the mechanism used to reference the next buffer descriptor from the current buffer descriptor. If the value of this pointer is zero then the current buffer is the last buffer in the queue. The host sets the Next_Descriptor_Pointer.

24.11.4.11.2.2 CPPI RX Data Word 1

Buffer Pointer

The byte aligned memory address of the buffer associated with the buffer descriptor. The host sets the Buffer_Pointer.

24.11.4.11.2.3 CPPI RX Data Word 2

Buffer Offset

Indicates how many unused bytes are at the start of the buffer. A value of 0000h indicates that there are no unused bytes at the start of the buffer and that valid data begins on the first byte of the buffer. A value of 000Fh (decimal 15) indicates that the first 15 bytes of the buffer are to be ignored by the port and that valid buffer data starts on byte 16 of the buffer. The port writes the Buffer_Offset with the value from the [CPDMA_RX_BUFFER_OFFSET](#) register value. The host initializes the Buffer_Offset to zero for free buffers. The Buffer_Length must be greater than the CPDMARX_BUFFER_OFFSET register value. The buffer offset is valid only on SOP.

Buffer Length

Indicates how many valid data bytes are in the buffer. Unused or protocol specific bytes at the beginning of the buffer are not counted in the Buffer_Length field. The host initializes the Buffer_Length, but the port may overwrite the host initiated value with the actual buffer length value on SOP and/or EOP buffer descriptors. SOP buffer length values will be overwritten if the packet size is less than the size of the buffer or if the offset is nonzero. EOP buffer length values will be overwritten if the entire buffer is not filled up with data. The Buffer_Length must be greater than zero.

24.11.4.11.2.4 CPPI RX Data Word 3

Packet Length

Specifies the number of bytes in the entire packet. Offset bytes are not included. The sum of the Buffer_Length fields should equal the Packet_Length. Valid only on SOP.

Start of Packet (SOP) Flag

When set, this flag indicates that the descriptor points to a packet buffer that is the start of a new packet. In the case of a single fragment packet, both the SOP and end of packet (EOP) flags are set. Otherwise, the descriptor pointing to the last packet buffer for the packet has the EOP flag set. This flag is initially cleared by the software application before adding the descriptor to the receive queue. This bit is set by the EMAC on SOP descriptors.

End of Packet (EOP) Flag

When set, this flag indicates that the descriptor points to a packet buffer that is last for a given packet. In the case of a single fragment packet, both the start of packet (SOP) and EOP flags are set. Otherwise, the descriptor pointing to the last packet buffer for the packet has the EOP flag set. This flag is initially cleared by the software application before adding the descriptor to the receive queue. This bit is set by the EMAC on EOP descriptors.

Ownership (OWNER) Flag

When set, this flag indicates that the descriptor is currently owned by the EMAC. This flag is set by the software application before adding the descriptor to the receive descriptor queue. This flag is cleared by the EMAC once it is finished with a given set of descriptors, associated with a received packet. The flag is updated by the EMAC on SOP descriptor only. So when the application identifies that the OWNER flag is cleared on an SOP descriptor, it may assume that all descriptors up to and including the first with the EOP flag set have been released by the EMAC. (Note that in the case of single buffer packets, the same descriptor will have both the SOP and EOP flags set.)

End of Queue (EOQ) Flag

When set, this flag indicates that the descriptor in question was the last descriptor in the receive queue for a given receive channel, and that the corresponding receiver channel has halted. This flag is initially cleared by the software application prior to adding the descriptor to the receive queue. This bit is set by the EMAC when the EMAC identifies that a descriptor is the last for a given packet received (also sets the EOP flag), and there are no more descriptors in the receive list (next descriptor pointer is NULL). The software application can use this bit to detect when the EMAC receiver for the corresponding channel has halted. This is useful when the application appends additional free buffer descriptors to an active receive queue. Note that this flag is valid on EOP descriptors only.

Teardown Complete (TDOWNCMPLT) Flag

This flag is used when a receive queue is being torn down, or aborted, instead of being filled with received data. This would happen under device driver reset or shutdown conditions. The EMAC sets this bit in the descriptor of the first free buffer when the tear down occurs. No additional queue processing is performed.

Pass CRC (PASSCRC) Flag

This flag is set by the EMAC in the SOP buffer descriptor if the received packet includes the 4-byte CRC. This flag should be cleared by the software application before submitting the descriptor to the receive queue.

Long (Jabber) Flag

This flag is set by the EMAC in the SOP buffer descriptor, if the received packet is a jabber frame and was not discarded because the RX_CEF_EN bit was set in the [SL_MACCONTROL](#) register. Jabber frames are frames that exceed the RXMAXLEN in length, and have CRC, code, or alignment errors.

Short (Fragment) Flag

This flag is set by the EMAC in the SOP buffer descriptor, if the received packet is only a packet fragment and was not discarded because the RX_CSF_EN bit was set in the [SL_MACCONTROL](#) register.

Control Flag

This flag is set by the EMAC in the SOP buffer descriptor, if the received packet is an EMAC control frame and was not discarded because the RX_CMF_EN bit was set in the [SL_MACCONTROL](#) register.

Overrun Flag

This flag is set by the EMAC in the SOP buffer descriptor, if the received packet was aborted due to a receive overrun.

Packet Error (PKT_ERR) Flag

Packet Contained Error on Ingress:

00 - no error

01 - CRC error on ingress

10 - Code error on ingress

11 - Align error on ingress

VLAN Encapsulated Packet (RX_VLAN_ENCAP)

Indicates when set that the packet data contains a 32-bit VLAN header word that is included in the packet byte count. This field is set by the port to be the value of the [CPSW_CONTROL](#) register RX_VLAN_ENCAP bit.

FROM_PORT

Indicates the port number that the packet was received on (ingress to the switch).

24.11.4.12 MDIO

The MII Management interface module implements the 802.3 serial management interface to interrogate and control two Ethernet PHYs simultaneously using a shared two-wire bus. Two user access registers ([MDIO_USERACCESS0](#)/[MDIO_USERACCESS1](#)) control and monitor up to two PHYs simultaneously.

24.11.4.12.1 MDIO Frame Formats

[Table 24-1300](#) shows the read format and [Table 24-1301](#) shows the write format of the 32-bit MII Management interface frames.

Table 24-1300. MDIO Read Frame Format

Pre-amble	Start Delimiter	Operation Code	PHY Address	Register Address	Turnaround	Data
FFFF FFFFh	01	10	AAAAA	RRRRR	Z0	DDDD.DDDD.DDDD.DDDD

Table 24-1301. MDIO Write Frame Format

Pre-amble	Start Delimiter	Operation Code	PHY Address	Register Address	Turnaround	Data
FFFF FFFFh	01	01	AAAAA	RRRRR	10	DDDD.DDDD.DDDD.DDDD

The default or idle state of the two wire serial interface is a logic one. All tri-state drivers should be disabled and the PHY's pull-up resistor will pull the MDIO line to a logic 1. Prior to initiating any other transaction, the station management entity shall send a preamble sequence of 32 contiguous logic 1 bits on the MDIO line with 32 corresponding cycles on MDCLK to provide the PHY with a pattern that it can use to establish synchronization. A PHY shall observe a sequence of 32 contiguous logic one bits on MDIO with 32 corresponding MDCLK cycles before it responds to any other transaction.

Preamble

The start of a frame is indicated by a preamble, which consists of a sequence of 32 contiguous bits all of which are a 1. This sequence provides the PHY a pattern to use to establish synchronization.

Start Delimiter

The preamble is followed by the start delimiter which is indicated by a 01 pattern. The pattern assures transitions from the default logic 1 state to logic 0, and back to logic 1.

Operation Code

The operation code for a read is 10, while the operation code for a write is a 01.

PHY Address

The PHY address is 5 bits allowing 32 unique values. The first bit transmitted is the MSB of the PHY address.

Register Address

The Register address is 5 bits allowing 32 registers to be addressed within each PHY. Refer to the 10/100 PHY address map for addresses of individual registers.

Turnaround

An idle bit time during which no device actively drives the MDIO signal shall be inserted between the register address field and the data field of a read frame in order to avoid contention. During a read frame, the PHY shall drive a zero bit onto MDIO for the first bit time following the idle bit and preceding the Data field. During a write frame, this field shall consist of a one bit followed by a zero bit.

Data

The Data field is 16 bits. The first bit transmitted and received is the MSB of the data word.

24.11.4.12.2 MDIO Functional Description

The MII Management I/F will remain idle until enabled by setting the ENABLE bit in the [MDIO_CONTROL](#) register. The MII Management I/F will then continuously poll the link status from within the Generic Status Register of all possible 32 PHY addresses in turn recording the results in the [MDIO_LINK](#) register. The LINKSEL bit in the [MDIO_USERPHYSEL0/1](#) register determines the status input that is used. A change in the link status of the two PHYs being monitored will set the appropriate bit in the [MDIO_LINKINTRAW](#) register and the [MDIO_LINKINTMASKED](#) register, if enabled by the LINKINT_ENABLE bit in the [MDIO_USERPHYSEL0/1](#) register.

The [MDIO_ALIVE](#) register is updated by the MII Management I/F module if the PHY acknowledged the read of the generic status register. In addition, any PHY register read transactions initiated by the host also cause the [MDIO_ALIVE](#) register to be updated.

At any time, the host can define a transaction for the MII Management interface module to undertake using the DATA, PHYADR, REGADR, and WRITE fields in a [MDIO_USERACCESS0/1](#) register. When the host sets the GO bit in this register, the MII Management interface module will begin the transaction without any further intervention from the host. Upon completion, the MII Management interface will clear the GO bit and set the USERINTRAW bit in the [MDIO_USERINTRAW](#) register corresponding to the [MDIO_USERACCESS0/1](#) register being used. The corresponding bit in the [MDIO_USERINTMASKED](#) register may also be set depending on the mask setting in the [MDIO_USERINTMASKSET](#) and [MDIO_USERINTMASKCLR](#) registers. A round-robin arbitration scheme is used to schedule transactions that may be queued by the host in different [MDIO_USERACCESS0/1](#) registers. The host should check the status of the GO bit in the [MDIO_USERACCESS0/1](#) register before initiating a new transaction to ensure that the previous transaction has completed. The host can use the ACK bit in the [MDIO_USERACCESS0/1](#) register to determine the status of a read transaction.

It is necessary for software to use the MII Management interface module to setup the auto-negotiation parameters of each PHY attached to a MAC port, retrieve the negotiation results, and setup the [SL_MACCONTROL](#) register in the corresponding MAC.

24.11.5 GMAC_SW Programming Guide

24.11.5.1 Transmit Operation

After reset, the host must write zeroes to all TX DMA State head descriptor pointers. The TX port may then be enabled. To initiate packet transmission the host constructs transmit queues in memory (one or more packets for transmission) and then writes the appropriate TX DMA state head descriptor pointers. For each buffer added to a transmit queue, the host must initialize the TX buffer descriptor values as follows:

1. Write the Next Descriptor Pointer with the 32-bit aligned address of the next descriptor in the queue (zero if last descriptor)
2. Write the Buffer Pointer with the byte aligned address of the buffer data
3. Write the Buffer Length with the number of bytes in the buffer
4. Write the Buffer Offset with the number of bytes in the offset to the data (nonzero with SOP only)
5. Set the SOP, EOP, and Ownership bits as appropriate
6. Clear the End Of Queue bit

The port begins TX packet transmission on a given channel when the host writes the channel's TX queue head descriptor pointer with the address of the first buffer descriptor in the queue (nonzero value). Each channel may have one or more queues, so each channel may have one or more head descriptor pointers. The first buffer descriptor for each TX packet must have the Start of Packet (SOP) bit and the Ownership bit set to one by the host. The last buffer descriptor for each TX packet must have the End of Packet (EOP) bit set to one by the host. The port will transmit packets until all queued packets have been transmitted and the queue(s) are empty. When each packet transmission is complete, the port will clear the Ownership bit in the packet's SOP buffer descriptor and issue an interrupt to the host by writing the packet's last buffer descriptor address to the queue's TX DMA State Completion Pointer. The interrupt is generated by the write, regardless of the value written. When the last packet in a queue has been transmitted, the port sets the End Of Queue bit in the EOP buffer descriptor, clears the Ownership bit in the SOP Descriptor, zeroes the appropriate DMA state head descriptor pointer, and then issues a TX interrupt to the host by writing to the queue's associated TX completion pointer (address of the last buffer descriptor processed by the port). The port issues a maskable level interrupt (which may then be routed through external interrupt control logic to the host).

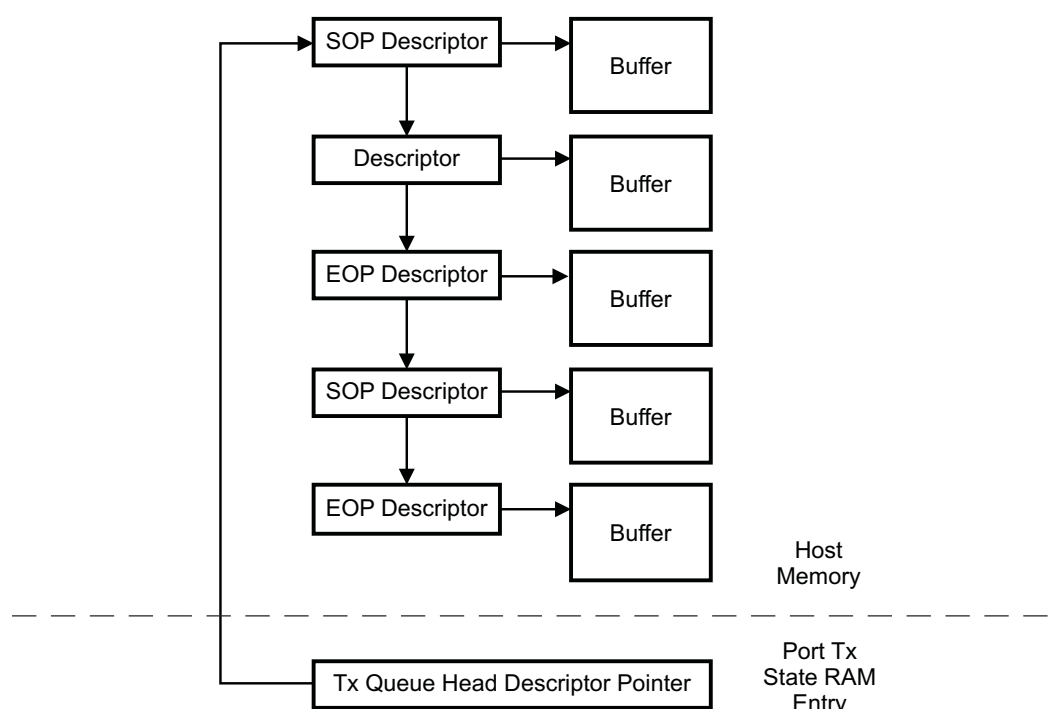
On interrupt from the port, the host processes the buffer queue, detecting transmitted packets by the status of the Ownership bit in the SOP buffer descriptor. If the Ownership bit is cleared to zero, then the packet has been transmitted and the host may reclaim the buffers associated with the packet. The host continues queue processing until the end of the queue or until a SOP buffer descriptor is read that contains a set Ownership bit indicating that the packet transmission is not complete. The host determines that all packets in the queue have been transmitted when the last packet in the queue has a cleared Ownership bit in the SOP buffer descriptor, the End of Queue bit is set in the last packet EOP buffer descriptor, and the Next Descriptor Pointer of the last packet EOP buffer descriptor is zero. The host acknowledges an interrupt by writing the address of the last buffer descriptor to the queue's associated TX Completion Pointer in the TX DMA State. If the host written buffer address value is different from the buffer address written by the port, then the level interrupt remains asserted. If the host written buffer address value is equal to the port written value, then the level interrupt is de-asserted. The port write to the completion pointer actually stores the value in the state register (RAM). The host written value is actually not written to the register location. The host written value is compared to the register contents (which was written by the port) and if the two values are equal, the interrupt is removed, otherwise the interrupt remains asserted. The host may process multiple packets previous to acknowledging an interrupt, or the host may acknowledge interrupts for every packet.

A mis-queued packet condition may occur when the host adds a packet to a queue for transmission as the port finishes transmitting the previous last packet in the queue. The mis-queued packet is detected by the host when queue processing detects a cleared Ownership bit in the SOP buffer descriptor, a set End of Queue bit in the EOP buffer descriptor, and a nonzero Next Descriptor Pointer in the EOP buffer descriptor. A mis-queued packet means that the port read the last EOP buffer descriptor before the host added the new last packet to the queue, so the port determined queue empty just before the last packet was added. The host corrects the mis-queued packet condition by initiating a new packet transfer for the mis-queued packet by writing the mis-queued packet's SOP buffer descriptor address to the appropriate DMA State TX Queue head Descriptor Pointer.

The host may add packets to the tail end of an active TX queue at any time by writing the Next Descriptor Pointer to the current last descriptor in the queue. If a TX queue is empty (inactive), the host may initiate packet transmission at any time by writing the appropriate TX DMA State head descriptor pointer. The host software should always check for and reinitiate transmission for mis-queued packets during queue processing on interrupt from the port. In order to preclude software underrun, the host should avoid adding buffers to an active queue for any TX packet that is not complete and ready for transmission.

The port determines that a packet is the last packet in the queue by detecting the End of Packet bit set with a zero Next Descriptor Pointer in the packet buffer descriptor. If the End of Packet bit is set and the Next Descriptor Pointer is nonzero, then the queue still contains one or more packets to be transmitted. If the EOP bit is set with a zero Next Descriptor Pointer, then the port will set the EOQ bit in the packet's EOP buffer descriptor and then zero the appropriate head descriptor pointer previous to interrupting the port (by writing the completion pointer) when the packet transmission is complete.

Figure 24-210. TX Queue Head Descriptor



24.11.5.2 Receive Operation

After reset, the host must write zeroes to all RX DMA State head descriptor pointers. The RX port may then be enabled. To initiate packet reception, the host constructs receive queues in memory and then writes the appropriate RX DMA state head descriptor pointer. For each RX buffer descriptor added to the queue, the host must initialize the RX buffer descriptor values as follows:

1. Write the Next Descriptor Pointer with the 32-bit aligned address of the next descriptor in the queue (zero if last descriptor)
2. Write the Buffer Pointer with the byte aligned address of the buffer data
3. Clear the Offset field
4. Write the Buffer Length with the number of bytes in the buffer
5. Clear the SOP, EOP, and EOQ bits
6. Set the Ownership bit

The host enables packet reception on a given channel by writing the address of the first buffer descriptor in the queue (nonzero value) to the channel's head descriptor pointer in the channel's RX DMA state. When packet reception begins on a given channel, the port fills each RX buffer with data in order starting with the first buffer and proceeding through the RX queue. If the Buffer Offset in the RX DMA State is nonzero, then the port will begin writing data after the offset number of bytes in the SOP buffer. The port performs the following operations at the end of each packet reception:

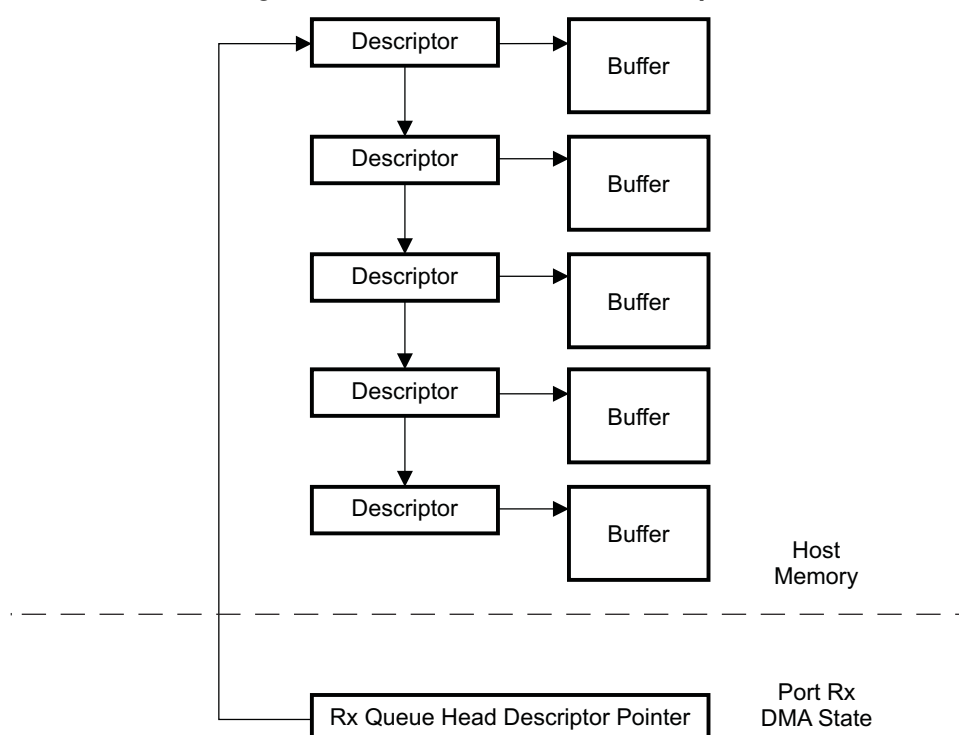
1. Overwrite the buffer length in the packet's EOP buffer descriptor with the number of bytes actually received in the packet's last buffer. The host initialized value is the buffer size. The overwritten value

will be less than or equal to the host initialized value.

2. Set the EOP bit in the packet's EOP buffer descriptor.
3. Set the EOQ bit in the packet's EOP buffer descriptor if the current packet is the last packet in the queue.
4. Overwrite the packet's SOP buffer descriptor Buffer Offset with the RX DMA state value (the host initialized the buffer descriptor Buffer Offset value to zero). All non SOP buffer descriptors must have a zero Buffer Offset initialized by the host.
5. Overwrite the packet's SOP buffer descriptor buffer length with the number of valid data bytes in the buffer. If the buffer is filled up, the buffer length will be the buffer size minus buffer offset.
6. Set the SOP bit in the packet's SOP buffer descriptor.
7. Write the SOP buffer descriptor Packet Length field.
8. Clear the Ownership bit in the packet's SOP buffer descriptor.
9. Issue an RX host interrupt by writing the address of the packet's last buffer descriptor to the queue's RX DMA State Completion Pointer. The interrupt is generated by the write to the RX DMA State Completion Pointer address location, regardless of the value written.

On interrupt the host processes the RX buffer queue detecting received packets by the status of the Ownership bit in each packet's SOP buffer descriptor. If the Ownership bit is cleared then the packet has been completely received and is available to be processed by the host. The host may continue RX queue processing until the end of the queue or until a buffer descriptor is read that contains a set Ownership bit indicating that the next packet's reception is not complete. The host determines that the RX queue is empty when the last packet in the queue has a cleared Ownership bit in the SOP buffer descriptor, a set End of Queue bit in the EOP buffer descriptor, and the Next Descriptor Pointer in the EOP buffer descriptor is zero.

A mis-queued buffer may occur when the host adds buffers to a queue as the port finishes the reception of the previous last packet in the queue. The mis-queued buffer is detected by the host when queue processing detects a cleared Ownership bit in the SOP buffer descriptor, a set End of Queue bit in the EOP buffer descriptor, and a nonzero Next Descriptor Pointer in the EOP buffer descriptor. A mis-queued buffer means that the port read the last EOP buffer descriptor before the host added buffer descriptor(s) to the queue, so the port determined queue empty just before the host added more buffer descriptor(s). In the transmit case, the packet transmission is delayed by the time required for the host to determine the condition and reinitiate the transaction, but the packet is not actually lost. In the receive case, receive overrun condition may occur in the mis-queued buffer case. If a new packet reception is begun during the time that the port has determined the end of queue condition, then the received packet will overrun (start of packet overrun). If the mis-queued buffer occurs during the middle of a packet reception then middle of packet overrun may occur. If the mis-queued buffer occurs after the last packet has completed, and is corrected before the next packet reception begins, then overrun will not occur. The host acts on the mis-queued buffer condition by writing the added buffer descriptor address to the appropriate RX DMA State Head Descriptor Pointer.

Figure 24-211. RX Queue Head Descriptor


24.11.5.3 MDIO Software Interface

24.11.5.3.1 Initializing the MDIO Module

The following steps are performed by the application software or device driver to initialize the MDIO device:

1. Configure the PREAMBLE and CLKDIV bits in the MDIO Control register ([MDIO_CONTROL](#)).
2. Enable the MDIO module by setting the ENABLE bit in [MDIO_CONTROL](#).
3. The MDIO PHY alive status register ([MDIO_ALIVE](#)) can be read in polling fashion until a PHY connected to the system responded, and the MDIO PHY link status register ([MDIO_LINK](#)) can determine whether this PHY already has a link.
4. Setup the appropriate PHY addresses in the MDIO user PHY select register ([MDIO_USERPHYSEL0/1](#)), and set the LINKINTENB bit to enable a link change event interrupt if desirable.
5. If an interrupt on general MDIO register access is desired, set the corresponding bit in the MDIO user command complete interrupt mask set register ([MDIO_USERINTMASKSET](#)) to use the MDIO user access register ([MDIO_USERACCESS0/1](#)). Since only one PHY is used in this device, the application software can use one [MDIO_USERACCESS0/1](#) to trigger a completion interrupt; the other [MDIO_USERACCESS0/1](#) is not setup.

24.11.5.3.2 Writing Data To a PHY Register

The MDIO module includes a user access register ([MDIO_USERACCESS0/1](#)) to directly access a specified PHY device. To write a PHY register, perform the following:

1. Check to ensure that the GO bit in the MDIO user access register ([MDIO_USERACCESS0/1](#)) is cleared.
2. Write to the GO, WRITE, REGADR, PHYADR, and DATA bits in [MDIO_USERACCESS0/1](#) corresponding to the PHY and PHY register you want to write.

3. The write operation to the PHY is scheduled and completed by the MDIO module. Completion of the write operation can be determined by polling the GO bit in [MDIO_USERACCESS0/1](#) for a 0.
4. Completion of the operation sets the corresponding USERINTRAW bit (0 or 1) in the MDIO user command complete interrupt register ([MDIO_USERINTRAW](#)) corresponding to [MDIO_USERACCESS0/1](#) used. If interrupts have been enabled on this bit using the MDIO user command complete interrupt mask set register ([MDIO_USERINTMASKSET](#)), then the bit is also set in the MDIO user command complete interrupt register ([MDIO_USERINTMASKED](#)) and an interrupt is triggered on the host processor.

24.11.5.3.3 Reading Data From a PHY Register

The MDIO module includes a user access register ([MDIO_USERACCESS0/1](#)) to directly access a specified PHY device. To read a PHY register, perform the following:

1. Check to ensure that the GO bit in the MDIO user access register ([MDIO_USERACCESSn](#)) is cleared.
2. Write to the GO, REGADR, and PHYADR bits in [MDIO_USERACCESS0/1](#) corresponding to the PHY and PHY register you want to read.
3. The read data value is available in the DATA bits in [MDIO_USERACCESS0/1](#) after the module completes the read operation on the serial bus. Completion of the read operation can be determined by polling the GO and ACK bits in [MDIO_USERACCESS0/1](#). After the GO bit has cleared, the ACK bit is set on a successful read.
4. Completion of the operation sets the corresponding USERINTRAW bit (0 or 1) in the MDIO user command complete interrupt register ([MDIO_USERINTRAW](#)) corresponding to [MDIO_USERACCESS0/1](#) used. If interrupts have been enabled on this bit using the MDIO user command complete interrupt mask set register ([MDIO_USERINTMASKSET](#)), then the bit is also set in the MDIO user command complete interrupt register ([MDIO_USERINTMASKED](#)) and an interrupt is triggered on the host processor.

24.11.5.4 Initialization and Configuration of CPSW

To configure the GMAC_SW Ethernet Subsystem for operation, the host must perform the following:

1. Select the Interface (G/MII, RGMII, RMII) Mode
2. Configure pads (PIN muxing), as per the interface selected.
3. Enable the GMAC_SW Ethernet Subsystem Clocks
4. Configure the PRCM registers CM_GMAC_CLKSTCTRL and CM_GMAC_GMAC_CLKCTRL to enable power and clocks to GMAC_SW Ethernet Subsystem.
5. Apply Soft Reset to GMAC_SW Subsystem, CPSW_3G, CPGMAC_SL1/2, and CPDMA
6. Initialize the HDPs (Header Description Pointer) and CPs (Completion Pointer) to NULL
7. Configure the Interrupts
8. Configure the [CPSW_CONTROL](#) register
9. Configure the [CPSW_STAT_PORT_EN](#) register
10. Configure the ALE
11. Configure the MDIO
12. Configure the CPDMA receive DMA controller
13. Configure the CPDMA transmit DMA controller
14. Configure the CPPI TX and RX Descriptors
15. Configure CPGMAC_SL1 and CPGMAC_SL2, as per the desired mode of operations.
16. Start up RX and TX DMA (Write to HDP of RX and TX)
17. Wait for the completion of Transfer (HDP cleared to 0)

24.11.6 GMAC_SW Register Manual

24.11.6.1 GMAC_SW Instance Summary

Table 24-1302. GMAC_SW Instance Summary

Module Name	Module Base Address	Size
SS	0x4848 4000	80 Bytes
PORT	0x4848 4100	1792 Bytes
CPDMA	0x4848 4800	256 Bytes
STATS	0x4848 4900	128 Bytes
STATERAM	0x4848 4A00	288 Bytes
CPTS	0x4848 4C00	240 Bytes
ALE	0x4848 4D00	88 Bytes
SL1	0x4848 4D80	64 Bytes
SL2	0x4848 4DC0	64 Bytes
MDIO	0x4848 5000	192 Bytes
WR	0x4848 5200	352 Bytes
SPF1	0x4848 5C00	512 Bytes
SPF2	0x4848 5E00	512 Bytes

NOTE: CPPI RAM address space starts at 0x4848 6000 and is 8 KiB deep. For details about CPPI RAM, see [Section 24.11.4.11](#), *CPPI Buffer Descriptors*.

24.11.6.2 SS Registers

24.11.6.2.1 SS Register Summary

Table 24-1303. SS Registers Mapping Summary

Register Name	Type	Register Width (Bits)	Address Offset	SS Physical Address
CPSW_ID_VER	R	32	0x0000 0000	0x4848 4000
CPSW_CONTROL	RW	32	0x0000 0004	0x4848 4004
CPSW_SOFT_RESET	RW	32	0x0000 0008	0x4848 4008
CPSW_STAT_PORT_EN	RW	32	0x0000 000C	0x4848 400C
CPSW_PTYPE	RW	32	0x0000 0010	0x4848 4010
CPSW_SOFT_IDLE	RW	32	0x0000 0014	0x4848 4014
CPSW_THRU_RATE	RW	32	0x0000 0018	0x4848 4018
CPSW_GAP_THRESH	RW	32	0x0000 001C	0x4848 401C
CPSW_TX_START_WDS	RW	32	0x0000 0020	0x4848 4020
CPSW_FLOW_CONTROL	RW	32	0x0000 0024	0x4848 4024
CPSW_VLAN_LTYPE	RW	32	0x0000 0028	0x4848 4028
CPSW_TS_LTYPE	RW	32	0x0000 002C	0x4848 402C
CPSW_DLR_LTYPE	RW	32	0x0000 0030	0x4848 4030
CPSW_EEE_PRESCALE	RW	32	0x0000 0034	0x4848 4034